nook

Complete Product & Engineering Handbook

The hostel and PG management platform — product strategy, experience design, spatial inventory, financial recordkeeping, architecture and delivery.

Version	**v1.0**
Date	28 October 2026
Status	DRAFT FOR REVIEW
Prepared for	Project-owner review
Pilot	One partnership-operated ladies' hostel, Pune, Maharashtra (~100 beds)

This handbook describes the intended product. It is not a statement that the product exists. Chapter 23 sets out, separately and factually, what is present in the repository today — a Milestone 0 engineering scaffold with no resident, booking, rent, payment or deposit functionality. Nothing here records an approval, a signature or a test result that has not occurred.

"nook" is a working product name. Trademark clearance has not been performed, the Nook / Nooks competitive conflict is unresolved and no domain is secured. Public brand launch remains blocked pending that review.

CHAPTER 1

Document Information and How to Read This Handbook

What this document is, what it is built from, the status vocabulary it uses throughout, and the distinction between the product described here and the code that exists today.

Purpose

This handbook explains Nook in enough detail that a new engineer, designer or reviewer can understand what is being built, why, how each module is intended to work, how the architecture fits together, what remains undecided, and how implementation is expected to proceed.

It serves several readers at once. A founder or reviewer should be able to read Parts I and VI and understand the product and its risks. An engineer joining the team should read Parts III and V. A designer should read Part II. A property owner evaluating the pilot should read Chapters 2, 9, 11 and 24.

The single most important distinction in this document

INTENDED PRODUCT VERSUS CURRENT IMPLEMENTATION

Most of this handbook describes a **proposed** product. The repository today contains a Milestone 0 scaffold: an API shell, a web shell, design tokens, a read-only demonstration UI with fictional fixtures, an intentionally empty database schema, and validation tooling. There is no resident record, no booking, no ledger, no authentication and no database table.

Every module chapter therefore carries a status line. Chapter 23 gives the complete factual inventory with command exit codes. Where this handbook and the repository disagree, the repository is the fact and this handbook is the intent.

Status vocabulary

These nine labels are used consistently and are never interchangeable. A reader should be able to rely on them without re-reading the surrounding prose.

STATUS	MEANING	WHO CAN CHANGE IT
CONFIRMED	Established by the project brief or by direct inspection of a source record. Not an approval.	Evidence
APPROVED IN PRINCIPLE	Accepted in direction by project review; specifics may remain open. Does not authorise implementation on its own.	Project owner
PROPOSED	A recommendation awaiting a decision. The default state of most of this handbook.	Project owner
OPEN	No decision exists. Never a silent default; the dependent feature stays disabled.	Project owner or named professional

STATUS	MEANING	WHO CAN CHANGE IT
BLOCKED	Cannot proceed until a named external condition clears.	The named condition
IMPLEMENTED	Code exists in the repository.	Engineering
VERIFIED	Implemented *and* demonstrated by an executed check with a recorded result.	Executed evidence
NOT IMPLEMENTED	No code exists. The normal state for nearly every module in this handbook.	Engineering
SUPERSEDED	Replaced by a later decision, retained for traceability.	Project owner

Implemented is not Verified. Code that compiles has not been shown to work. This distinction is load-bearing in Chapter 23, where several components compile and build cleanly but have never been executed against a running database.

Source material this handbook was built from

Every factual claim about the codebase, the brand or the agreed requirements traces to one of these. Chapter 25 carries the full inventory with checksums where available.

SOURCE	STATE	USED FOR
`/Users/atharva/Downloads/nook`	INSPECTED	Implementation status, toolchain versions, design tokens, demo UI behaviour
`docs/MILESTONE_0_VALIDATION.md`	INSPECTED	Executed check results, exit codes and acceptance blockers in Chapter 23
`nook-brand-assets.zip`	APPROVED	All logo assets in this publication; colour palette; clear-space and minimum-size rules
`nook-complete.zip`	INSPECTED	Token system provenance, Option A/B palette decision, competitive colour research
Requirements package v0.2 (16 documents)	PROPOSED	Business rules, architecture decisions, permission vocabulary, test classification, blocker register

> **IDENTIFIER CONTINUITY**
>
> The requirements package was written under the project's earlier working name. Its identifier families carry forward unchanged into this handbook: `BR-*` business rules, `ADR-*` architecture decisions, `RG-*` release gates, `M0-*` Milestone 0 tasks and `FIN-*` financial test scenarios. A reference such as `BR-REN-001` means the same rule in both documents.

What this handbook deliberately does not do

- It does not decide any business rule the property owner has not decided. Seventy-five rules remain open; Chapter 24 lists them and Chapter 11 explains what each one blocks.
- It does not record an approval. The architecture decision register and the owner sign-off checklist remain blank instruments, kept outside this handbook.
- It does not state a market size, a competitor's capability, a commercial figure or a legal position as settled fact.

- It does not fabricate a delivery date. Chapter 22 sequences work and names exit gates; it assigns no calendar commitments.
- It does not claim a security audit, a passing test suite or a restore that has not been executed and recorded.

CONTENTS

What is in this handbook

FRONT MATTER

1 Document Information and How to Read This Handbook 1

PART I · PRODUCT

2 Executive Summary 8
3 Product Vision and Strategy 13
4 Complete Feature and Module Map 19
5 Personas, Roles and User Journeys 25

PART II · EXPERIENCE

6 Product UI/UX and Design System 32
7 Building the Interactive Spatial Map 39

PART III · DOMAIN MODULES

8 Property, Building and Inventory Management 49
9 Booking, Admission and Allocation 53
10 Resident and Tenant Management 58
11 Financial Management 62
12 Operations and Property Services 68

PART IV · PLATFORM

13 Organisation Isolation and SaaS Administration 73
14 Notifications and External Integrations 76

PART V · ENGINEERING

15 Technical Architecture 80
16 Proposed Data Model and Data Dictionary 85
17 API Design and Contracts 88
18 Security, Privacy and Trust 91
19 Non-Functional Requirements 95
20 Testing and Quality Assurance 97
21 Deployment and Operations 101

PART VI · DELIVERY

22 Delivery Roadmap 106
23 Current Implementation Status 109

24 Risks, Assumptions and Open Decisions 113
25 Glossary, References and Appendices 117

PART I

Product

What Nook is, who it is for, the modules it is made of, and the journeys those modules have to support.

2 Executive Summary
3 Product Vision and Strategy
4 Complete Feature and Module Map
5 Personas, Roles and User Journeys

CHAPTER 2

Executive Summary

Nook in one chapter: the problem it solves, who uses it, what makes it different, how far it has actually been built, and the decisions that currently stand between the design and a working pilot.

What Nook is

Nook is a management platform for hostels, PGs and co-living properties. It holds one authoritative record of a property's physical inventory — buildings, floors, rooms and individual beds — and attaches to that inventory the people staying in them, the money they owe, and the money they have paid.

The platform is designed around a single organising idea: **a bed is a real place in a real building, not a row in a table.** Nook models the property spatially, shows it as an interactive floor plan, and uses that same spatial model for both the resident choosing where to live and the staff managing who lives where. One source of truth, two audiences.

Around that core sit the operational modules a property actually runs on: enquiries and admissions, resident records and lifecycle, a rent ledger with manual payment recording, security deposits, checkout, maintenance and complaints, staff roles and permissions, and reporting. Above it sits the multi-tenant layer that lets Nook serve many unrelated property businesses from one deployment.

The problem

The first customer — a partnership-operated ladies' hostel in Pune with roughly 100 beds and 91–100 current residents — runs on paper registers. CONFIRMED

That produces a specific, expensive failure: there is no single answer to "who owes money, and how much?". The answer currently lives in a paper register, a WhatsApp thread containing a UPI screenshot, and the owner's memory. These three disagree, and nobody can prove which is right. The consequences compound — residents who skipped a month and were never chased; deposit settlements argued in a doorway without an itemised history; hours of manual reconciliation every month; and, in a partnership, no auditable books for the partners to agree on.

> **WHAT THE FIRST CUSTOMER IS ACTUALLY BUYING**
>
> Not attendance tracking, not visitor logs, not a resident mobile app. The pilot succeeds if the owner can answer *"who owes what, and why?"* faster and more reliably in Nook than in the register — and if deposit disputes stop happening because every rupee has a traceable history.

Who uses it

USER	WHAT THEY NEED FROM NOOK
Owner / Partner	Occupancy and dues at a glance, financial reports both partners can agree on, control over who can do what

USER	WHAT THEY NEED FROM NOOK
Property Manager	Admissions, bed allocation, resident lifecycle, daily follow-up on arrears
Finance Clerk / Cashier	Record a payment correctly against the right charge, issue a receipt, reconcile cash and transfers
Warden / Front Desk	Who is in which bed, day-to-day operational tasks — with no financial or document access by default
Prospective resident	See what the property is actually like, which beds are free, and where they are in the building
Current resident	Their own statement, receipts and requests LATER PHASE

Differentiators

Nook competes in the same category as established hostel and PG software. Three things are intended to separate it, and the handbook is honest that only the first is genuinely distinctive.

DIFFERENTIATOR	WHY IT MATTERS	STATUS
Spatial bed selection	A resident picks a bed by seeing where it is — corner, window, near the stairwell — not by reading "Bed 203-C" from a dropdown. Staff manage occupancy on the same map. This is the product's distinguishing idea.	PROPOSED
Financial auditability	Every outstanding balance can be explained line by line, from first charge to last allocation. Posted records are never edited; corrections are reversals. This is table stakes done properly rather than a novel feature.	PROPOSED
Multi-tenancy from day one	Organisation isolation is in the design before the first customer, not retrofitted after. A one-customer pilot does not get a single-tenant schema.	APPROVED IN PRINCIPLE

Why the visual bed-selection experience matters

In a shared room, beds are not interchangeable. The bed by the window, the one next to the door, the upper bunk and the corner bed are materially different things to the person who will sleep there for a year. Every incumbent product treats them as identical rows in a list.

A floor plan collapses the explanation. A prospective resident understands the property in seconds rather than through a phone call. Staff stop mentally translating between a label and a location. And crucially, both sides are looking at the **same** data — the map a resident browses is generated from the inventory staff maintain, not a marketing illustration drawn separately and left to drift.

> **THE MAP IS A VIEW, NEVER THE AUTHORITY**
>
> A floor plan showing a bed as free is a *display*, computed at the moment the page rendered. It is not a reservation and not a guarantee. The backend re-verifies availability when an allocation is committed, inside a transaction, and rejects conflicting simultaneous allocations. Chapter 7 defines the three distinct states — displayed availability, user selection, and confirmed allocation — and Chapter 9 defines the atomic allocation that enforces them.

Property, resident and finance — how the three fit

Nook's modules are not a flat feature list; they form a dependency chain, and that chain determines build order.

Property management establishes what exists: the buildings, floors, rooms and beds, their labels, their positions and their operational state. Nothing else can be correct until this is. **Resident management** attaches a person and a stay to a bed for a period of time, and preserves that history when they move room or leave. **Finance** attaches money to the stay: recurring rent charges, payments recorded manually, allocations linking the two, deposits held separately as a liability, and a settlement at checkout.

Each layer depends on the one beneath. A rent charge without a correct stay is a wrong number. A stay without correct inventory is a resident in a bed that does not exist.

The Pune pilot

~100	91–100	1 + 2	3
Beds in the pilot property	Current residents	Owner plus one to two staff	Payment methods: cash, UPI, bank transfer

The pilot is deliberately narrow. One organisation, one property, staff-facing web application, manual payment recording, no payment gateway, no resident app, no document upload. Residents pay the property directly; **Nook records and reconciles, it does not take custody of funds.** CONFIRMED

The pilot is not a business. At one customer it does not pay for its own infrastructure. It is a funded design partner, and the platform economics begin somewhere around ten to twenty properties when onboarding is repeatable and infrastructure is shared. Chapter 3 treats this honestly rather than projecting a market.

Current maturity — what actually exists

> **READ THIS BEFORE ANYTHING ELSE IN THE HANDBOOK**
>
> The repository contains a **Milestone 0 engineering scaffold** and nothing more. There is a NestJS API shell with health endpoints, a React/Vite web shell, the approved design tokens, a read-only demonstration UI built from fictional fixtures, an **intentionally empty** database schema with no models and no migrations, and validation tooling.
>
> There is no authentication, no resident record, no booking, no ledger, no payment, no deposit, no spatial map and no database table. Five tests pass; they cover configuration parsing and brand-token invariants. Milestone 0 itself is **not yet accepted** — a dependency security advisory, the database, the API runtime and CI all remain unverified.

AREA	STATUS	EVIDENCE
Toolchain, lint, typecheck, build	VERIFIED	All exit 0 from the repository root
Unit tests	VERIFIED	5 tests pass — config parsing and design tokens only
Design tokens, light and dark	IMPLEMENTED	38 tokens; approved Option A palette

AREA	STATUS	EVIDENCE
Demonstration property UI	IMPLEMENTED	Read-only, fictional fixtures, no persistence
Database schema	NOT IMPLEMENTED	Empty by design; no models, no migrations
PostgreSQL connectivity	BLOCKED	Docker daemon absent; no connection ever established
API runtime / health endpoints	BLOCKED	Startup exits 1; endpoints never exercised
Dependency security	BLOCKED	Unresolved advisory; audit endpoint unreachable
CI execution	NOT IMPLEMENTED	Workflow present; no Git metadata, never run
Every product module in this handbook	NOT IMPLEMENTED	—

Chapter 23 gives the complete inventory with command exit codes and the precise diagnosis of each blocker.

Major constraints and unresolved decisions

Five things currently stand between this design and a working pilot. None is a coding problem.

CONSTRAINT	WHAT IT BLOCKS	OWNER
75 business rules are unanswered	Automated rent generation, allocation ordering, late fees, deposit deductions and refunds, final settlement. Each stays disabled and returns an explicit unconfigured-policy refusal.	Property owner
No architecture decision is approved	Seventeen decisions sit as proposals with a blank approval record. Milestone 0 acceptance depends on them.	Project owner
Milestone 0 is not accepted	Everything. No feature work should begin until the dependency advisory, database, runtime and CI blockers clear.	Engineering
Brand name is unclear	Public launch and domain commitment. Trademark clearance has not been performed and a competitive naming conflict is unresolved.	Legal review
The spatial map is new scope	It does not appear in the approved requirements package at all. It is the product's headline idea and currently its least specified area.	Project owner

A SCOPE OBSERVATION THE READER SHOULD NOT MISS

The interactive spatial map, public property discovery, phone verification and the enquiry workflow — Chapters 7 and 9 — are **not present in the approved requirements package** that governs the rest of this product. They are new product direction introduced with this handbook.

That is not an objection; the spatial map is the most compelling thing about Nook. But it means these chapters carry a different evidential weight from, say, Chapter 11, whose rules trace to a reviewed register. They are proposals awaiting their first approval, and they expand the pilot's scope materially. Chapter 24 records this as an open decision.

What a reader should take away

1. The product idea is sound and the distinguishing feature — spatial bed selection — is real and worth building.
2. The engineering foundation is deliberately minimal, honestly documented, and not yet accepted.
3. The financial design is the part that must be right first, and it is blocked on decisions only the property owner can make.
4. Nothing in this handbook has been approved. It is a proposal in a reviewable form.

CHAPTER 3

Product Vision and Strategy

Where Nook is trying to get to, who it serves, how it is positioned against incumbents, and what would have to be true for it to become a business rather than one hostel's software.

Mission and vision

Mission. Give independent accommodation operators one dependable record of their property, their residents and their money — so that the answer to any question about any of the three is the same wherever it is asked.

Vision. Make the physical building the interface. A property's layout, occupancy and availability should be something you look at and understand, not something you reconstruct from a spreadsheet. If Nook succeeds, choosing a bed online feels like choosing a seat on a flight, and managing a hundred of them feels like looking at a floor plan.

Customer pain points

These are drawn from the first customer and stated at the level the evidence supports. They are not generalised to the market without qualification.

PAIN	WHAT IT LOOKS LIKE IN PRACTICE	EVIDENCE
No single source of truth for dues	Register, WhatsApp screenshots and memory disagree. Nobody can prove which is right.	CONFIRMED
Silent revenue leakage	A resident skips a month and is never chased because nothing surfaces it.	INFERRED
Deposit disputes at checkout	A year of informal records gets argued over at the door. The owner either over-refunds or loses goodwill.	INFERRED
Manual reconciliation burden	Cash, UPI and bank credits reconciled by hand every month, for a partnership that needs agreed numbers.	CONFIRMED
Occupancy is opaque	Which bed is free, from when, and who was in it before — reconstructed from memory and paper.	CONFIRMED
Enquiries are high-friction	Every prospective resident requires a call or a visit to understand what is available.	UNVALIDATED

Inferred means a consequence that follows from the confirmed facts but has not been measured at the property. **Unvalidated** means it has not been established that this is a problem the first customer actually has — the enquiry workflow is new scope and its demand is assumed, not evidenced.

Target market

> **NO MARKET SIZING IS CLAIMED**
>
> This handbook does not state a number of hostels in Pune, a total addressable market, a growth rate or a competitor's revenue. No such research has been conducted for this project, and an invented figure in a founder-facing document tends to survive into a pitch deck as though it were sourced. The market characterisation below is qualitative and should be read as a hypothesis to validate.

The intended segment is **independent, owner-operated accommodation businesses** — hostels, PGs and small co-living properties — typically in the 30 to 300 bed range, operating one to a handful of properties, currently running on paper or on general-purpose spreadsheets.

Characteristics that make this segment coherent as a target: the operator is also the decision-maker, so a sale does not require a committee; the pain is financial and therefore quantifiable to them; switching cost from paper is low; and the workflows are similar enough between properties that one product can serve many without bespoke work. Characteristics that make it hard: price sensitivity is high, technical confidence is low, and support expectations are personal.

Geographic start is Pune, for the pragmatic reason that the first customer and the developer are both there and an in-person pilot is possible. Expansion beyond Pune is a later-phase question and is not designed for yet beyond keeping the data model multi-property and multi-organisation.

Customer personas

Chapter 5 covers the full set of system roles and their journeys. The three that matter commercially are these.

PERSONA	GOALS	FRUSTRATIONS	WHAT WINS THEM
The owner-operator Runs one property, often with a partner	Know the money is right. Spend less time chasing. Have books a partner can audit.	Software that needs more input than it returns. Being asked to change how the business works.	The dues screen answering a question faster than the register does.
The property manager Day-to-day operations, often the only full-time staff	Allocate beds without conflicts. Keep resident records straight. Follow up arrears.	Duplicate entry. Screens that hide what they need behind navigation.	Doing a full admission in one flow without leaving the page.
The prospective resident Student or working woman, searching on a phone	Understand the property, see real availability, know what it costs.	Listings with no actual availability. Having to call to learn anything.	Seeing the floor plan and which beds are genuinely free.

Core value propositions

TO	NOOK OFFERS	MEASURED BY
Owner	One explainable number for what every resident owes	Owner stops maintaining the paper register
Owner	Deposit settlements with an itemised, traceable history	Settlement disputes at checkout

TO	NOOK OFFERS	MEASURED BY
Manager	Bed allocation that cannot double-book	Allocation conflicts reaching a resident
Staff	Access scoped to the job — no incidental financial or document access	Permission matrix approved and tested
Prospective resident	Spatial understanding of the property and real availability	Enquiries that arrive already informed
Platform	A second property onboarded without a developer writing code	Time and code changes required to onboard

Business model possibilities

> **NOTHING HERE IS APPROVED PRICING**
>
> No price, tier, limit or billing policy in this section has been approved. They are structures to evaluate, not commitments. Subscription billing is explicitly out of scope for the pilot.

MODEL	FIT FOR NOOK	STATUS
Per-bed subscription	Scales with the customer's own capacity and is easy to explain. The billable-bed definition must be settled first — total beds, occupied beds, or configured beds — because the three differ materially at a property with vacancy.	PROPOSED
Per-property flat tier	Simpler to price and forecast; penalises small properties and under-charges large ones.	OPEN
Onboarding and migration fee	One-off, reflects genuine effort. Historical data migration is real work and should not be absorbed into a subscription.	PROPOSED
Paid integrations / optional modules	Messaging, payment gateway and similar carry per-use cost that must be recovered rather than bundled.	OPEN
Payment custody or float	**Explicitly out of scope.** Nook records and reconciles; residents pay the property directly. Taking custody of funds changes the regulatory position entirely.	EXCLUDED

The cost reality that pricing has to clear

At one customer the arithmetic does not work, and it is better to say so than to discover it later. A managed database with backups, an application host, private object storage and a messaging provider consume a meaningful share of a single small property's subscription before any support time, developer hours or transaction fees. A per-bed price that looks reasonable to one owner does not fund the product; ten to twenty properties sharing that infrastructure begin to.

The practical consequences for strategy are concrete: do not promise unlimited support or bespoke development at a per-bed rate; build onboarding to be repeatable rather than hand-held; and treat the first property as a funded design partner rather than as proof of a business model. A cost model is a required artifact before any price is quoted.

Competitive positioning

Nook enters an existing category with established products. The honest position is that Nook is not first, is not cheapest by default, and cannot claim feature parity with anything.

AXIS	NOOK'S INTENDED POSITION
Spatial experience	The genuine differentiator. Incumbents treat beds as list rows. This is where to compete.
Financial rigour	Competitive necessity, not differentiation. Expected to be correct; noticed only when wrong.
Breadth of modules	Deliberately behind. Attendance, outpass, visitor logs and meal plans are not pilot scope.
Price	Undetermined. No competitor price has been verified and none is quoted here.
Trust and support	Currently a weakness: one developer, a student, with no support guarantee. Chapter 24 treats this as a risk, not a strength.

AN UNUSUAL COMPETITIVE FINDING WORTH RECORDING

Colour research conducted during brand development measured a close convergence between Nook's accent colour and a significant competitor's brand colour — approximately 0.2° apart in CIE LCh hue, with page surfaces around ΔE 2.1 apart. The convergence was accidental and predates the research.

An alternative palette with a deep teal accent exists as **Option B** and is proposed but not approved. The decision is coupled to the unresolved naming conflict and should be taken together with it rather than separately. Nothing in the product currently references Option B.

Product principles

1. **One source of truth for inventory.** The resident-facing map and the staff-facing inventory read the same records. Two representations of the same building will drift, and the drift will reach a resident.
2. **The server is the authority.** A map, a cached page or a hidden button is a convenience. Availability, permissions and money are decided server-side, inside transactions.
3. **Posted financial records are immutable.** Corrections are reversals and adjustments with a reason and an actor. Nothing is silently edited or deleted.
4. **An unresolved policy disables its feature.** No reasonable-looking default is substituted for a decision the owner has not made. The operation refuses explicitly and says which rule is missing.
5. **Collect the minimum.** Identity documents are the highest-consequence data in the system and the least necessary for the problem being solved. Default is not to collect them.
6. **Isolation is structural.** Organisation scoping exists from the first commit, because retrofitting it means migrating live financial data.
7. **Evidence over assertion.** A feature is complete when a test demonstrates it, a backup is proven by a restore, and an approval exists as a signature.

First-pilot strategy

The pilot's purpose is to prove the core loop — inventory, residents, charges, payments, dues, checkout — at one real property with real money, and to earn the right to a second customer. It deliberately excludes the payment gateway, the resident app, document upload, automated messaging and the operations modules.

The sequencing that follows from the dependency chain is: accept Milestone 0; approve the architecture; answer the money rules; build inventory, then residents, then the ledger; then reports and checkout; then migrate opening balances under reconciliation; then run in parallel with the register until the owner trusts it. Chapter 22 sets this out with exit gates.

> **THE MOST LIKELY WAY THE PILOT FAILS**
>
> Not a bug. The owner quietly keeps using the paper register. It happens when the software is ninety per cent right: one number looks wrong, one workflow takes six taps where paper takes one, and within three weeks the register is back on the desk "just as a backup". Then there are two sources of truth and the paper one wins.
>
> The counter is not more features. It is a supervised pilot where every workaround observed in the office is treated as a defect rather than user error — and a go-live defined as the day the owner can answer "who owes money?" faster in Nook than in the register.

Long-term expansion strategy

HORIZON	FOCUS	GATE TO THE NEXT HORIZON
Pilot	One property. Prove the core loop and the financial record.	Owner retires the paper register and signs acceptance.
Early adopters	A handful of Pune properties. Standardise onboarding; validate pricing and real support effort.	A property onboards with no developer code change.
Platform	Self-serve organisation setup, subscription administration, multi-property reporting.	Unit economics clear infrastructure and support cost.
Product depth	3D building map, resident portal, payment gateway, verified integrations.	Demand evidenced by paying customers, not assumed.

Success metrics and how they are measured

Targets are deliberately absent. A baseline has not been recorded at the property, and inventing a percentage before measuring the starting point produces a number that cannot be honestly reported against. The first task of the pilot is to record the baseline.

METRIC	MEASUREMENT METHOD	BASELINE
Time to answer "who owes what?"	Timed, same question, register versus Nook, with the owner	NOT RECORDED
Parallel recordkeeping	Whether staff maintain any duplicate record for the same transaction	Binary; target is none
Unexplained financial discrepancy	Count at go-live reconciliation; every difference resolved or formally accepted	Target is zero

METRIC	MEASUREMENT METHOD	BASELINE
Allocation conflicts reaching a resident	Incident count during the pilot	Target is zero
Deposit settlement disputes	Count per checkout, before versus after	NOT RECORDED
Onboarding effort for customer two	Developer hours and code changes required	Target is zero code changes

CHAPTER 4

Complete Feature and Module Map

Twenty-five modules, what each is for, what it depends on, which release phase it belongs to and what exists today. A module appearing here is not approved for the first release.

The Nook module map

Solid violet = proposed pilot scope · tinted = next release · outlined = later phase

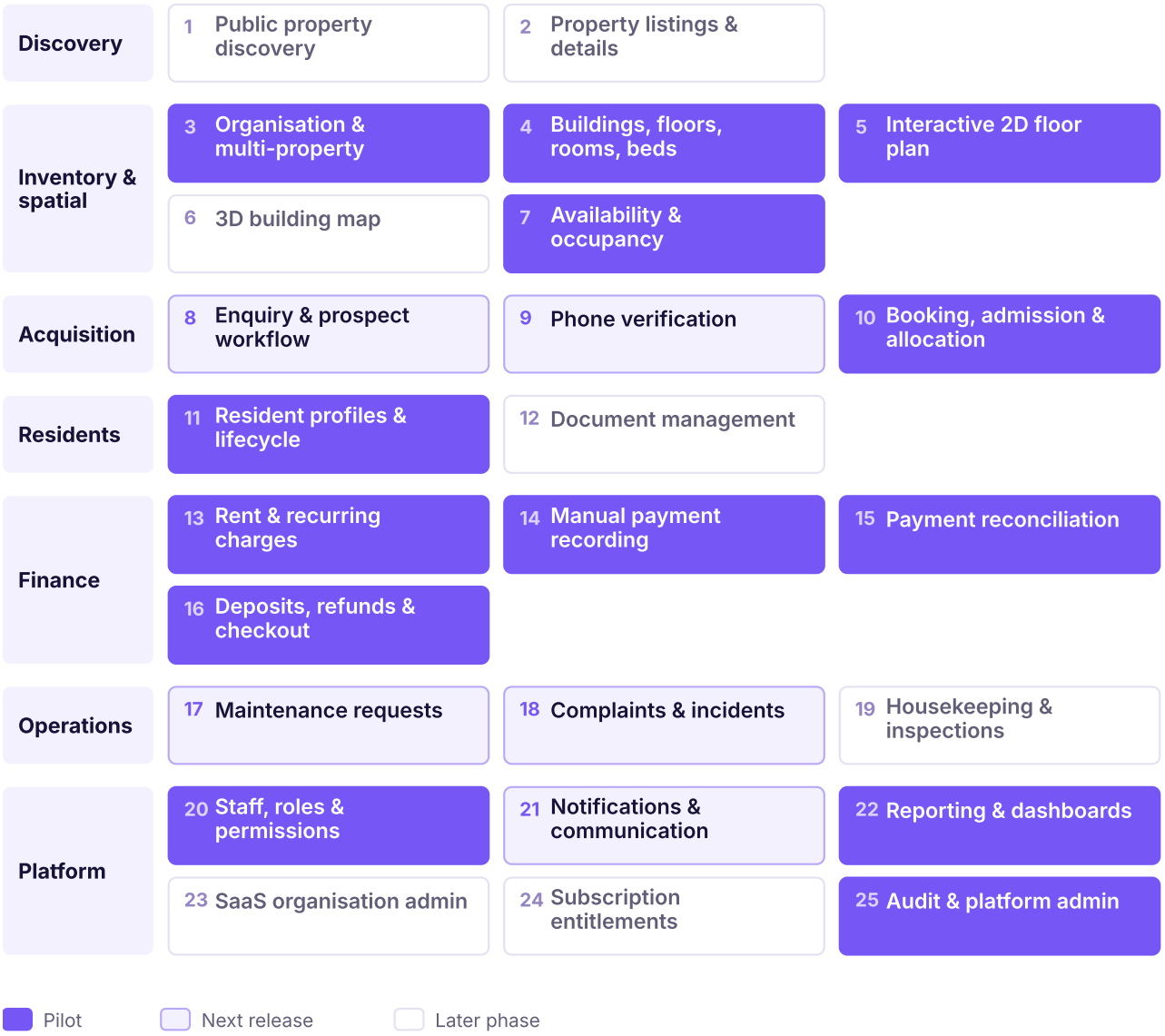


Figure 4.1 — The Nook module map. Solid violet marks modules proposed for the pilot release; tinted marks the release after it; outlined marks a later phase. Grouping follows the dependency chain: nothing in Finance can be correct until Inventory and Residents are.

PHASE ASSIGNMENT IS A PROPOSAL, NOT A COMMITMENT

Several modules shown as pilot scope — public discovery, the 2D floor plan, enquiry and phone verification — do not appear in the approved requirements package at all. They are new product direction introduced with this handbook and materially expand the pilot. Chapter 24 records the scope decision as open. Where a module is new scope, its row below says so.

Module register

The register gives, for each module, the release phase proposed, what it depends on, and its present state in the repository. Detailed profiles follow by group; the deepest treatments are Chapter 7 (spatial), Chapter 9 (booking) and Chapter 11 (finance).

#	MODULE	PHASE	DEPENDS ON	PRIMARY RISK	TODAY
1	**Public property discovery** · new scope	PILOT	Listings, availability	Exposing resident data through a public surface	NONE
2	**Property listings & details** · new scope	PILOT	Property, inventory	Listing drifts from real inventory	NONE
3	**Organisation & multi-property**	PILOT	Identity	Isolation retrofitted rather than built in	NONE
4	**Buildings, floors, rooms, beds**	PILOT	Organisation	Allocatable unit modelled wrongly for the real property	NONE
5	**Interactive 2D floor plan** · new scope	PILOT	Inventory, layout	Map treated as the availability authority	NONE
6	**3D building map** · new scope	FUTURE	2D layout model	Divergent spatial model; performance; accessibility	NONE
7	**Availability & occupancy**	PILOT	Inventory, stays	Race conditions producing double allocation	NONE
8	**Enquiry & prospect workflow** · new scope	POST-PILOT	Discovery, residents	Unvalidated demand; collecting data with no purpose	NONE
9	**Phone verification** · new scope	POST-PILOT	SMS provider	Per-message cost and abuse; no provider approved	NONE
10	**Booking, admission & allocation**	PILOT	Availability, residents	Concurrency; unresolved date semantics	NONE
11	**Resident profiles & lifecycle**	PILOT	Inventory	Over-collection; losing history on room change	NONE
12	**Document management**	EXCLUDED	Private storage	Identity documents exposed; retention unresolved	NONE
13	**Rent & recurring charges**	PILOT	Stays, terms	Automated generation blocked on six unanswered rules	NONE

#	MODULE	PHASE	DEPENDS ON	PRIMARY RISK	TODAY
14	**Manual payment recording**	PILOT	Charges	Duplicate recording; status policy unresolved	NONE
15	**Payment reconciliation**	PILOT	Payments	Report definitions undefined, so totals disputed	NONE
16	**Deposits, refunds & checkout**	PILOT	Payments, stays	Deposit treated as revenue; automatic net-off	NONE
17	**Maintenance requests**	POST-PILOT	Property, staff	Scope creep into general ticketing	NONE
18	**Complaints & incidents**	POST-PILOT	Residents, staff	Incident records need restricted access	NONE
19	**Housekeeping & inspections**	FUTURE	Inventory, staff	Low value relative to build cost	NONE
20	**Staff, roles & permissions**	PILOT	Identity	Role-name checks instead of permission checks	NONE
21	**Notifications & communication**	POST-PILOT	Provider, consent	Arrears disclosed to the wrong recipient	NONE
22	**Reporting & dashboards**	PILOT	All records	Totals presented before definitions are agreed	NONE
23	**SaaS organisation admin**	FUTURE	Organisation	Built before customer three exists	NONE
24	**Subscription entitlements**	FUTURE	Org admin	Pricing not approved; billable-bed undefined	NONE
25	**Audit & platform admin**	PILOT	Identity	Logged too thinly to reconstruct a balance	NONE

Every module reads **None** in the final column. That is accurate: no product module exists in the repository. The demonstration property UI described in Chapter 23 renders fictional fixtures and is not module 4, 5 or 7.

Group profiles

Discovery — modules 1–2

Purpose	Let a prospective resident find and understand a property without contacting anyone.
Users	Public visitor, prospective resident. No authentication.
Core screens	Search and results; property detail with photographs, amenities, rates and the floor plan.
Workflows	Search → filter → property detail → view layout and availability → enquire.
Permissions	Unauthenticated read of published data only. No permission grants anything public.
Business policy	What a property chooses to publish is the owner's decision; publication is opt-in per property. OPEN
API	Read-only public endpoints, aggressively rate-limited, serving a projection — never the internal record.

Tests	No resident name, contact, document or financial field is reachable through any public response. Verified by a test that asserts the projection's field set.

> **THE DEFINING CONSTRAINT ON ANYTHING PUBLIC**
>
> A public surface over a resident database is the highest-risk thing in this product. The public endpoint must serve a **separate projection** containing only published property attributes and derived counts — never a filtered view of the resident-bearing record. A bed on a public map shows a status; it never carries, and must never be joinable to, the identity of whoever occupies it.

Inventory and spatial — modules 3–7

Purpose	Hold the authoritative model of what physically exists and what is free, and render it spatially.
Users	Owner, property manager, front desk; the public and prospective residents see a derived view.
Core screens	Property setup; building/floor/room/bed management; layout editor; interactive floor plan; occupancy dashboard.
Workflows	Onboard property → define buildings and floors → create rooms and beds → position them on a layout → publish → maintain state as residents arrive and leave.
Key concepts	Allocatable unit; layout version; bed state; effective-dated assignment.
Permissions	`inventory.read`, `inventory.write`, `assignment.manage`, `property.manage`.
Dependencies	Organisation and identity. Everything else depends on this group.
Business policy	Whether the allocatable unit is always a bed; which states reserve versus occupy; whether a resident may hold two beds during a transfer. All OPEN — BR-LIF-002, BR-LIF-004, BR-LIF-006.
Tests	Concurrent allocation of one bed yields exactly one success; layout changes never silently detach an occupied bed.

Acquisition — modules 8–10

Purpose	Turn interest into a confirmed, allocated resident without double-booking anyone.
Users	Prospective resident; property manager approves.
Core screens	Enquiry form; verification step; bed selection on the floor plan; staff approval queue; admission and check-in.
Workflows	Enquiry → verification → selection → staff approval → atomic allocation → check-in. Alternatives: rejection, expiry, no-show, cancellation.
Permissions	`stay.manage` for admission; `assignment.manage` for allocation.
Business policy	The pilot uses **manual admission with staff approval**. CONFIRMED Instant allotment must not be enabled without explicit owner approval. Date interval semantics, cancellation, expiry, eligibility and any payment required at booking remain OPEN.
Tests	Two simultaneous selections of one bed; allocation after the displayed availability has gone stale; retry of a timed-out confirmation.

Residents — modules 11–12

Purpose	Hold the person, their stays and their history, separately and without loss.
Users	Property manager, front desk; owner for oversight.
Core screens	Resident list; profile with stay history, current bed, terms, statement, deposits and change history.

Key concepts	Resident identity is separate from a stay, so a repeat admission does not overwrite earlier history.
Permissions	`resident.read`, `resident.write`, `stay.manage`. `document.read` is a separate grant and is never implied by profile access.
Business policy	Mandatory fields, required identity documents and retention are all OPEN — BR-LIF-008, BR-LIF-009, BR-OPS-003.
Tests	A former resident re-admitted retains the earlier stay; a profile edit changes no issued financial record.

Finance — modules 13–16

Purpose	Make every rupee a resident owes explainable from first charge to last allocation.
Users	Owner, finance clerk; manager for read access where granted.
Core screens	Rent and dues; resident statement; record payment; payment detail and receipt; deposits ledger; checkout settlement; collection and outstanding reports.
Key concepts	Six immutable record types — invoice, payment, allocation, deposit, adjustment, reversal. Balance is derived, never stored as truth.
Permissions	`payment.record` is separate from `payment.confirm`; `finance.read` is separate from `data.export`. Deposit deduction, refund and settlement are blocked by default.
Business policy	The largest concentration of open rules in the product. Automated billing, allocation order, late fees, deposit deductions, refunds and settlement are each disabled pending sign-off.
Tests	Eleven architecture invariants enforceable now; seven policy scenarios that cannot be asserted until the owner supplies expected results. Chapter 20 classifies each.

Operations — modules 17–19

Purpose	Record what broke, who is handling it and whether it was resolved.
Users	Front desk and manager log; residents report through staff in early phases.
Scope discipline	Deliberately thin. No SLAs, no cost tracking, no vendor management. These make it a maintenance product rather than a module.
Permissions	Operational roles; incident records carry restricted access because they may contain sensitive detail.
Business policy	Whether residents report directly — which requires a portal — is OPEN.

Platform — modules 20–25

Purpose	Make Nook a product serving many unrelated businesses rather than one hostel's application.
Users	Owner for staff access; Nook platform operator for tenant administration, under audited just-in-time access.
Key concepts	Permission identifiers rather than role-name checks; server-derived organisation context; append-only audit.
Business policy	Role-to-permission grants are the owner's decision — BR-SEC-001 to BR-SEC-003. Pricing, tiers and limits are not approved.
The test that matters	A second organisation can be onboarded end to end **without a developer writing code**. Until that is true, Nook is a consulting engagement with a login screen.

What is deliberately absent

These appear in competitor feature lists and are not planned for the pilot: attendance and roll-call, outpass and leave approval, visitor logs, meal plans and mess management, metered electricity sub-billing, vendor and purchase management, and a public marketplace.

Attendance and outpass additionally warrant a deliberate decision rather than a default. The pilot property houses adult working women and students; building monitoring features because a competitor lists them is not a product decision. If an operator requires them, that is a conversation with a specific justification, not an assumption.

CHAPTER 5

Personas, Roles and User Journeys

Eight roles, what each is trying to achieve, the journeys they take including the ways those journeys fail, and the permission baseline that decides what each may actually do.

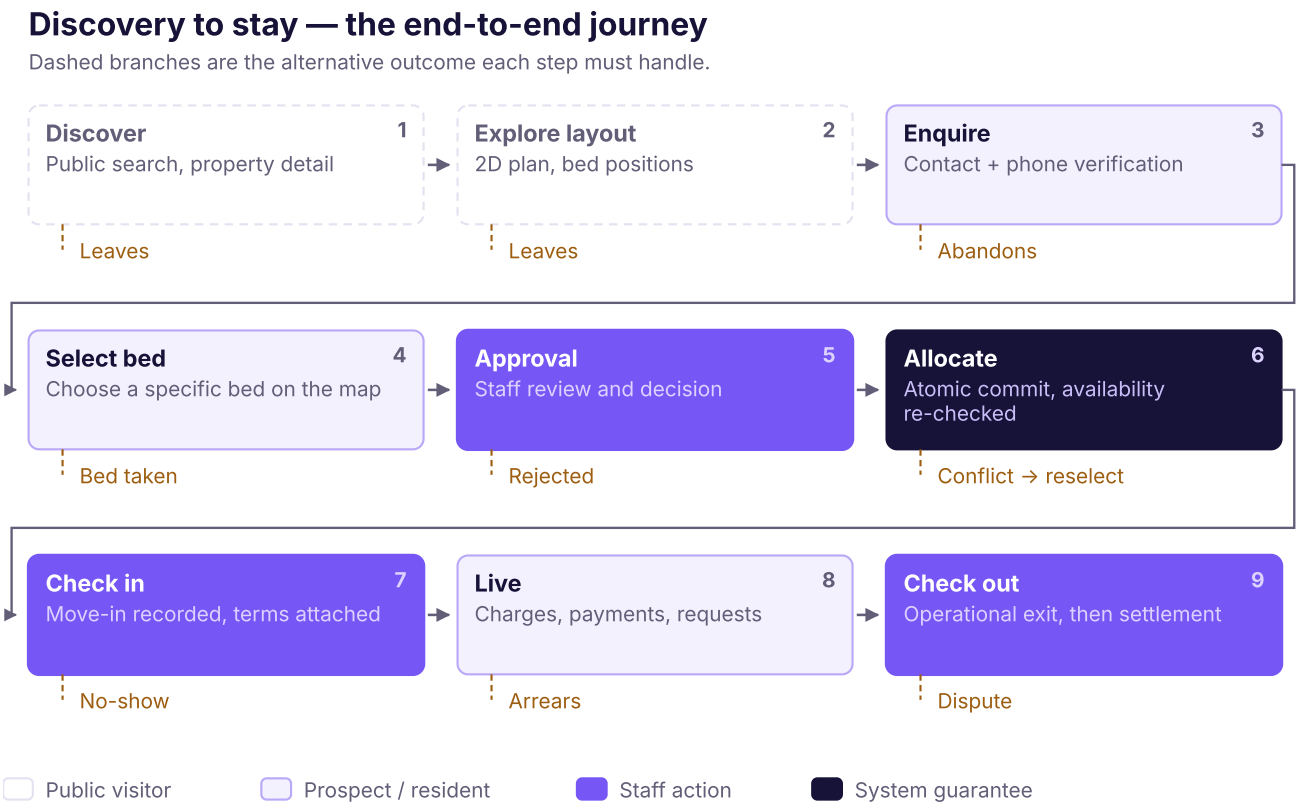


Figure 5.1 — Discovery to stay. The happy path runs left to right; the dashed branches are the alternative outcomes each step must handle. Four of the nine steps are new scope introduced with this handbook and are not in the approved requirements package.

The eight roles

ROLE	PRIMARY JOB	BOUNDARY THAT DEFINES IT
Public visitor	Understand a property without identifying themselves	Sees only published projections; never a resident-bearing record
Prospective resident	Find a bed, enquire, be admitted	Identified but not yet a resident; no access to any stay
Current resident	See their own statement, receipts and requests	Own records only. Future phase.
Owner / Partner	Oversight of occupancy, money and staff access	Broadest access, but cannot bypass financial invariants

ROLE	PRIMARY JOB	BOUNDARY THAT DEFINES IT
Property Manager	Resident lifecycle, allocation, daily operations	Operational authority; financial access only where granted
Finance Clerk / Cashier	Record payments, issue receipts, reconcile	May record money arriving; may not attest that it did
Warden / Front Desk	Who is where; day-to-day tasks	No financial and no document access by default
Nook platform operator	Tenant administration and support	No standing access to customer data; just-in-time, reason-coded, audited

> **THREE SEPARATIONS THAT MUST NEVER BE COLLAPSED**
>
> `payment.record` is not `payment.confirm` — recording that money arrived is not the authority to attest that it did.
> `resident.read` is not `document.read` — reading a profile never implies reading that person's identity documents.
> `finance.read` is not `data.export` — viewing a balance on screen is not the authority to take the data out of the system.

Journeys

Prospective resident — discovery to admission NEW SCOPE

1. Searches, opens a property, reads amenities and rates.
2. Opens the floor plan, switches floors, sees which beds are free and where they physically are.
3. Selects a bed and submits an enquiry with contact details. Phone verification runs if enabled.
4. Staff review the enquiry and approve or reject it.
5. On approval the system re-checks availability and commits the allocation atomically.
6. Move-in is recorded; rent terms are attached to the stay.

Failure paths. The chosen bed is taken between selection and approval — the resident is told plainly and returned to the map with current state, never silently reassigned. The enquiry is rejected — a reason is recorded, and no stay or charge is created. Verification fails or is abandoned — the enquiry remains unverified and no bed is held. The resident does not arrive — no-show handling is OPEN (BR-LIF-012).

> **SELECTING A BED DOES NOT HOLD IT**
>
> A pre-checkout hold is **not** part of this design and must not be introduced without explicit owner authorisation. Until it is authorised, selection expresses a preference; the bed stays available to everyone else until an allocation is committed. Holds change occupancy economics — a held bed earns nothing — and that is the owner's decision, not an implementation convenience.

Property manager — admission and allocation

1. Opens the enquiry queue, reviews the applicant against the property's criteria.
2. Confirms or changes the requested bed; the map shows live state, not the state at enquiry time.
3. Approves. The system validates the resident, the bed and the transition inside one transaction.
4. Records move-in; the stay becomes active and billing can begin once terms exist.

Failure paths. Another staff member allocated the bed seconds earlier — the operation is rejected with current state rather than overwriting. Required resident fields are missing — the stay cannot activate. Rent terms are not yet approved — the stay exists but charge generation refuses.

Finance clerk — recording a payment

1. Opens the resident, sees outstanding charges oldest-first.
2. Enters amount, date received, method and the reference — UPI id, bank UTR, or a receipt number for cash.
3. **Explicitly selects which charge the payment settles.** No charge is preselected while allocation order is unresolved.
4. Saves. The payment and its allocation commit together or not at all.
5. A numbered receipt is produced and handed over.

Failure paths. The reference duplicates an existing payment — refused with a link to the original. The request times out after the server committed — a retry with the same idempotency key returns the original result rather than creating a second payment. The allocation exceeds what remains allocatable — refused with the current remaining amount.

Owner — the daily question

Opens the dashboard and asks who owes money. Sees outstanding by resident with ageing, today's collections split by method, and current occupancy. Drills into a resident and reads the statement line by line: every charge, every payment, every allocation, every adjustment, in date order, summing to the displayed balance.

Failure path that matters most. A number looks wrong. If the owner cannot immediately see *why* the number is what it is, the pilot fails regardless of correctness — which is why the statement is a first-class screen rather than a report export.

Warden / front desk — a day

Looks up which resident is in which bed, logs a maintenance request, records a visitor if that module is enabled. Sees no balances, no payment references and no identity documents. If this role needs financial visibility at a particular property, that is an explicit owner grant recorded against BR-SEC-002, not a default.

Platform operator — supporting a customer

Has no standing access to any customer's residents or finances. To assist, access is requested with a reason, granted for a limited time, and every action taken under it is audited and visible to the customer. A general-purpose support console is deferred; until it exists, support is a conversation rather than a login.

Permission baseline

Roles are named bundles of permissions, granted per organisation. **The server checks the permission identifier, never a role-name string.** This table is the proposed starting baseline; the grants are the owner's decision and are recorded against BR-SEC-001 to BR-SEC-003.

Yes = default capability · **Scoped** = only with explicit grant or assigned property · **No** = not granted by default · **Blocked** = disabled until the governing policy is signed off.

CAPABILITY	PERMISSION	OWNER	MANAGER	FINANCE	WARDEN	RESIDENT	PLATFORM
Organisation settings & members	`settings.manage` `member.manage`	Yes	No	No	No	No	No
Resident profile (non-document)	`resident.read` `resident.write`	Yes	Scoped	Scoped	Scoped	Own	No
Rooms, beds, assignments	`inventory.*` `assignment.manage`	Yes	Yes	Read	Scoped	No	No
Stays & operational checkout	`stay.manage`	Yes	Yes	No	Scoped	No	No
Financial balances & reports	`finance.read` `report.read`	Yes	Scoped	Yes	No	Own	No
Record a payment	`payment.record`	Yes	No	Yes	No	No	No
Confirm a payment	`payment.confirm`	Policy	No	No	No	No	No
Allocate a payment	`payment.allocate`	Policy	No	Grant	No	No	No
Reverse allocation / refund	`allocation.reverse` `payment.refund`	Policy	No	No	No	No	No
Create or adjust charges	`charge.*` `rental_terms.manage`	Policy	No	No	No	No	No
Receive a deposit	`deposit.receive`	Yes	No	Grant	No	No	No
Deductions, refunds, settlement	`deposit.*` `settlement.*`	**Blocked for every role until the governing policy is signed off**					
View identity documents	`document.read`	Scoped	No	No	No	Own	No
Export / import data	`data.export` `migration.*`	Yes	No	Grant	No	No	No
View audit events	`audit.read`	Yes	Scoped	Grant	No	No	No

Policy means the capability exists for this role but only once the governing business rule is approved. **Grant** means it requires an explicit owner grant rather than coming with the role.

Organisation and property boundaries

Two boundaries operate together, and conflating them is the most common multi-tenant mistake.

The **organisation boundary** is absolute. A user who is not a member of an organisation cannot reach any of its records through any route — a direct identifier, a list, a search, a report, an export, an import, a background job or a file. A request naming an organisation the actor does not belong to returns *not found* rather than *forbidden*, so organisation existence cannot be probed.

The **property boundary** operates inside an organisation. A manager may be assigned to one property of three; the other two are invisible to them. This is a grant within a tenancy, not a tenancy of its own.

The active organisation is derived from the authenticated server-side session and verified membership. **A client-supplied organisation identifier is never authority** — not in a path, a query string, a request body or a header. Where a user belongs to more than one organisation, switching requires an explicit authenticated operation that re-confirms membership.

> **NOTHING IN THIS CHAPTER IS IMPLEMENTED**
>
> No role, permission, journey or boundary described here exists in the repository. There is no authentication, no user record and no permission check. This is the specification the implementation will be tested against, not a description of behaviour that can be observed today.

PART II

Experience

The design system built on the approved brand, and the interactive spatial map that is the product's distinguishing idea.

6 Product UI/UX and Design System
7 Building the Interactive Spatial Map



CHAPTER 6

Product UI/UX and Design System

The approved brand identity, the validated token system built on it, and the component, layout and accessibility rules that turn those tokens into a product three very different audiences can use.

Brand principles

The identity is **wordmark-led**. The primary logo is the lowercase `nook` wordmark; the standalone symbol is the lowercase `n` monogram, drawn from the identical vector path as the wordmark's own `n` rather than redrawn to look similar. Every asset in the package derives from one approved geometry: 100-unit x-height, 16-unit monoline stroke, 104-unit `o` outer diameter with a two-unit optical overshoot, and 45° symmetric arm and leg on the `k`.

The letterforms are closed vector paths. **No font is referenced, embedded or required.** A consequence worth stating for implementers: the wordmark can never be reproduced by typing "nook" in any typeface, and the product UI's eventual type choice is an independent decision that cannot break the logo.

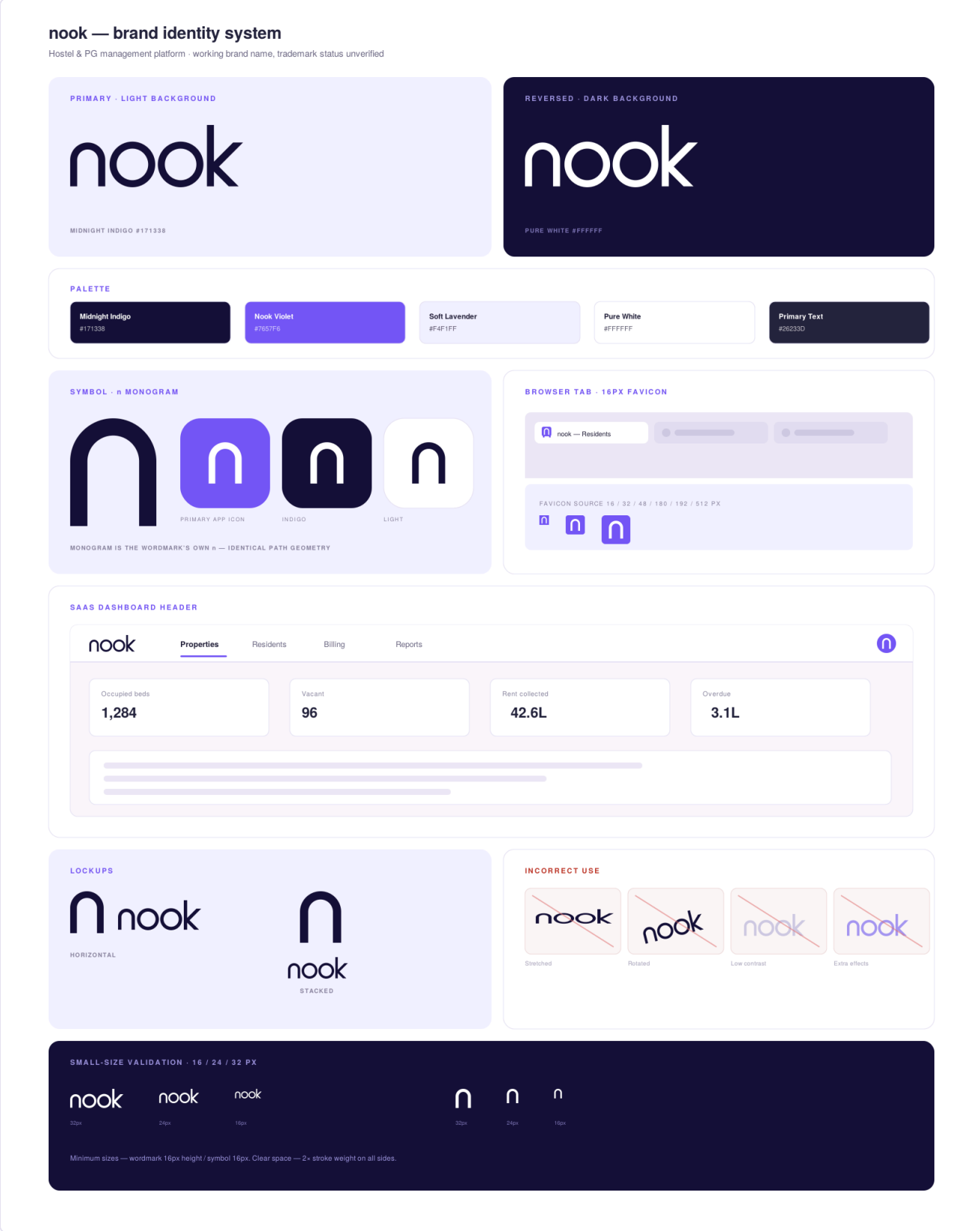


Figure 6.1 — The approved brand identity system. Primary and reversed lockups, the five-colour palette, the `n` monogram and its app-icon treatments, browser-tab rendering, a dashboard header application, correct and incorrect use, and small-size validation at 16, 24 and 32 px. Reproduced from the approved asset package. APPROVED

Colour

Five brand colours. The logo must remain effective in a single solid colour, and **colour never carries meaning in the mark**.

NAME	HEX	ROLE	CONTRAST
Midnight Indigo	`#171338`	Primary wordmark on light grounds; dark section fields; headings	17.70:1 on white — AAA
Nook Violet	`#7657F6`	Accent; app-icon ground; selected states; diagram emphasis	4.70:1 on white — AA
Soft Lavender	`#F4F1FF`	Surface and panel tint	Indigo on lavender 15.90:1 — AAA
Pure White	`#FFFFFF`	Reversed wordmark on dark grounds; page background	17.70:1 on indigo — AAA
Primary Text	`#26233D`	Body copy. **Not a logo colour.**	—

These are the **Option A** palette and are approved. An alternative **Option B** with a deep teal accent exists, is proposed, and is referenced by nothing in the product. The decision between them is coupled to the unresolved naming conflict (Chapter 24) and should be taken with it.

> **THIS DOCUMENT EXTENDS THE PALETTE, AND SAYS SO**
>
> The brand guidelines define **five colours and no others** — no neutrals, no semantic colours. A printed document of this kind needs hairline rules, panel tints and status signals that readers can tell apart in greyscale, so this publication declares **fourteen extension values** alongside the five approved ones: secondary ink and rule greys, lavender and violet tints for panels and reversed type, and three semantic pairs (green, amber, red) carrying the status vocabulary.
>
> They are **typographic extensions for this document**, not brand colours and not logo colours. No mark anywhere in this package is rendered in any of them. The package's verification script asserts that the five approved values appear unaltered and that every other colour used is one of the declared extensions rather than an ad-hoc literal — but the extension set itself has OPEN status and needs a decision alongside the Option A / Option B choice: a product UI will need neutrals and semantic colours too, and they should be chosen once, deliberately, rather than inherited from a handbook.

Design tokens

Thirty-eight tokens, defined for light and dark, with 82 contrast measurements recorded during validation. The token file is shipped at the root and referenced as `var(--color-*)`. IMPLEMENTED — this is one of the few things in this handbook that exists in the repository today.

TOKEN GROUP	MEMBERS	RULE
Brand	`--color-brand-primary`, `--color-brand-accent`, `--color-brand-subtle`	Identity surfaces only
Accent states	`--color-accent-text`, `-hover`, `-pressed`	Interaction, not decoration
Surfaces	`--color-background`, `--color-surface`, `--color-surface-raised`	Elevation through surface, not shadow
Text	`--color-text-primary`, `-secondary`, `-on-accent`, `-on-dark`	Pick by ground, not by preference
Borders	`--color-border-subtle`, `--color-border-strong`	Strong for controls; subtle for dividers only

TOKEN GROUP	MEMBERS	RULE
Status	success, warning, danger, info, neutral — each `-text`, `-solid`, `-tint`	Always paired with text or an icon
Charts	`--color-chart-1` … `-5`, `--color-chart-grid`	Categorical, distinguishable in greyscale

TWO TOKEN SPLITS THAT ARE EASY TO IMPLEMENT WRONGLY

`--color-accent-text` is for violet used as *text*; `--color-brand-accent` is for violet used as a *fill*. They are different values because text needs more contrast than a surface does.

`--color-border-strong` goes on anything a user must identify as a control; `--color-border-subtle` is for dividers only. Using subtle on an input makes the input invisible to anyone with reduced contrast sensitivity.

Typography and spacing

The product typeface is OPEN. The logo carries no font dependency, so this decision is genuinely independent and can be taken later without rework. The scale and spacing system below is proposed and does not depend on which face is chosen.

ROLE	SIZE	WEIGHT	USE
Display	28–32 px	650	Page title, one per screen
Heading	20 px	650	Section within a page
Subheading	16 px	600	Card and panel titles
Body	14 px	400	Default. Never below 14 px for sustained reading.
Small	12.5 px	400	Table cells, metadata, helper text
Micro	11 px	600	Status pills and labels only; never a sentence
Numeric	inherit	—	Tabular figures for every money and count column

Spacing uses a 4 px base: 4, 8, 12, 16, 24, 32, 48, 64. Dense data tables use the 4–12 range; page sections use 24–48. **Money is always tabular-aligned and right-aligned** so columns of figures can be scanned and compared without reading each one.

Components

COMPONENT	BEHAVIOUR THAT MATTERS
Button	Primary, secondary, ghost, destructive. A destructive action always states its consequence rather than asking "Are you sure?". A financial submit is disabled while pending but relies on backend idempotency for actual safety.
Status pill	Tint plus text. Never colour alone — a paid/unpaid distinction carried only by green and red is invisible to a significant minority of users and in greyscale print.
Data table	Stable sort, pagination, filters preserved on return. Money columns right-aligned and tabular. Sensitive columns absent by default rather than hidden by CSS.
Money input	Accepts rupees as typed, converts once at the boundary, transmits integer paise as a string. Never performs authoritative arithmetic in the browser.

COMPONENT	BEHAVIOUR THAT MATTERS
Statement row	Charge, payment, allocation, adjustment and deposit entries are visually distinct and individually traceable. This is the screen the pilot is judged on.
Bed cell	The atom of the spatial map. Carries a label, a state and a selection affordance, and must be operable by keyboard and announced to a screen reader.
Blocked action	A control whose governing policy is unresolved is visible, disabled, and states which rule is missing. Hiding it would make the gap invisible to the owner who must close it.

Navigation and layouts

Two distinct shells, because the audiences are not the same.

The **authenticated shell** carries a left sidebar with organisation and property context, a main content area with page title, primary action and filters, and an account menu. On mobile the sidebar collapses; wide tables become row cards rather than horizontally scrolling grids. Navigation is permission-aware, driven by permission identifiers — but hiding a menu item is usability, never a control.

The **public shell** has no sidebar, no organisation context and no authenticated state. It is a separate layout serving a separate projection, which is a security boundary as much as a design one.

Pilot navigation

Overview · Residents · Rooms & Beds · Rent & Dues · Payments · Deposits · Reports · Imports & Exports · Settings and Staff Access.

There is no Documents section, because binary document upload is excluded from the pilot. If it is later approved, documents appear inside the resident profile for holders of `document.read` — never as a top-level destination.

Dashboard patterns

The Overview answers one question: *what needs attention today?* It is a work queue, not a vanity board. Candidate tiles are occupied and vacant beds, outstanding dues with ageing, collections for a chosen period, residents overdue, and unallocated or unverified payments.

> **A METRIC WHOSE RULE IS UNRESOLVED SHOWS ITS ABSENCE, NOT A ZERO**
>
> An overdue count depends on an approved due-date rule and grace period. Until those exist, the tile states that overdue cannot be computed. It does not display zero, a dash, or a plausible-looking number. A zero reads as "nobody owes anything", which is a false statement about money.

States every data screen must define

STATE	REQUIREMENT
Loading	Skeleton or progress. Never flash a misleading zero.
Empty	Explain why nothing is here and give the next permitted action.
No results	Distinguish "no match" from "not permitted to see matches".
Validation error	Inline, field-level, actionable. Preserve safe input.

STATE	REQUIREMENT
Conflict	Explain what changed, show current state, let the user retry deliberately. Never silently overwrite.
Permission denied	Plain explanation, no detail about the protected resource.
Not found / concealed	Identical response whether absent or belonging to another organisation.
Policy not configured	Name the missing rule in plain language, say who resolves it, offer the manual alternative. Corresponds to the API's explicit refusal.
Network failure	Never claim a save succeeded unless confirmed. Offer a safe retry and show the request id.
Partial workflow	Say what committed and what did not, and how to retry safely.

Responsive breakpoints

BREAKPOINT	WIDTH	BEHAVIOUR
Compact	< 640 px	Single column; sidebar as drawer; tables become cards; floor plan pans and pinch-zooms with a floor switcher
Medium	640–1024 px	Collapsible sidebar; tables scroll horizontally with a frozen first column
Wide	> 1024 px	Persistent sidebar; full tables; floor plan beside a details panel

Staff do a meaningful share of their work on a phone — a manager allocating a bed while standing in the corridor is the normal case, not the exception. Phone is a first-class target for the operational screens, not a degraded fallback.

Accessibility

Target is **WCAG 2.1 AA** PROPOSED. The palette was validated against it: every brand pairing in the table above meets or exceeds AA, and the measurements are recorded rather than assumed.

- **Contrast.** 4.5:1 for body text, 3:1 for large text and meaningful graphics.
- **Never colour alone.** Every status carries text or an icon as well as a tint.
- **Keyboard.** Every interactive element reachable and operable, in a logical order, with a visible focus ring using `--color-focus`. This explicitly includes bed selection on the floor plan.
- **Screen reader.** Semantic landmarks; form fields with real labels; errors associated with their field; live regions for async results. A bed announces its label, its room, its floor and its state.
- **Targets.** Minimum 44×44 px for touch. Bed cells on a dense floor plan must meet this at mobile zoom or offer a list alternative.
- **Motion.** Respect `prefers-reduced-motion`; map pan and zoom must not be the only way to reach information.

THE SPATIAL MAP'S ACCESSIBILITY OBLIGATION

A floor plan is a graphical interface to information that some users cannot see. An equivalent **non-visual path to the same action** is mandatory, not a nice-to-have: a structured, keyboard-navigable list of floors, rooms and beds with states, from which a bed can be selected and the same enquiry submitted. If the only way to choose a bed is to click a shape on a canvas, the product excludes users it has no business excluding.

Designing for three audiences

AUDIENCE	DESIGN PRIORITY	WHAT TO AVOID
Owner	Answer "is the money right?" in one screen. Dense is acceptable; ambiguity is not. Every figure states its cutoff and inclusion rule.	Vanity metrics; totals whose definition is undocumented; charts that replace the statement.
Staff	Speed and error resistance on repeated tasks. Record a payment in one screen without losing context. Mobile parity.	Multi-step wizards for routine work; destructive actions adjacent to common ones.
Resident and prospect	Clarity on a phone, from a cold start, with no training. The floor plan carries the explanation.	Internal vocabulary — "allocation", "stay", "tenancy". Any exposure of another resident's data.

Implementation constraints

- Display components stay separate from domain calculation. The backend is the calculation authority; billing formulae are never duplicated in React.
- Hidden controls are not permissions. Every action is authorised server-side regardless of what the UI renders.
- A typed API client with shared validation and error mapping, generated from the reviewed contract.
- Cached data is invalidated after any payment, allocation, charge, deposit or assignment action.
- No identity document, financial record or secret in browser persistence. No secret in a build-time variable exposed to the client bundle.
- No optimistic UI on financial writes unless rollback behaviour is rigorously defined — showing a payment as saved before the server confirms is a correctness bug, not a responsiveness win.

Status. The token system is implemented. Every component, layout, screen and accessibility behaviour described in this chapter is `NOT IMPLEMENTED`. The repository's demonstration UI exercises the tokens and a handful of states against fictional fixtures; it is not the design system.

CHAPTER 7

Building the Interactive Spatial Map

Nook's distinguishing idea, in depth: the spatial hierarchy, the 2D floor plan, the hard rules about availability and concurrency, the accessible equivalent, and the migration path to a full-building 3D view.

SCOPE STATUS BEFORE ANYTHING ELSE

The spatial map does not appear in the approved requirements package. It is **new product direction introduced with this handbook** and has received no review or approval. Everything in this chapter is PROPOSED, and adopting it expands the pilot materially. That decision is recorded as open in Chapter 24.

7.1 Product purpose

A resident should be able to understand a property's layout and choose an individual bed based on where it actually is — subject to the property's approved booking policy. Staff should maintain that same spatial and availability data as part of their ordinary work, not as a separate marketing exercise.

In a shared room beds are not interchangeable. The bed by the window, the one beside the door, the upper bunk, the corner — these are materially different to the person who will live there for a year, and every incumbent product reduces them to identical rows in a dropdown. A floor plan collapses the explanation that a phone call would otherwise have to carry.

The operational half matters as much as the resident half. A manager looking at a floor plan sees occupancy as a shape — which rooms are full, where the vacancies cluster, which floor is emptying — in a way a list cannot convey. The same record serves both, which is the point.

7.2 Spatial hierarchy

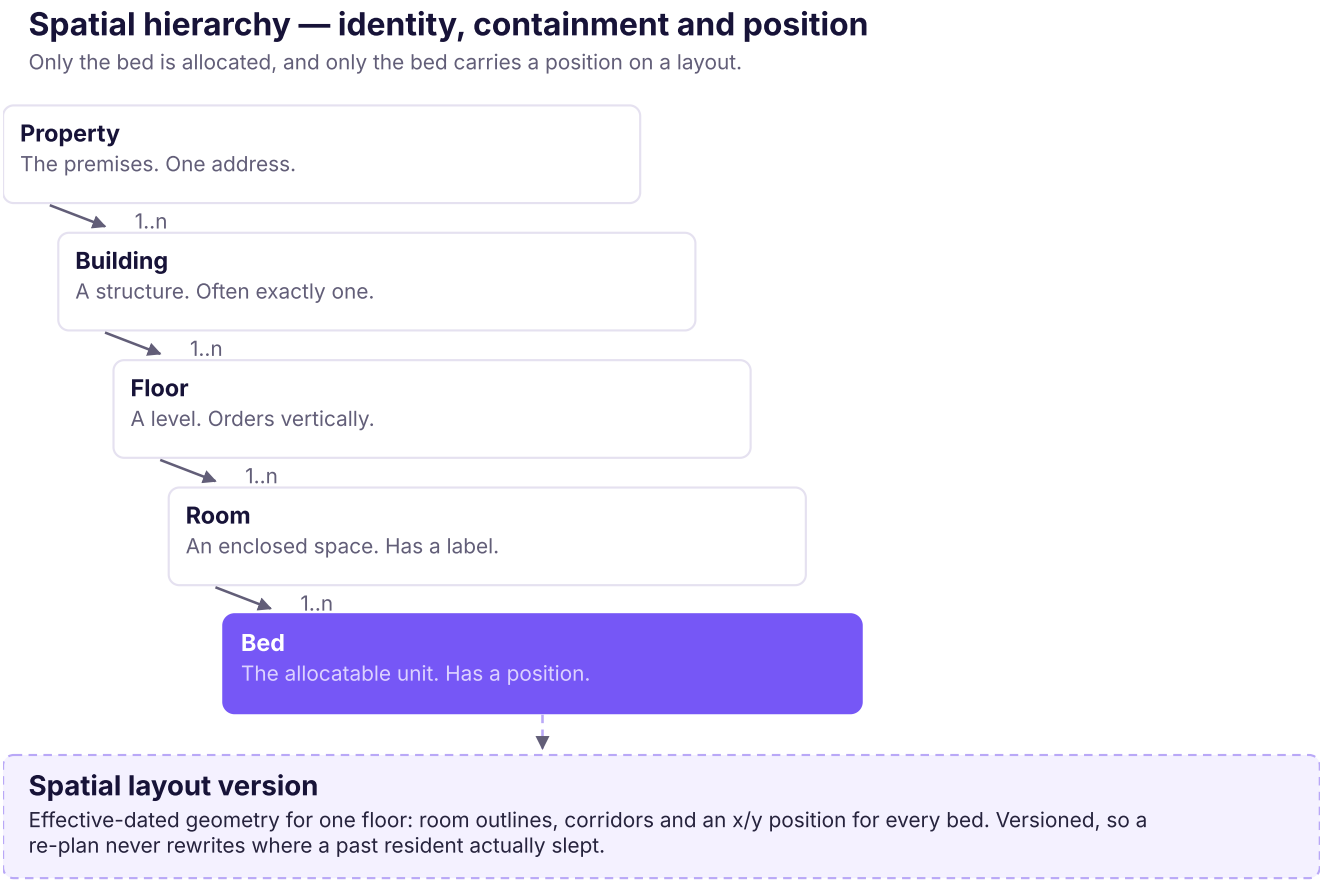


Figure 7.1 — The spatial hierarchy. Containment runs left to right. The bed is the only level that is allocated and the only level that carries a position; everything above it provides identity and context.

LEVEL	IDENTITY	RELATIONSHIP TO PHYSICAL POSITION
Property	One premises at one address, owned by one organisation.	None. A container.
Building	A structure within the property. Often exactly one; the level exists so a second does not require a data migration.	None, until a site plan is introduced.
Floor	A level within a building, with an ordering so "above" and "below" are meaningful.	Carries the layout. A floor is the unit the map renders.
Room	An enclosed space with a label the property already uses — "203", "Annexe 2".	An outline on the floor's layout.
Bed	**The allocatable unit.** Labelled within its room — A, B, C, D.	An x/y position within the room outline.

Identifiers and positions are separate concerns

A bed's identifier is stable and meaningful to staff: *Room 203, Bed C*. Its position is a pair of coordinates on a layout version. **Changing where a bed is drawn must never change which bed it is**, because the identifier appears on receipts, in the ledger and in the resident's agreement. Conversely, correcting a label must not silently move a resident.

WHY LAYOUTS ARE VERSIONED

Properties are re-planned. A room is split, a bed is added, a corridor is reconfigured. If the layout were mutable in place, a floor plan rendered for a stay in March would silently show the June arrangement, and the record of where someone actually slept would be lost.

A **spatial layout version** is therefore effective-dated. The current version drives today's map; a historical assignment resolves against the version in force at the time. This also makes re-planning safe: a new version is drafted and published rather than edited live under residents.

Open decision. Whether the allocatable unit is always a bed has not been confirmed (BR-LIF-002). Some properties let an entire private room as one unit; some have temporary extra beds. The model treats "one reservable place" as the atom and calls it a bed, but the real property must be surveyed before this is fixed in a schema.

7.3 The interactive 2D floor plan

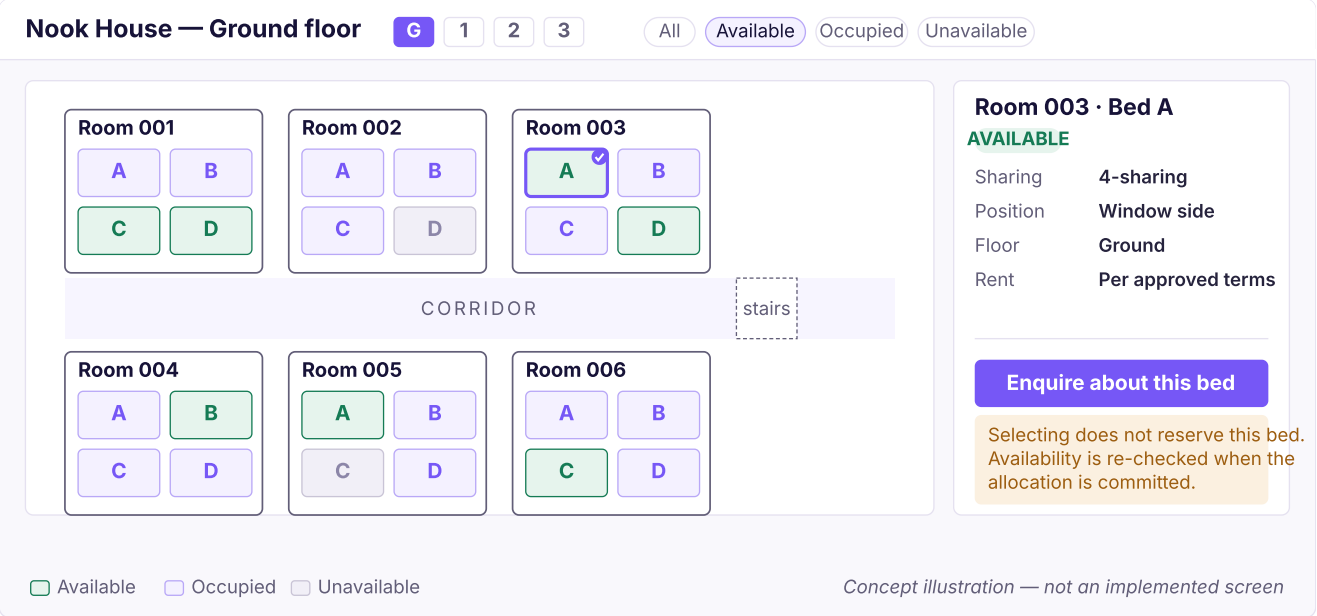


Figure 7.2 — 2D floor plan, concept. Top-down layout with rooms, corridor and individually positioned beds; floor switcher, status filters, selection state and a details panel. The panel states plainly that selection does not reserve. CONCEPT

What the view must do

CAPABILITY	REQUIREMENT
Top-down layout	Rooms as outlines with their real labels; corridors and circulation shown so orientation is possible; stairs and lifts marked where they affect desirability.
Positioned beds	Each bed drawn at its own coordinates inside its room, labelled, at a size that remains a legible touch target.
Selection state	Exactly one bed selected at a time, visually unambiguous, and distinct from hover and focus.
Floor switching	Direct switch between floors without losing filters. A floor with no rooms configured says so rather than rendering blank.
Room details	Selecting a room or bed opens a panel: sharing, position, floor, state and the permitted next action.

CAPABILITY	REQUIREMENT
Pan and zoom	Pointer drag and wheel on desktop, one-finger pan and pinch on touch, with bounds so the plan cannot be lost off-screen, plus a reset.
Search and filter	Filter by state and room attributes; search by room or bed label. Filtering dims rather than removes, so the spatial context survives.
Availability indication	Three states minimum — available, occupied, unavailable — carried by tint *and* text, never colour alone.
Loading and failure	Skeleton while loading. If live state cannot be fetched, the map says the data is stale and disables selection rather than showing a cached availability as current.

Mobile interaction

A prospective resident is almost certainly on a phone. One-finger pan, pinch zoom, tap to select, bottom-sheet details panel, and a floor switcher reachable with a thumb. Bed targets must meet the 44 px minimum at the default zoom for a typical room, or the view must open zoomed to one room at a time with a room-level overview above it.

Keyboard and screen-reader behaviour

This is a requirement, not an enhancement. A canvas full of clickable shapes is unusable without it.

- Floors, then rooms, then beds are reachable in document order. Arrow keys move between beds within a room; `Tab` moves between rooms.
- The focused bed carries a visible focus ring drawn outside its fill so it survives any status tint.
- Each bed announces its full context — *"Room 203, Bed C, window side, available, selected"* — not just its letter.
- Selection changes announce through a live region; the details panel is associated with the selection rather than merely appearing.
- Zoom and pan are never the only route to information. Everything reachable by panning is reachable by keyboard.

> **THE ACCESSIBLE LIST IS A PEER, NOT A FALLBACK**
>
> Alongside the plan, a structured list of floors → rooms → beds with states supports the same selection and the same enquiry. It is built from the same service, shipped in the same release, and tested in the same suite. It is not a degraded mode to be added later — if the only way to choose a bed is to click a shape, the product excludes people it has no reason to exclude.

Administrative layout editing

Staff maintain the layout. The editor places room outlines and bed positions on a floor, and it is the same data the resident-facing map reads. Required behaviours:

- Draft and publish. Edits accumulate in a draft version; publishing makes it current from an effective date.
- **Occupied beds are protected.** A bed with an active assignment cannot be deleted. It can be repositioned — moving a drawing is not moving a person — but removing it requires the assignment to end first.

- Validation refuses overlapping beds, beds outside their room's outline, duplicate labels within a room, and rooms overlapping other rooms.
- A floor with unplaced beds is publishable but flagged: those beds exist and are allocatable, they simply have no position yet, and the map must say so rather than omit them.

Open decision. What happens to an occupied bed when a room is re-planned out of existence is not decided and must not be invented. The system refuses the destructive operation and surfaces the conflict (BR-LIF-004, BR-LIF-007).

7.4 Availability and concurrency

The map must never become the authority for inventory or reservations. This is the single most important engineering rule in the chapter.

A rendered floor plan is a photograph of availability at the moment the page was built. Between that moment and a committed allocation, anything can happen — another resident selects the same bed, a staff member allocates it directly, a layout is republished. A product that treats the picture as the truth will double-book somebody, and the first person to discover it will be a resident standing in a doorway.

CONCEPT	WHAT IT IS	WHAT IT IS NOT
Displayed availability	A derived, cacheable status computed when the view was rendered. Advisory, timestamped, may be stale.	Not a reservation. Not a promise. Not a lock.
Selected bed	Client-side intent. The resident's preference, carried into an enquiry.	Not a hold. Confers no claim. The bed stays available to everyone else.
Confirmed allocation	A committed, effective-dated assignment of one bed to one stay, written inside a serialised transaction after a fresh availability check.	Not reversible by a page refresh. The only state that means anything.

What the backend must guarantee

1. **Re-check at commit.** Availability is evaluated again inside the allocation transaction, against current records — never against the state the client believed.
2. **Serialise conflicting writes.** Two concurrent allocations for the same bed over overlapping dates: exactly one succeeds. The other receives a conflict response carrying current state.
3. **Enforce in the database.** Overlap prevention is a constraint, not only application logic, so a code path that forgets the check still cannot create the conflict.
4. **Atomic with its side effects.** The assignment, the bed state and the stay transition commit together or not at all.
5. **Idempotent.** A retried confirmation after a client timeout returns the original allocation rather than creating a second.

NO PRE-CHECKOUT HOLD WITHOUT EXPLICIT AUTHORISATION

A hold — reserving a bed for N minutes while someone completes a form — is **not** part of this design and must not be introduced on implementation judgement. It changes occupancy economics, because a held bed earns nothing and a hold that expires badly loses a resident. Whether holds exist, how long they last and whether money is taken to create one are the owner's decisions (BR-LIF-003), and all three are OPEN .

The user-facing consequence

Losing a bed between selection and confirmation is a *normal* outcome, not an error condition, and must be designed rather than handled. The resident is told plainly that the bed was taken, returned to the map with current state, and shown comparable alternatives — the same room, the same floor, the same sharing. They are never silently reassigned to a different bed than the one they chose.

7.5 The future 3D building map

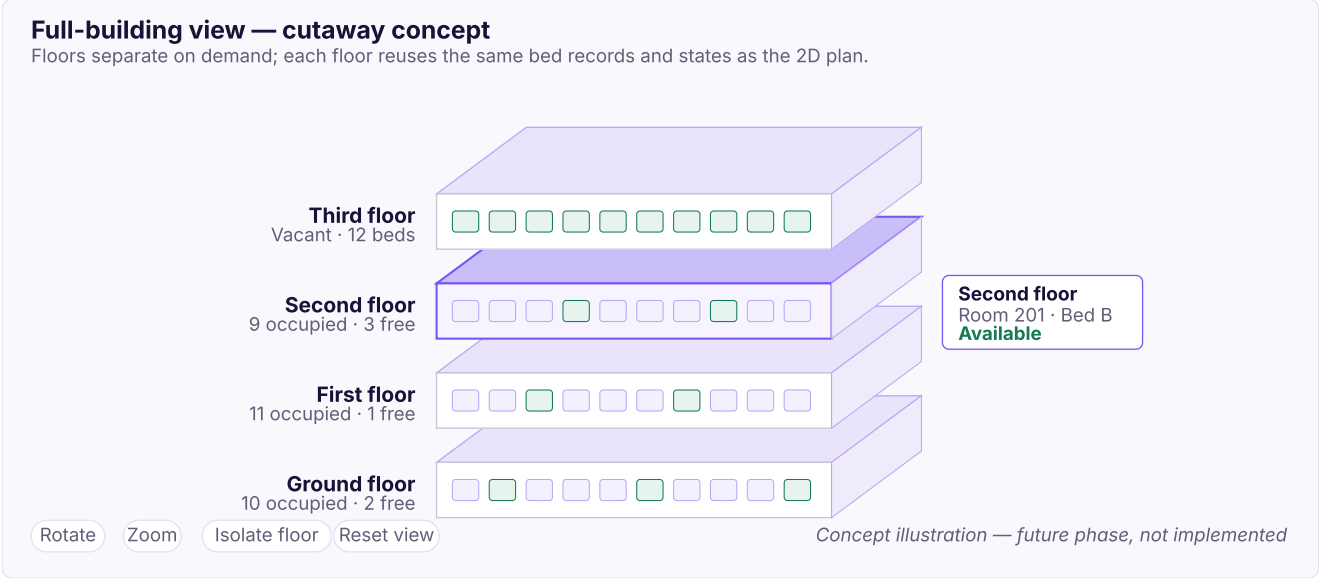


Figure 7.3 — Full-building cutaway, concept. Floors separate on demand with per-floor occupancy; selection and highlighting carry through to the same details panel as the 2D view. CONCEPT · FUTURE PHASE

The long-term requirement is a full-building dollhouse or cutaway view showing multiple floors, corridors, rooms and individual beds together — so a resident understands the building, not just a floor, and an owner sees occupancy across the whole property at once.

REQUIREMENT	DETAIL
Floor visibility controls	Isolate, hide, explode or stack floors; the 2D plan remains reachable from any floor.
Camera navigation	Orbit, pan, zoom, with constrained bounds and a reset. No free-fly camera.
Selection and highlighting	Beds selectable in 3D, resolving to the same record and the same details panel as 2D.
Occupancy states	The same three states, the same tokens, legible at building scale and not relying on colour alone.
Responsive behaviour	Degrades to the 2D plan on low-capability devices rather than rendering slowly.
Accessible alternative	The same structured list. 3D adds no information that exists only in 3D.

REQUIREMENT	DETAIL
Performance	Budgeted against a mid-range Android phone, not a developer laptop. Level-of-detail and texture compression assumed necessary.

Migration path from 2D to 3D

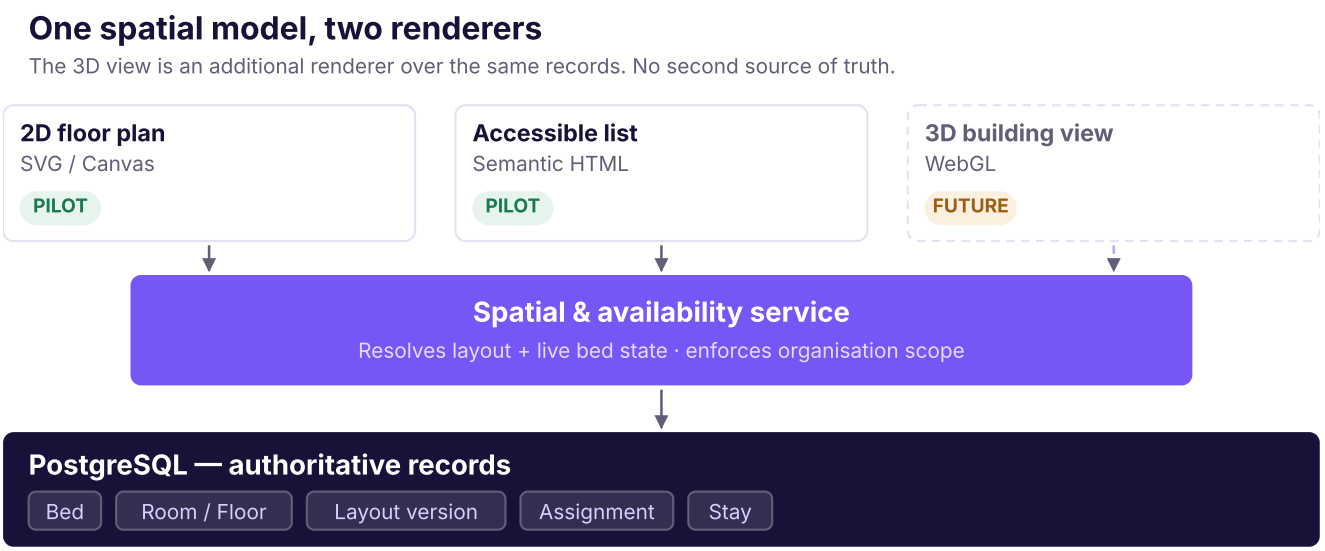


Figure 7.4 — One spatial model, two renderers. The 3D view is an additional renderer over the same service and the same records. It introduces no second source of truth and no second availability path.

The migration is deliberately undramatic, and that is the design goal. 2D ships first. The layout model already carries per-floor geometry and per-bed positions; 3D adds floor elevation and room height and reads everything else unchanged. Availability resolution, permission checks and organisation scoping live in the service beneath both renderers, so the 3D view cannot develop its own notion of what is free.

The requirement must not quietly disappear. 2D first and 3D later is a sequencing decision, not a cancellation. Chapter 22 carries 3D as a named roadmap phase with its own gate.

Rendering approaches — for approval, not for selection

The comparison below is documented so a decision can be made. **No library has been selected, added as a dependency, or integrated.** Figures are drawn from current public comparison material and are approximate; the one source consulted contradicted itself on bundle sizes, which is itself a reason to re-measure rather than quote.

APPROACH	FIT	TRADE-OFFS	BUNDLE
SVG / Canvas 2D For the 2D plan	Native to the DOM, so keyboard focus, screen-reader semantics and CSS tokens work without reimplementation. Sufficient for floor-level geometry at pilot scale.	Does not extend to 3D. Very large plans need canvas rather than SVG.	None
React Three Fiber over three.js	Declarative React integration, so the 3D view composes with the rest of the app rather than living in an imperative island. Helper ecosystem suits architectural staging.	Still requires three.js concepts. Accessibility must be built deliberately — WebGL contributes no semantics.	~150–200 KB tree-shaken for simple scenes, per the source

APPROACH	FIT	TRADE-OFFS	BUNDLE
Babylon.js	Stronger asset pipeline and inspector for large or complex environments and walkthroughs.	Largest API surface and bundle; React integration via a separate wrapper rather than natively.	~300 KB minimal, substantially more with extras
Pre-rendered imagery	Cheapest route to a convincing visual.	**Rejected as a product answer.** Not interactive, drifts from real inventory, and cannot carry live availability — reintroducing exactly the two-sources-of-truth problem the map exists to remove.	—

Recommendation for approval. SVG for the 2D plan in the pilot; React Three Fiber as the candidate for the later 3D phase, re-measured against a real scene before any dependency is added. Licensing, accessibility strategy and a performance budget on a mid-range device must each be confirmed at that point rather than assumed now. PROPOSED

Source consulted: PkgPulse — three.js vs React Three Fiber vs Babylon.js (2026). Treat the figures as indicative only.

Test criteria for this chapter

TEST	PASS CONDITION	CLASS
Concurrent allocation of one bed	Exactly one success; the other gets a conflict with current state	Invariant
Allocation against stale displayed availability	Refused at commit; no partial write	Invariant
Retried confirmation after timeout	Returns the original allocation; no duplicate	Invariant
Selection creates no server state	No hold, no lock, no record written	Invariant
Public map exposes no resident data	No name, contact, document or financial field in any public response	Invariant
Keyboard-only bed selection	A bed can be found, selected and enquired about with no pointer	Invariant
Screen-reader announcement	Bed announces room, floor, position and state	Invariant
Occupied bed deletion	Refused with the blocking assignment named	Invariant
Overlapping geometry	Layout publish refused with the offending objects identified	Invariant
Historical layout resolution	A past assignment renders against the version in force then	Invariant
Hold behaviour	*Cannot be tested.* No hold policy exists (BR-LIF-003)	Policy

Status. None of this exists. The repository's demonstration UI renders a fictional four-floor property from static fixtures with no positions, no layout version, no availability service and no persistence. It demonstrates the token system and a filter interaction; it is not module 5.

PART III

Domain modules

The five functional areas that carry the business: inventory, booking, residents, money and operations.

8 Property, Building and Inventory Management
9 Booking, Admission and Allocation
10 Resident and Tenant Management
11 Financial Management
12 Operations and Property Services



CHAPTER 8

Property, Building and Inventory Management

How a property gets into Nook, how its physical inventory is defined and maintained, and what happens when the building changes while people are living in it.

Property onboarding

Onboarding is the first thing a new customer does and the thing that most determines whether they continue. It must be completable by the owner without a developer.

1. Organisation is created and the owner's account provisioned. No public self-service registration in the pilot.
2. Owner creates a property: name, address, timezone and the operational settings the hostel actually uses.
3. Buildings and floors are defined, with a floor ordering so "above" and "below" are meaningful.
4. Rooms are created with the labels the property already uses — not labels Nook invents.
5. Beds are created within rooms and labelled.
6. Optionally, beds are positioned on a floor layout (Chapter 7).
7. Staff are invited and granted permissions per property.
8. Opening resident and financial data is migrated under reconciliation (Chapter 20).

> **USE THE PROPERTY'S OWN VOCABULARY**
>
> If the hostel calls a room "Annexe 2", Nook calls it "Annexe 2". Renaming a property's inventory to fit a tidy scheme guarantees that staff maintain a mental translation table, and the translation will be wrong at exactly the wrong moment. Labels are free text within an organisation-scoped uniqueness rule.

Rooms, beds and the allocatable unit

The **allocatable unit** is the thing a resident occupies and the thing availability is computed for. The model treats it as a bed, including for single-occupancy private rooms, so that availability logic stays uniform rather than branching on room type.

This must be validated against the real property before it is fixed in a schema. Open questions (BR-LIF-002, BR-003): does the property let entire rooms as single units; are there temporary or extra beds; are there non-bed spaces that are nevertheless let?

ATTRIBUTE	PURPOSE	CONSTRAINT
Label	How staff refer to it; appears on receipts and agreements	Unique within its room
Room	Containment and sharing count	Required; same organisation
Position	Where it is drawn on the layout	Optional; within the room outline

ATTRIBUTE	PURPOSE	CONSTRAINT
Operational state	Whether it can be allocated at all	See state table below
Attributes	Window side, upper bunk, accessible — the things that make beds non-interchangeable	Property-defined OPEN

Bed states

Two things are deliberately separate: whether a bed is *operationally usable*, and whether it is *currently occupied*. Conflating them makes "under maintenance" indistinguishable from "someone lives here", which produces wrong occupancy reports.

STATE	MEANING	ALLOCATABLE?
Available	Usable and unoccupied for the period in question	Yes
Occupied	An active assignment covers the period	No
Blocked	Deliberately withheld by the owner — staff room, storage, held for a returning resident	No
Under maintenance	Temporarily unusable	No
Reserved	Held pending a decision	ONLY IF HOLDS ARE AUTHORISED — BR-LIF-003

Occupancy is **derived from effective-dated assignments**, not stored as a flag on the bed. A stored flag drifts the moment a checkout is recorded late, and the drift is invisible. Operational state is stored, because it reflects a decision rather than a fact about assignments.

Layout editing and existing occupancy

Layout changes are the main way inventory management can hurt a resident, so the rules are conservative.

CHANGE	WITH NO ACTIVE ASSIGNMENT	WITH AN ACTIVE ASSIGNMENT
Reposition a bed on the layout	Allowed	Allowed — moving a drawing is not moving a person
Rename a bed or room	Allowed, audited	Allowed, audited, and the resident's historical records retain the label in force at the time
Change room sharing count	Allowed	Allowed only if it does not reduce capacity below current occupancy
Set a bed to Blocked or Maintenance	Allowed	**Refused.** End the assignment first.
Delete a bed	Allowed if never assigned; otherwise deactivate	**Refused**, naming the blocking assignment
Delete a room or floor	Allowed if empty	**Refused**, naming the blocking beds

NO DESTRUCTIVE BEHAVIOUR IS INVENTED FOR OCCUPIED INVENTORY

What *should* happen when a property genuinely re-plans a room that people are living in — a forced reallocation, a notice, a manual migration — is a business decision that has not been made. The system refuses the destructive operation and surfaces the conflict with the specific blocking records. It does not cascade, does not auto-reassign and does not soft-delete the resident's history (BR-LIF-004, BR-LIF-007). OPEN

Deactivation rather than deletion

Inventory that has ever been assigned is deactivated, never removed. A deactivated bed stops appearing in availability and in new allocations, but remains resolvable so that a historical stay, charge or receipt referring to it still renders correctly. Deleting a bed that a receipt references breaks the receipt.

Inventory validation

- Duplicate labels within their uniqueness scope are refused.
- A bed must belong to a room, which must belong to a floor, in one building, in one property, in one organisation — the whole chain is verified, not just the immediate parent.
- Layout geometry is validated on publish: no overlapping beds, no bed outside its room, no overlapping rooms.
- Beds without positions are permitted and flagged — they exist and are allocatable, they simply are not drawn yet.
- Capacity consistency: a room's sharing count and its bed count must agree, or the discrepancy is surfaced rather than silently reconciled.

Property settings and staff assignment

Settings that belong to a property rather than the organisation: billing parameters once approved, grace period, deposit policy, receipt numbering series, notice period, and which operational modules are enabled. Every one of these is currently OPEN pending owner decisions.

Staff are granted access **per property**, not per organisation. A manager responsible for one of three properties sees one. This is a grant inside a tenancy, not a tenancy of its own — Chapter 13 draws the distinction precisely.

Multi-property ownership

The pilot has one property. The model supports many from the first schema because adding the relationship later means migrating live data. A second property for an existing owner requires no developer involvement: create the property, configure it, build its inventory, grant staff. Reports roll up across properties for the owner and stay scoped per property for staff.

Change history

Every inventory change records actor, timestamp, before and after state, and the request that caused it. Assignment history is append-only, so a bed's full occupancy history is reconstructable for any past

date. This is what makes "who was in bed 203-C last March?" answerable — a question that arises in deposit disputes and in police verification enquiries.

Permissions, API and tests

Permissions	`inventory.read`, `inventory.write`, `assignment.manage`, `property.manage`. Warden has scoped read; finance has read only where granted.
API	Properties, rooms and beds as nested resources; bed availability as a query with explicit date semantics; layout versions as draft-then-publish. Full inventory in Chapter 17.
Tests	Concurrent allocation yields one success; deleting an occupied bed is refused; a deactivated bed still resolves on a historical receipt; overlapping geometry is refused on publish; cross-organisation inventory access returns not-found.
Release phase	Pilot. This module gates every other domain module.
Status	NOT IMPLEMENTED

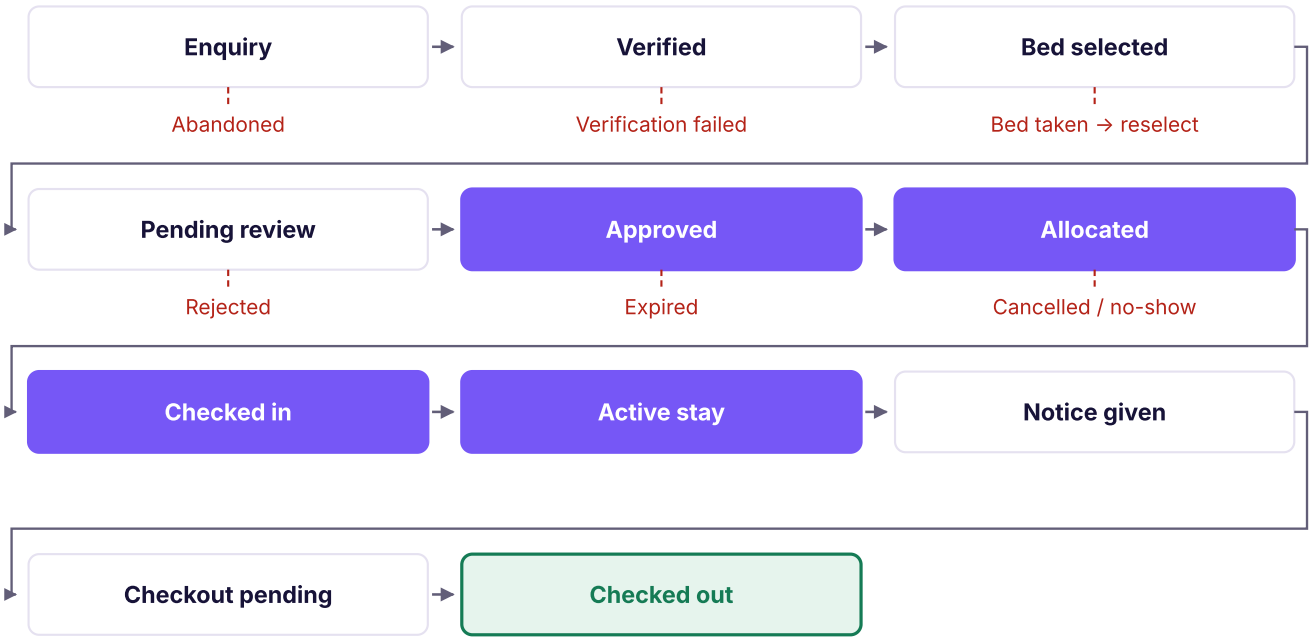
CHAPTER 9

Booking, Admission and Allocation

From enquiry to a committed bed: the state machine, the atomic allocation that makes double-booking impossible, and the many date and policy questions that remain unanswered.

Booking and allocation — state machine

Violet states are the committed path. Dashed exits depend on rules that are still open.



Financial settlement is a separate workflow
Checked out is an operational state. It changes no balance, nets no deposit and releases no bed until the approved effective-date rule exists.

Figure 9.1 — Booking and allocation states. The committed path runs along the top and folds into the stay lifecycle. Terminal outcomes sit below. Dashed transitions depend on rules that have not been decided, and those transitions are not implementable until they are.

The pilot's admission model

MANUAL APPROVAL, CONFIRMED. INSTANT ALLOTMENT, FORBIDDEN.

The pilot property admits residents through a **paper application and manual approval**. CONFIRMED Nook reproduces that: an enquiry reaches staff, staff decide, and only then is a bed allocated.

Instant allotment must not be enabled for this property without explicit owner approval. A ladies' hostel approving residents by hand is making a screening decision, not performing an administrative formality. Automating it away because the software supports it would remove a control the owner relies on.

Enquiry and eligibility

An enquiry captures who is asking, how to reach them, when they want to move in, and which bed they are interested in. It creates no stay, no charge and no claim on inventory.

Eligibility criteria — gender, student or working status, minimum duration, age, documentation — are **property policy, not product logic**. The pilot property is a ladies' hostel, which is itself an eligibility rule. Nook records criteria the owner defines and surfaces mismatches to staff; it does not auto-reject, because an automatic rejection on a protected characteristic is a decision no product should make unsupervised. OPEN

Phone verification

Verification establishes that an enquirer controls the number they gave, which reduces nuisance enquiries and makes follow-up possible.

CONCERN	REQUIREMENT
Provider	MSG91 remains a **candidate**, not an approved or active integration. No credentials exist. CANDIDATE
Cost	Each message costs money and the cost is borne per enquiry, including from people who never become residents. Rate limits are an economic control, not only a security one.
Abuse	Per-number, per-IP and global rate limits; code expiry; attempt caps; no enumeration of which numbers exist.
Privacy	A phone number is personal data collected for a stated purpose. Retention for unconverted enquiries must be defined. OPEN
Fallback	If the provider is unavailable, staff can mark an enquiry verified manually, audited. Verification must not become a single point of failure for admissions.

Bed selection

Selection is covered in Chapter 7 and bears restating in the booking context, because this is where the rule actually bites.

Selecting a bed does not hold it. The selection travels with the enquiry as a preference. Between selection and approval the bed remains available to everyone, and may be taken. That outcome is designed for — the resident is told, returned to current state and offered comparable alternatives — rather than treated as an error.

A pre-checkout hold is not part of this design and **must not be introduced without explicit owner authorisation** (BR-LIF-003). OPEN

Approval and atomic allocation

Approval is where intent becomes commitment, and it is the most concurrency-sensitive operation in the product.

1. Staff review the enquiry and confirm or change the requested bed, seeing **live** state rather than the state at enquiry time.
2. The server opens a transaction and takes a lock on the bed.

3. It re-checks availability for the requested period against current assignments — not against anything the client supplied.
4. It verifies the resident record, the bed, the organisation scope and the legality of the state transition.
5. It writes the assignment, updates bed state and transitions the stay — all inside the one transaction.
6. It records the idempotency key with the outcome, so a retry returns this result rather than repeating the work.
7. It commits. On any failure, nothing is written.

> **ENFORCED TWICE, DELIBERATELY**
>
> Overlap prevention lives in the application logic *and* as a database constraint. The application check gives a good error message; the constraint guarantees correctness even when a future code path forgets to check. A guarantee that depends on every developer remembering is not a guarantee.

Duplicate and conflicting requests

SITUATION	BEHAVIOUR
Two staff approve the same enquiry simultaneously	One succeeds; the other receives a conflict showing the allocation that won.
Two enquiries approved for the same bed, overlapping dates	One succeeds; the other is refused with alternatives offered.
Client times out after the server committed	Retry with the same idempotency key returns the original allocation. No second assignment.
Same resident enquires twice for the same bed	Detected and surfaced to staff. Not auto-merged — two enquiries may be deliberate.
Approval arrives after the bed was blocked for maintenance	Refused. Operational state outranks a pending approval.

Check-in, extensions and room changes

Check-in records actual arrival and activates the stay. Whether the stay's billing start equals the allocation date, the check-in date or an agreed date is OPEN (BR-LIF-005, BR-REN-003) and must not be assumed — the three differ for a resident who arrives three days late, and the difference is money.

Extensions lengthen a stay. The extension must re-check that the bed is free for the additional period; an extension into a period already allocated to someone else is refused.

Room and bed changes are an atomic re-allocation: the old assignment ends the day before the effective date, the new one begins on it, both inside one transaction, and both remain in history. The old bed shows the resident for its real dates. If the new bed's rate differs, the current charge is **not** edited — a pro-rata adjustment is written instead, with a reason, and only once the rule governing it is approved (BR-LIF-007). OPEN

Cancellation, expiry and no-show

All three are OPEN, and all three involve money, which is why none may be given a plausible default.

EVENT	UNANSWERED QUESTIONS	RULE
Cancellation	Before check-in: is anything retained? Does it depend on notice? Who may waive?	BR-LIF-012
Expiry	Does an unapproved enquiry lapse? After how long? Does the resident get told?	BR-LIF-003
No-show	An approved, allocated resident never arrives. How long is the bed held? Is anything charged?	BR-LIF-012

Until these are answered the corresponding transitions are **not implemented**. An unapproved enquiry stays pending and visible to staff rather than silently expiring, which is the conservative behaviour: a stale queue item is a nuisance, whereas an auto-expiry that loses a resident, or an auto-charge nobody authorised, is a real failure.

Check-out

Covered fully in Chapters 10 and 11, and stated here because this is where the booking lifecycle ends and the most common design error occurs.

> **OPERATIONAL CHECKOUT AND FINANCIAL SETTLEMENT ARE DIFFERENT THINGS**
>
> Recording that a resident has left is an **operational** event. It transitions the stay, preserves history and stops future charge generation beyond the vacate date. It does **not** compute a final balance, net a deposit against dues, issue a refund or release the bed.
>
> Bed release happens on the owner-approved operational effective date (BR-LIF-004, BR-LIF-011) — which is not decided, so release is not automated. Settlement is a separate workflow with its own permission, its own approval and its own audit trail, and it is blocked until the rules in Chapter 11 are signed off.

Date interval semantics

Every assignment and every charge period has a start and an end, and the system's correctness depends on knowing precisely what those mean. **None of this is decided.**

- Is the end date inclusive or exclusive? A resident leaving "on the 30th" either does or does not occupy the bed that night, and the next resident either can or cannot arrive that day.
- Can a new assignment start the same day a previous one ends, or must there be a clear day?
- Are periods compared as dates in the property's timezone, or as timestamps? Billing dates must be date-only values in Asia/Kolkata; a server running UTC that treats them as timestamps will bill the 1st on the 31st.
- What is a "month" for a resident admitted on the 31st?

These are BR-REN-002, BR-REN-006 and BR-LIF-004. Until they are answered, overlap detection cannot be specified exactly, and a constraint written against a guess will be subtly wrong in a way that only shows up at month boundaries.

Permissions, API and tests

Permissions	`stay.manage` for enquiry, approval, check-in and operational checkout; `assignment.manage` for allocation and room change. Settlement requires `settlement.approve`, which is blocked.
API	Business actions, not status edits: `move-in`, `room-change`, `notice`, `checkout`. Every financial-adjacent write requires an idempotency key.
Tests	Concurrent allocation; allocation against stale availability; retried confirmation; extension into an allocated period; room change preserving both assignments; checkout changing no balance.
Release phase	Pilot, with enquiry and verification proposed as post-pilot.
Status	NOT IMPLEMENTED

CHAPTER 10

Resident and Tenant Management

The person, their stays and their history — held separately so none of the three overwrites another — plus the privacy obligations that come with holding them.

Two meanings of "tenant"

> **A TERMINOLOGY TRAP WORTH DISARMING EARLY**
>
> In SaaS architecture, a **tenant** is a customer organisation — the hostel business whose data is isolated from every other business on the platform. In accommodation, a **tenant** is the person living in the room.
>
> This handbook avoids the collision by never using the word for a person. The organisation is an **organisation**; the person is a **resident**. Chapter 13 covers the first; this chapter covers the second.

Resident identity is separate from a stay

A **Resident** is a person. A **Stay** is one period during which that person occupied a bed at a property, with its own start, end, terms and assignment history. One resident may have several stays over the years.

Keeping them separate is what makes a repeat admission safe. If the resident record carried the current bed and dates directly, re-admitting someone would overwrite the record of their previous stay — and with it the context for every historical charge and receipt. Separation means a returning resident is recognised rather than duplicated, and their earlier history survives intact.

ENTITY	HOLDS	LIFETIME
Resident	Name, contact, emergency contact, approved identifying fields	Persists across stays; retained after departure per policy
Stay	Property, start and end dates, lifecycle state, rental terms reference	One admission to one departure
Assignment	Which bed, over which dates, with actor and reason	Append-only; many per stay if rooms change
Lifecycle event	Each transition with actor and timestamp	Append-only, permanent

Resident profile

The field set is governed by one principle: **collect only what a named workflow uses.** Every field has a cost — in staff entry time, in breach exposure, and in the retention obligation it creates.

FIELD GROUP	JUSTIFICATION	STATUS
Name, contact number	Required to operate at all — identify a resident and reach them about arrears	PROPOSED
Email	Receipts and statements where the resident prefers it	PROPOSED

FIELD GROUP	JUSTIFICATION	STATUS
Permanent address	Commonly collected; purpose should be stated rather than assumed	OPEN
Emergency contact	Defensible for a residential property. Confirm it is wanted and who may view it.	OPEN
Occupation — student or working	Only if it drives a policy difference. If it does not, do not collect it.	OPEN
Identity document numbers	**Not collected by default.** See below.	EXCLUDED

Documents and verification

NO BINARY DOCUMENT UPLOAD IN THE PILOT

Identity documents for roughly ninety-five women are the highest-consequence data this product could hold, and the least necessary for the problem the owner is paying to solve — which is knowing who owes what.

Upload is therefore **excluded from the default pilot scope**. It is included only if the owner demonstrates it is necessary for safe initial operation, and only after categories, purpose, permitted viewers, lawful basis, resident notice, file constraints, malware handling, private storage, link expiry, audit and the retention schedule are each approved (BR-SEC-004, BR-SEC-005, BR-OPS-003).

Answering "no" costs nothing that cannot be added later. Answering "yes" opens a substantial amount of work and a permanent obligation.

Where documents are eventually approved: binaries live in private object storage, never a public bucket; metadata and access policy live in the database; every download is authorised server-side before a short-lived signed URL is minted; and `document.read` is a **separate permission** that `resident.read` never implies. Automated identity verification and e-signature integration are deferred pending necessity and professional review.

Stay lifecycle

States and transitions follow the booking machine in Chapter 9. What matters here is what is preserved across them.

- Every transition is an explicit event with an actor and a timestamp, not a mutated status field.
- A stay never loses its assignment history. A resident who moves twice has three assignment records and the full picture of where they were when.
- Checkout does not delete, archive or hide the stay. The resident's financial history remains attached and queryable.
- An outstanding balance at checkout is flagged to a recovery list rather than written off. Writing it off is a financial decision with its own approval.

Room changes

A room change is an atomic re-allocation (Chapter 9) whose resident-facing requirement is simple: **history must show where the resident actually was on any given date.** The old bed shows them for its real dates. A statement for a past month resolves against the bed and the layout version in force then, not today's arrangement.

Whether rent changes on a move, and from when, is OPEN (BR-LIF-007). Until it is answered, a room change makes no charge adjustment at all.

Resident communication

Communication in the pilot is a **list, not an automation**. The system produces a reminder candidate list based on approved due dates and outstanding amounts; staff act on it by phone. Automated sending requires verified contact data, consent where applicable, template approval, delivery tracking, idempotent job handling and a per-message budget — all of which are post-pilot (Chapter 14).

> **AN ARREARS MESSAGE IS SENSITIVE**
>
> A reminder discloses that a named person owes money. Sent to a stale number, it discloses that to a stranger. Recipients are therefore re-resolved at send time rather than taken from a queued payload, and amounts are kept out of message previews and lock-screen notifications.

Privacy controls

CONTROL	REQUIREMENT
Minimisation	Only fields a named workflow uses. A field nobody reads is a liability with no benefit.
Access separation	Profile, financial and document access are three distinct permissions. Warden has none of the latter two by default.
No public exposure	No resident name, contact, document or balance is reachable through any public page, the shared map or any resident-facing API. The public surface serves a separate projection.
Resident sees only their own	When a resident portal exists, it resolves to the authenticated resident's own records only — never by a supplied identifier.
Search discretion	Result lists carry the minimum useful identifiers. Document numbers never appear in a list.
Audit	Access to documents and exports is logged with actor, target and time — without logging the content itself.

Data correction, retention and deletion

Correction of ordinary profile data is a permissioned, audited edit. **Correction never retroactively alters an issued financial record** — a corrected name does not rewrite a receipt; the receipt retains what was true when issued.

Retention is OPEN and requires professional confirmation (BR-OPS-003, BR-OPS-004). The questions are concrete: how long after departure are resident records kept; which categories differ; how does a

resident request access, correction or deletion; and how does deletion interact with financial records that may need to be retained for accounting or tax reasons.

> **DELETION AND FINANCIAL HISTORY CONFLICT, AND THE CONFLICT IS NOT OURS TO RESOLVE**
>
> A deletion request may collide with an obligation to retain accounting records. The product must be able to express both — removing or pseudonymising personal data while preserving the financial record's integrity — but **which** data is removed, and after how long, is a legal determination. Qualified advice is required before any retention job is written. Nothing is deleted automatically in the meantime.

Deactivation and archival

A departed resident is not deleted. The record becomes inactive: excluded from operational lists and occupancy, retained for history, and resolvable so past charges and receipts still render. Re-admission reuses the record where the person is recognised — whether the hostel wants one record per person or one per admission is `OPEN`, and it changes how duplicates are matched.

Permissions, API and tests

Permissions	`resident.read`, `resident.write`, `stay.manage`. `document.read` and `document.manage` separately, granted to nobody by default.
API	Residents and stays as nested resources; lifecycle changes as business actions; search returning a minimised projection.
Tests	Repeat admission retains the earlier stay; a profile edit alters no issued record; a user with `resident.read` but not `document.read` cannot reach document content by any route; no resident field appears in any public response; cross-organisation resident access returns not-found.
Release phase	Pilot. Documents excluded; resident portal deferred.
Status	`NOT IMPLEMENTED`

CHAPTER 11

Financial Management

The part of Nook the first customer is actually paying for. The record model, every money workflow with its actors and error cases, and a precise account of which policies remain undecided and what each one disables.

NOOK RECORDS MONEY; IT DOES NOT HOLD MONEY

Residents pay the property directly, in cash, by UPI or by bank transfer. CONFIRMED Nook records what arrived and reconciles it against what was owed. It takes no custody of funds, operates no float and processes no payment. A payment gateway is excluded from the pilot and would change the regulatory position materially if introduced.

The record model

Six record types — and the one derived number

Nothing posted is ever edited or deleted. The balance is computed, never stored as truth.

Charge
A claim for a period. Amount frozen at issue.

Payment
Money that arrived. Method + reference.

Allocation
Links one payment to one charge, for an amount.

Adjustment
Signed correction with reason and approver.

Reversal
Negates a posted record. Original stays visible.

Deposit
A liability. Never allocated to a charge.

Outstanding balance — derived every time

```
Σ charges + Σ debit adj − Σ allocations − Σ credit adj
```

Money is integer paise in BIGINT — never binary floating point

API carries amountMinor as a base-10 integer string with an explicit currency: {"amountMinor":"123450","currency":"INR"} = ₹1,234.50

Figure 11.1 — The financial record model. Six immutable record types and one derived number. Allocation is its own record rather than a field on either side, which is what makes a balance explainable.

RECORD	PURPOSE	MUTABLE AFTER POSTING?
Charge	A claim on a resident for a billing period. Amount frozen at issue from the terms version then in force.	No
Payment	Money that arrived. Carries method, reference and whether it was staff-recorded or independently confirmed.	No
Allocation	Links one payment to one charge for an amount. Many-to-many both ways.	No
Deposit	A liability held against a resident. Never revenue, never allocated to a charge.	Status only

RECORD	PURPOSE	MUTABLE AFTER POSTING?
Adjustment	A signed correction with a reason and an approver — waiver, discount, damage charge, pro-rata.	No
Reversal	Points at a posted record and negates it. The original remains visible.	No

Five rules that make the model work

1. **Money is `bigint` paise.** Never float, never rupees in storage. A branded type plus a build-breaking lint rule on raw numbers in money positions. The API carries `amountMinor` as a base-10 integer string with explicit currency; conversion happens once, at the boundary.
2. **Nothing posted is updated or deleted.** Corrections are new rows. A charge's paid status is derived from its allocations, not stored as a mutable truth.
3. **Balance is computed, never cached as truth.** A materialised value may exist for list performance, but it is rebuilt from source and reconciled nightly; a mismatch raises an alert rather than being silently corrected.
4. **Allocation is constrained on both sides.** Allocations against a payment cannot exceed its allocatable amount; allocations against a charge cannot exceed its total. Enforced inside a serialised transaction *and* by database constraints.
5. **Idempotency is a column, not a convention.** Unique keys on the billing event and on the payment reference. A duplicate is rejected by the database, not by a prior read that can race.

> **THE DEPOSIT IS THE MOST COMMONLY MIS-MODELLED THING IN THIS CATEGORY**
>
> A deposit is **not** a payment against a charge. It is a separate liability with its own lifecycle, and it must be excluded from every collection and revenue figure *by construction* — not by a filter someone has to remember to apply. Recording it as a payment because that was the quickest path at admission makes every financial report wrong, quietly, from the first week.

Workflow 1 — Rent and recurring charges

Actors	System on a schedule. No human initiates routine billing.
Inputs	Active rent schedule, terms version in force, billing period, due date.
Output	One charge per resident per period, with a frozen amount and a stable event key.
Recordkeeping	The charge references the exact terms version used, so it can be explained later.
Audit	Run start, run completion, charges created, charges skipped as already existing.
Errors	Job runs twice → idempotency key prevents duplicates. Job does not run → a heartbeat alert fires if no run completed by the configured hour. Terms change mid-period → the issued charge is never rewritten.

> **BILLING IS PER RESIDENT, NOT PER CALENDAR MONTH**
>
> Residents have different due dates based on their admission dates. CONFIRMED A resident admitted on the 17th is billed on the 17th. This single fact rules out any month-end batch design, and it is the most consequential structural input the product has.

Blocked. Automated generation cannot be implemented. Six rules are unanswered: what determines the due date (BR-REN-001); what dates a charge covers and whether they are inclusive (BR-REN-002); when the first payment is due and what it covers (BR-REN-003); how a mid-period admission is charged (BR-REN-004); what the rent stop date is at checkout (BR-REN-005); and what happens when a due date falls on the 29th to 31st in a shorter month (BR-REN-006).

Until these are answered the billing-run operation returns an explicit unconfigured-policy refusal naming the missing rules. **A manual charge with an amount and reason supplied by an authorised user is permitted** — that requires no policy, because a human decided the number.

Workflow 2 — Manual payment recording

Actors	Finance clerk or owner, holding `payment.record`.
Inputs	Resident, amount, date received, method, reference — UPI id, bank UTR, or a server-minted receipt number for cash.
Output	One payment record marked staff-recorded, plus a numbered receipt.
Recordkeeping	Receipt numbers are gapless per property per financial year.
Audit	Actor, timestamp, before and after state, originating request.
Errors	Duplicate reference → refused with a link to the existing payment. Double-click → same idempotency key returns the original. Client timeout after commit → retry returns the original, never a second payment.

Partly blocked. Recording works without policy. What a payment's *status* means does not: whether staff may save a payment as reported-pending before independent confirmation, who may mark it confirmed and on what evidence (BR-PAY-002, BR-PAY-003). Until that is settled, **no status may be presented as confirmed collection**, and the confirmation operation is disabled.

Automatic duplicate rejection based on a real-world reference is separately blocked (BR-PAY-005). Request idempotency — which prevents a retry creating a second record — is an architecture invariant and already works. **The two are different things and neither substitutes for the other.**

Workflow 3 — Allocation

Actors	Finance clerk or owner, holding `payment.allocate`.
Inputs	A payment, a target charge, an amount.
Output	An allocation record; the charge's derived settled amount changes.
Audit	Every allocation and every reversal, with actor and reason.
Errors	Amount exceeds what remains allocatable → refused with the current remaining figure. Two concurrent allocations consume the same amount → one succeeds, one receives a conflict. Charge and payment belong to different organisations → not-found.

NO OLDEST-FIRST DEFAULT

Which charge a payment settles when the resident does not say is **the owner's decision** (BR-ALLOC-002). Oldest-first is the obvious guess and it is still a guess — and a guess made in the pilot becomes the billing rule for every later customer, invisibly.

Until an order is approved, allocation **requires an explicit target charge**. The UI does not preselect, does not offer "apply automatically", and the API refuses an allocation request that omits the charge.

Also blocked: whether surplus is held as unallocated credit, returned or allocated by choice (BR-ALLOC-003); whether payments may be recorded before they can be matched (BR-ALLOC-004); and how an allocation is corrected after an error (BR-ALLOC-005).

Workflow 4 — Corrections and reversals

Actors	Restricted. Who may correct, reverse or refund is unanswered (BR-PAY-009).
Inputs	The original record, a reason, an approver where policy requires one.
Output	A linked reversal or adjustment. **The original remains visible.**
Audit	Mandatory — this is the highest-risk financial action in the system.
Errors	Reversing an already-reversed record → refused. Reversing beyond the original amount → refused. Reversal without a reason → refused.

There is no API path that edits or deletes a posted record, including for administrators. This is verified by test rather than left to convention.

Workflow 5 — Deposits

Actors	Finance clerk or owner for receipt. Deduction, refund and settlement are blocked for every role.
Inputs	Resident, amount, date, method, evidence.
Output	A deposit ledger entry. Liability, not income.
Audit	Every receipt, deduction, refund, correction and reversal.
Errors	Refund exceeds available balance → refused. Any attempt to allocate a deposit to a rent charge → structurally impossible.

Blocked. Seven rules are unanswered: amount determination (BR-DEP-001); collection timing and instalments (BR-DEP-002); accounting classification (BR-DEP-003); refundability conditions (BR-DEP-004); permitted deduction types and evidence (BR-DEP-005); offset against unpaid rent (BR-DEP-006); available-balance treatment (BR-DEP-007); refund deadline and process (BR-DEP-008); and the approval roles and thresholds (BR-DEP-009).

Offsetting a deposit against outstanding rent never happens automatically, under any circumstance, regardless of what other rules are approved. It requires an agreement basis, a legal basis, an accounting basis and an explicit approval.

Workflow 6 — Checkout settlement

Actors	Preview by finance; approval by owner, holding `settlement.approve`.
Inputs	Deposit held, outstanding charges, any final pro-rata charge, itemised deductions with reasons.
Output	An itemised settlement statement and a net refund or recovery.
Audit	The preview, the approval, the executed refund and the evidence of payment out.

ENTIRELY BLOCKED, AND SEPARATE FROM OPERATIONAL CHECKOUT

Recording that a resident left changes no balance, nets no deposit, releases no bed and issues no refund. Settlement is a different workflow, with a different permission and a different approval.

It cannot be implemented until BR-REN-005, BR-LIF-010, BR-LIF-012, BR-DEP-005, BR-DEP-006 and BR-DEP-008 are signed off **and approved expected-result tests exist**. A settlement computed from a guessed rule is a wrong number handed to a departing resident at the moment they are least willing to accept one.

Workflow 7 — Reconciliation

Actors	Finance clerk daily; owner reviews.
Inputs	Recorded payments for a period; counted cash; bank or UPI statement.
Output	A collection report split by method with differences surfaced, never absorbed.
Errors	Recorded total ≠ counted cash → the discrepancy is displayed and requires disposition. The system never adjusts a record to make two totals agree.

Blocked in part: which source is treated as authoritative evidence and who confirms receipt (BR-PAY-007, BR-PAY-008).

Reporting

Four reports matter in the pilot: outstanding dues with ageing, collections by method and period, deposits held, and occupancy. **Every report states its cutoff, its filters and its status-inclusion rule on its face.**

Each definition is OPEN (BR-REP-001 to BR-REP-004). What "outstanding" includes, whether collections are counted by recorded date or confirmation date, what the deposit report shows — these are not implementation details. Two people reading the same number must mean the same thing by it, or the partnership has a dispute rather than a report. Until a definition is approved, the report lists records and withholds the authoritative total.

Audit requirements

Every financial write records actor, timestamp, before and after state, and the originating request. The trail must be sufficient to reconstruct, for any resident at any past date, exactly how their balance reached the figure displayed — which is the difference between a dispute that ends in two minutes and one that ends in a lost customer.

Audit records are append-only to ordinary application roles. Access to the trail is itself controlled and logged.

Staff permissions

ACTION	PERMISSION	NOTE
Record a payment	`payment.record`	Finance clerk by default
Confirm a payment	`payment.confirm`	Separate authority. Blocked pending BR-PAY-003.
Allocate a payment	`payment.allocate`	Explicit target required

ACTION	PERMISSION	NOTE
Reverse an allocation	`allocation.reverse`	Blocked pending BR-ALLOC-005
Create or adjust a charge	`charge.create`, `charge.adjust`	Reason mandatory
Receive a deposit	`deposit.receive`	Permitted
Deduct, refund, settle	`deposit.*`, `settlement.*`	**Blocked for every role**
Export financial data	`data.export` + `finance.read`	Both required; audited

What is blocked, and what it blocks

UNANSWERED RULE	WHAT STAYS DISABLED
BR-REN-001 … BR-REN-006	Automated rent generation and billing runs. Manual charges still work.
BR-REN-008	Grace periods and late fees; overdue shown with penalties.
BR-REN-012	Any tax line or tax invoice.
BR-PAY-002, BR-PAY-003	Marking a payment confirmed; any confirmed-collection figure.
BR-PAY-005	Automatic duplicate rejection on a UPI or bank reference.
BR-ALLOC-002	Automatic allocation. Staff must choose the target charge.
BR-ALLOC-003, BR-ALLOC-004	Unmatched receipts; advance credit.
BR-ALLOC-005	Reversing or reassigning an allocation.
BR-DEP-004 … BR-DEP-010	Deductions, refunds and all settlement.
BR-DEP-006	Any netting of deposit against dues — never automatic regardless.
BR-REP-001 … BR-REP-004	Authoritative totals on the corresponding reports.

A blocked operation returns an explicit unconfigured-policy refusal naming the rule, writes no record and applies no default. It does not fail silently, and it is not a bug.

Status

Nothing in this chapter is implemented. There is no charge, payment, allocation, deposit or ledger in the repository, and the database schema is intentionally empty. The money type, the idempotency infrastructure and the audit contract are the first things that can be built once the architecture is approved — none of them requires a business rule.

CHAPTER 12

Operations and Property Services

Maintenance, complaints, incidents and housekeeping — deliberately the thinnest module group in the product, and an explanation of why keeping it thin is a design decision rather than an omission.

> **SCOPE DISCIPLINE**
>
> Every module in this chapter is POST-PILOT or later. None is required for the pilot to succeed, and each one is a plausible route into building a maintenance product instead of a hostel management product. They are specified here so the boundary is explicit, not so they get built early.

Maintenance requests

Purpose	Record that something is broken, who is dealing with it and whether it was fixed.
Actors	Staff log and resolve. Residents report through staff in early phases; direct reporting needs a portal.
Inputs	Property, optionally room or bed, category, description, priority, reporter.
Lifecycle	Open → In progress → Resolved, or Closed with a reason. Each transition records actor and time.
Output	A resolution record and a per-room history, so recurring faults become visible.
Permissions	Operational roles. A request may reference a bed without exposing who occupies it.
Deliberately excluded	SLAs, cost tracking, vendor management, parts inventory, scheduling.

The one genuinely valuable derived view is **recurring issues per room**. A bed whose fittings fail four times in a year is a capital decision, not a maintenance queue item, and only a record makes that visible.

Complaints and incidents

These are two different things and conflating them is a privacy mistake.

TYPE	CONTENT	ACCESS
Complaint	Service dissatisfaction — noise, cleanliness, facilities. Operational.	Operational roles
Incident	Something that happened involving people. May concern safety, conduct or a dispute between residents.	**Restricted.** Owner, plus explicitly named roles only.

INCIDENT RECORDS NEED A DECISION BEFORE THEY NEED A FEATURE

An incident record in a women's hostel may contain allegations, safety concerns or conduct matters about named residents. Who may read it, how long it is kept, whether the resident may see what was recorded about them, and what happens to it at checkout are questions with legal weight.

Until they are answered, **the incident module is not built**. A free-text field that staff use for sensitive matters without an access model around it is worse than no field, because it creates an unmanaged record with no retention and no restriction. OPEN

Housekeeping and inspections

FUTURE. Scheduled cleaning, room inspections at move-in and move-out, checklist completion and evidence capture.

The one connection to an approved workflow is the **move-out inspection**, because its output feeds deposit deductions. That link is the reason it cannot be built yet: deduction policy is unresolved (BR-DEP-005, BR-DEP-010), and an inspection record designed before the deduction rules it must support would almost certainly capture the wrong evidence.

Staff assignment and status tracking

Requests may be assigned to a staff member, and escalate to the manager if a high-priority item stays open past a configured age. The escalation threshold is a property setting, not a product default — and no SLA is encoded until one is agreed (BR-OPS-006). OPEN

Evidence and attachments

Photographs are the obvious attachment for maintenance and inspections, and they are **blocked by the same decision that blocks document upload** (ADR-012). Binary upload is excluded from the pilot in its entirety, and a maintenance photograph is a file in private storage with retention, access and malware considerations exactly like any other.

Until upload is approved, requests carry text descriptions only. This is a real limitation and worth stating plainly: describing a damaged fitting in words is meaningfully worse than photographing it.

Operational reports

Open requests by category and age; resolution time distribution; recurring issues per room; requests by assignee. All derived from source records, all respecting organisation and property scope.

Proposed features versus first-release commitments

CAPABILITY	PHASE	GATE
Maintenance request logging and resolution	POST-PILOT	Pilot accepted; owner confirms the workflow is wanted
Complaints	POST-PILOT	As above
Incidents with restricted access	OPEN	Access, retention and resident-visibility decisions

CAPABILITY	PHASE	GATE
Photographic evidence	BLOCKED	Document upload approved (ADR-012)
Housekeeping schedules	FUTURE	Demand evidenced by a paying customer
Move-out inspections	FUTURE	Deposit deduction policy signed off
Resident-submitted requests	FUTURE	Resident portal exists

Status. NOT IMPLEMENTED. No operations module exists in the repository.

PART IV

Platform

What makes Nook a multi-tenant product rather than one hostel's application, and how it reaches the outside world.

13 Organisation Isolation and SaaS Administration
14 Notifications and External Integrations



CHAPTER 13

Organisation Isolation and SaaS Administration

What makes Nook a platform rather than one hostel's application: the tenancy boundary, how it is enforced at every layer, and the administration that sits above it.

Two boundaries, often confused

BOUNDARY	NATURE	VIOLATION MEANS
Organisation	Absolute. Separates unrelated businesses.	One customer sees another's residents and money. Company over.
Property	A grant inside a tenancy. Separates sites of one business.	A manager sees a property they do not run. Serious, recoverable.

A ONE-CUSTOMER PILOT IS NOT A SINGLE-TENANT SCHEMA

The temptation is to defer isolation because there is only one customer. Retrofitting it later means migrating live financial data under a correctness constraint — the one operation you least want to perform on a customer who has started to trust you. Organisation scoping exists from the first commit.

Seven layers of enforcement

1. **No trusted client identifier.** The session identifies a user; membership is resolved server-side. An organisation the actor does not belong to returns *not found*, never *forbidden*, so existence cannot be probed.
2. **One tenant context object** carrying user, organisation, role, properties and permissions, constructed by one middleware and passed explicitly into every service call. No ambient globals.
3. **Physical scope columns.** Every organisation-owned row carries `organization_id` and `property_id` directly, never resolved through a join chain. A missing scope column fails CI.
4. **Repository-layer injection.** Repositories take the tenant context and add the scope predicate themselves. A lint rule forbids importing data models from controllers, so an unscoped query cannot be written in the place where people forget.
5. **Private files.** Object keys are namespaced per organisation and per resident. No public bucket, no durable URL. Access mints a short-lived signed URL only after authorisation, and the mint is audited.
6. **Scoped background work.** Jobs reconstruct a tenant context from their payload and run through the same repositories. Leakage through a cron job is the failure mode teams forget.
7. **Database backstop.** PostgreSQL row-level security, with the application connecting as a role that cannot bypass it. If all six layers above fail, the database still returns nothing.

OPEN — ADR-007 LEAVES ADOPTION UNDECIDED

Plus a standing test. A cross-tenant suite enumerated automatically from the router issues every registered route as Organisation B against Organisation A's identifiers — including lists, searches, reports, exports, imports, jobs and files. It runs on every commit and a single leak fails the build. This is release gate RG-2.

Membership and context switching

A user may belong to several organisations, but access to each is granted independently. The active organisation lives in the authenticated server-side session. Switching requires an explicit authenticated operation that re-confirms membership and refreshes session context; it is never a client-side filter or a request parameter.

Within an organisation, staff are granted access per property. Revoking a membership or changing a role invalidates existing sessions — a revoked staff member must lose access immediately, not when a token happens to expire.

Subscription, entitlements and pricing

> **NOTHING IN THIS SECTION IS APPROVED**
>
> No price, tier, limit or billing policy has been decided. Subscription billing is explicitly out of pilot scope and should not be built before a third customer exists. The structures below exist so the data model does not preclude them, not because they are planned.

CONCEPT	OPEN QUESTION
Billable unit	Total beds, configured beds or occupied beds? They differ materially at a property with vacancy, and the choice changes the invoice every month.
Tier limits	What a plan caps — beds, properties, staff seats — and what happens at the limit.
Lifecycle	Trial, active, past due, suspended, cancelled; and what each state does to access.
Suspension	**The hardest question.** See below.
Export and deletion	What a departing customer takes with them and when their data is removed.

> **SUSPENSION MUST NEVER HOLD A PROPERTY'S RECORDS HOSTAGE**
>
> If a customer stops paying, the obvious lever is to cut access. But the data is the record of who lives in their building and who owes them money — a hostel locked out of that cannot operate, and residents are affected by a commercial dispute they are not party to.
>
> The proposed position: suspension restricts **write** access and new admissions while preserving read access and export. A customer must always be able to retrieve their own data. Whatever is decided, it is a commercial and ethical decision, not an implementation default. OPEN

Platform administration

A Nook platform operator administers organisations — creating them, inviting owners, investigating problems. The trust boundary is strict:

- **No standing access** to any customer's resident or financial records.

- Access is just-in-time, reason-coded, time-limited and audited.
- Every action taken under elevated access is visible to the customer.
- MFA is required for platform accounts. PROPOSED
- A general-purpose support console is deferred. Until it exists, support is a conversation, not a login.

Onboarding a second organisation

The test that decides whether Nook is a product:

1. Platform admin creates the organisation and invites its owner. No existing user gains access.
2. The owner completes self-serve setup: property, policies, inventory, staff.
3. Historical data is imported into a staging state the owner approves before it posts.
4. The cross-tenant suite runs against the new organisation as part of onboarding.

This must be completable without a developer writing code. Until it is, Nook is a consulting engagement with a login screen. NOT IMPLEMENTED

CHAPTER 14

Notifications and External Integrations

Every outside service Nook might depend on, what each would cost in money and failure modes, and the rule that governs all of them: nothing is active, nothing has credentials.

NO PROVIDER IS APPROVED, INTEGRATED OR CREDENTIALED

MSG91 is a **candidate** SMS provider. Razorpay is a **future candidate** payment provider; Razorpay Route is an optional later evaluation. Neither has been selected, contracted or integrated. No real credential exists in the repository or in any configuration, and none may be added during documentation work.

Integration register

INTEGRATION	PURPOSE	PHASE	STATUS
SMS / phone verification	Prove an enquirer controls their number	Post-pilot	CANDIDATE
WhatsApp	Rent reminders and receipts on the channel residents actually read	Post-pilot	OPEN
Email	Statements, receipts, staff invitations, recovery	Post-pilot	OPEN
Payment gateway	Collect rent online rather than recording it manually	Future	EXCLUDED FROM PILOT
File storage	Private object storage for documents	Conditional	BLOCKED — ADR-012
3D rendering library	Full-building view	Future	CANDIDATE — CH 7
Monitoring / error reporting	Know when the service is failing	Pilot	OPEN

Messaging — the design that applies to all channels

Messaging is where a correctness mistake becomes a privacy incident, because the output leaves the system and reaches a human.

CONCERN	REQUIREMENT
Provider abstraction	One `MessageChannel` interface with a fake for tests. Provider-specific logic never leaks into domain code.
Consent	Recorded per resident per channel, with the basis and date. Absent consent means no send.
Template approval	WhatsApp requires pre-approved templates; content is reviewed before it can be sent at all.
Idempotency	A retried job must not send a second message. Reminders are keyed on resident, charge and schedule slot.
Delivery tracking	Every attempt recorded with channel, recipient, template, status and provider reference.
Timeouts and retries	Bounded retries with backoff; a permanently failing message surfaces to staff rather than retrying forever.
Outage behaviour	Queue and degrade. Staff see which reminders did not go out and can act by phone.

CONCERN	REQUIREMENT
Cost	Per-message, borne by the operator. A reminder schedule is a budget decision as much as a product one.

> **AN ARREARS REMINDER IS SENSITIVE CONTENT**
>
> It discloses that a named person owes money. Three consequences: recipients are re-resolved at send time rather than from a stale queued payload; amounts stay out of previews and lock-screen notifications; and production sending is disabled outside production by configuration, not by discipline — a test run that messages real residents about their rent is not recoverable.

Payment gateway — why it is excluded

Online collection is the obvious next step and deliberately not the first one. It introduces provider onboarding and KYC, webhook retry and replay handling, settlement reconciliation, refunds, disputes, and a materially different regulatory position once money moves through the platform.

None of that helps the owner answer "who owes money?" any sooner — which is the problem being solved. The architecture leaves a provider boundary; it does not build an abstract payment platform in advance.

When it is evaluated, two requirements are already known. **Webhook processing must be idempotent on the provider's event identifier**, because providers retry and a double-credited resident is a financial defect. And the UI must distinguish a staff-recorded payment from a provider-confirmed one, under the approved status policy (BR-PAY-003).

Monitoring and error reporting

The one integration that belongs in the pilot, because an unmonitored service fails silently. Required: application errors, availability, database health and job failures; alerts on repeated authentication failures, abnormal permission denials, failed backups, failed restores, import anomalies and unexpected financial operation rates.

Constraint: a third-party monitoring service is a data processor. Personal data must not be logged into it casually, and its region, retention and access settings need review before it carries anything from a resident record.

Rules that apply to every integration

1. Behind an adapter interface with a fake for tests. No provider SDK in domain code.
2. Credentials in a secret manager, never in source, never in a client bundle, never in this handbook.
3. Enabled per environment. A provider active in development that is live in production by accident is a predictable incident.
4. Every external call has a timeout, bounded retries and a defined failure behaviour.
5. An outage degrades the feature; it never takes down the product. Verification unavailable means staff verify manually, not that admissions stop.
6. Cost is modelled before activation, because every one of these has a per-use price.

Status. NOT IMPLEMENTED. No provider SDK, adapter, credential or configuration exists in the repository.

PART V

Engineering

Architecture, data, contracts, security and the evidence standards that decide when something may ship.

15 Technical Architecture
16 Proposed Data Model and Data Dictionary
17 API Design and Contracts
18 Security, Privacy and Trust
19 Non-Functional Requirements
20 Testing and Quality Assurance
21 Deployment and Operations



CHAPTER 15

Technical Architecture

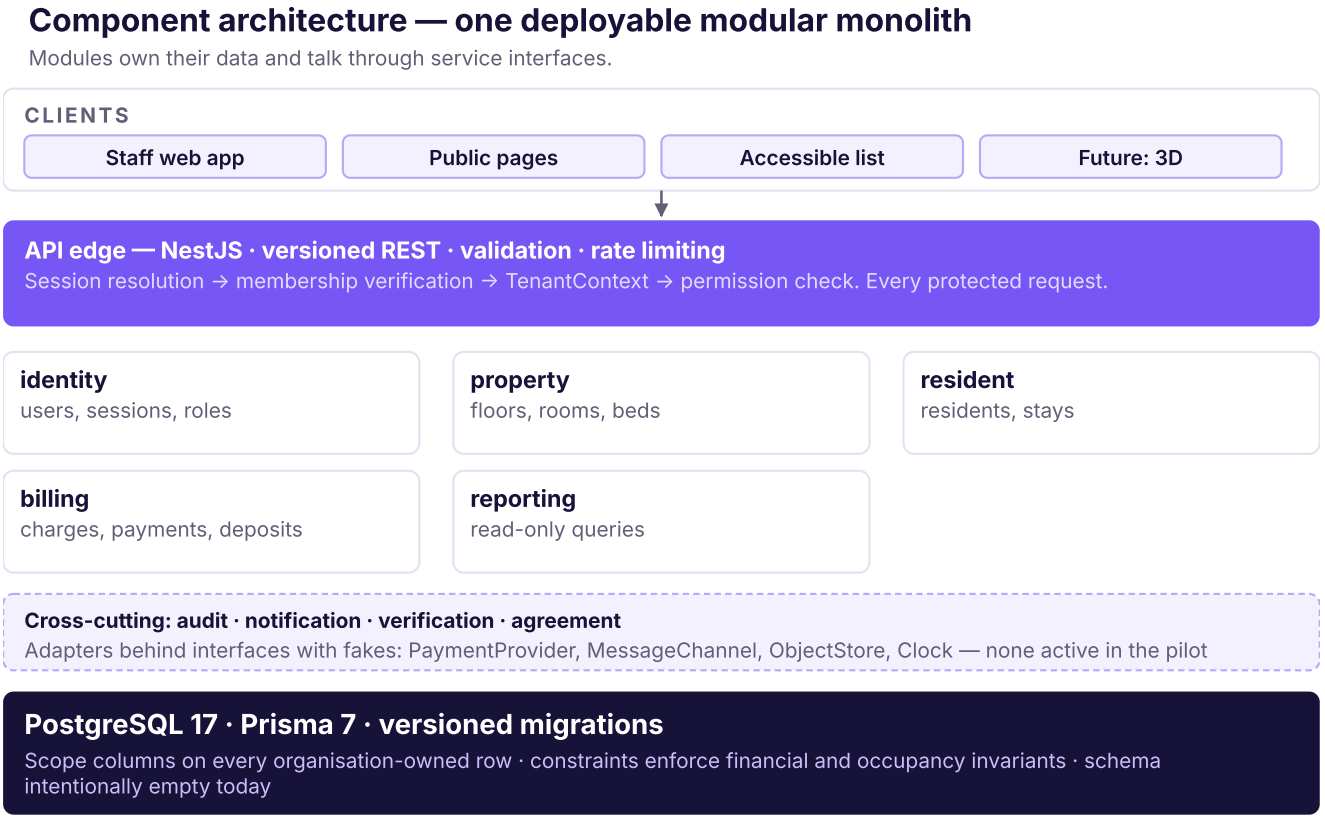
The stack as it stands in the repository, the module boundaries proposed above it, and the two guarantees — tenant isolation and allocation atomicity — that the architecture exists to make unbreakable.

The stack, as actually pinned

These versions are read from the repository, not proposed.

LAYER	CHOICE	VERSION	STATE
Runtime	Node.js	24.22.1	PINNED via `.nvmrc` and `engines`
Package manager	npm	11.6.2	PINNED via `packageManager`
Backend	NestJS + TypeScript	12.2.0	SHELL ONLY — health endpoints, config, database service
Frontend	React + Vite	19.3.1 / 8.3.5	SHELL + DEMO UI
Database	PostgreSQL	17	NEVER CONNECTED — no running instance
ORM	Prisma	7.10.1	CONFIGURED, schema intentionally empty
TypeScript	—	5.9.3	TYPECHECK PASSES

Component architecture



> **Figure 15.1 — Component architecture.** One deployable process with explicit module boundaries. Every protected request passes through session resolution, membership verification, tenant context construction and a permission check before reaching a module.

A modular monolith, not microservices. Core workflows need atomic transactions across related records — a payment and its allocation, an assignment and a stay transition. Distributing those across services would introduce distributed consistency problems before product demand justifies any of it. Two developers and one customer do not need independent deployability; they need transactions.

The discipline that makes it work: each module owns its data and exposes a service interface. Modules call each other through those interfaces, never by reaching into another module's tables. That keeps a later extraction possible without ever forcing one.

Request and authentication flow

1. Request arrives with an opaque session cookie — `HttpOnly`, `Secure` in deployed environments, an appropriate `SameSite`, narrow path scope.
2. CSRF protection runs on every state-changing request.
3. The session is resolved server-side against the session store. Revoked means revoked immediately, not at token expiry.
4. Membership is resolved for the session's active organisation. A client-supplied organisation identifier is **never** consulted.
5. A `TenantContext` is constructed carrying user, organisation, role, properties and permissions.

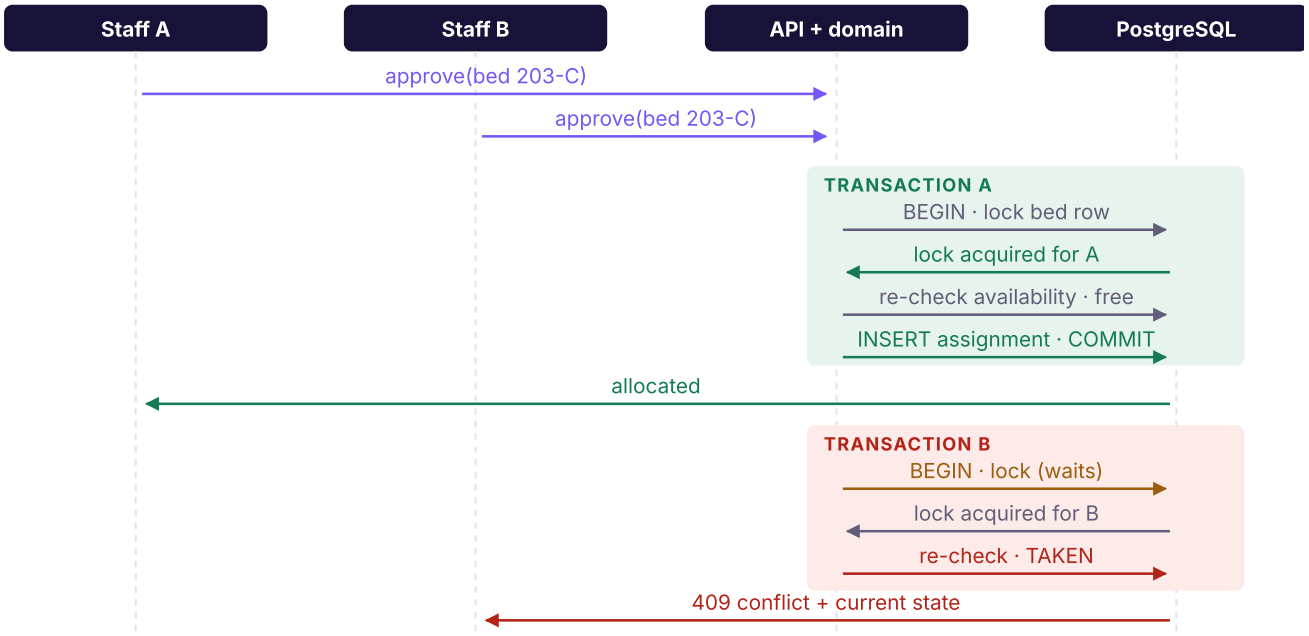
6. The permission identifier required by the route is checked against that context — never a role-name string.
7. The service executes, with repositories injecting the organisation scope predicate automatically.
8. The audit module records the action where it is sensitive.

Sessions are server-side and revocable rather than JWTs, because revocation has to actually work: a staff member whose access is withdrawn must lose it at once. Expiry values, MFA policy and the recovery channel remain `OPEN`.

Bed allocation and concurrency

Two staff allocate the same bed at the same moment

The displayed map told both of them it was free. Exactly one allocation may survive.



The re-check runs inside the transaction against current records. A database constraint enforces the same rule, so a code path that forgets still cannot double-book.

> **Figure 15.2 — Concurrent allocation.** Both staff saw the bed as free. The re-check happens inside the transaction against current records, and a database constraint enforces the same rule independently of application logic.

Enforced in two places deliberately: application logic produces a good error message, and the database constraint guarantees correctness when a future code path forgets to check. A guarantee that depends on every developer remembering is not a guarantee.

Payment recording and reconciliation

The same transactional discipline applies to money. A payment and its allocation commit together or not at all. Idempotency is enforced at the persistence boundary by unique constraints — on the billing event key for charge generation, and on the payment reference per property — rather than by a prior read that can race.

Request idempotency and real-world duplicate detection are separate mechanisms with separate rules; Chapter 11 explains why neither substitutes for the other.

Spatial data and availability

One service resolves layout geometry and live bed state, enforces organisation scope, and is the only path to spatial data. The 2D plan, the accessible list and any future 3D view are renderers over it (Figure 7.4). Availability is derived from effective-dated assignments, never stored as a flag that can drift.

Shared validation contracts

Schemas defined once and shared between client and server, generating the OpenAPI document from the same source. One definition means the two ends cannot drift, and the contract is reviewable rather than inferred from implementation. PROPOSED

Background jobs

Required for charge generation, reminder dispatch and nightly balance reconciliation. A database-backed queue is proposed rather than a separate service: one fewer thing to run, back up and secure, and transactional with the data it acts on.

Jobs are tenant-scoped. Each reconstructs a tenant context from its payload and runs through the same repositories as a request. Cross-tenant leakage through a background job is the failure mode teams forget, because nobody is watching when it runs.

Observability and configuration

Structured logs with correlation identifiers matching audit events; application error reporting; health endpoints that expose status without leaking connection details. Personal data is not logged casually into a third-party service.

Configuration is environment-specific and validated at startup, failing loudly on a missing or invalid required value rather than starting in a broken state. `.env.example` carries placeholders only. Secrets live in the host's secret manager. Any variable exposed to the client bundle is public by definition and may never hold a secret. IMPLEMENTED — config parsing and validation exist and are tested.

Testing, CI and deployment

Covered in Chapters 20 and 21. The architectural requirement is that integration tests run against real PostgreSQL rather than mocks — the constraints are the thing being tested, and a mock cannot demonstrate that a unique index prevents a double allocation.

Architectural decisions and their status

DECISION	POSITION	STATUS
ADR-001	Modular monolith on NestJS + TypeScript	PROPOSED
ADR-002	React + TypeScript + Vite	PROPOSED
ADR-003	PostgreSQL + Prisma + versioned migrations	POSTGRES APPROVED IN PRINCIPLE
ADR-004	Server-side revocable sessions, opaque cookies	PROPOSED
ADR-007	Organisation isolation; RLS undecided	RLS OPEN

DECISION	POSITION	STATUS
ADR-008	Money as integer paise, decimal-string API	PROPOSED
ADR-013	Financial policy guardrail	CONFIRM
ADR-015	No payment gateway in the pilot	PROPOSED

Seventeen decisions exist in total; the approval record is blank. Starting implementation is not approval.

CHAPTER 16

Proposed Data Model and Data Dictionary

The conceptual entities, their ownership and lifecycle, and the constraints that protect correctness. A proposal — the live schema is intentionally empty and must stay that way until this is approved.

THIS IS A CONCEPTUAL MODEL, NOT A SCHEMA

`prisma/schema.prisma` contains a generator, a datasource and no models, by design. No migration exists. **No domain entity may be added until this model and the business rules for the features it supports are approved**, and no schema was changed during the production of this handbook.

Entity register

Full attribute-level detail is in **Appendix B — Data Dictionary**. This register gives purpose, tenant ownership and the property that most constrains each entity.

ENTITY	PURPOSE	SCOPE	DEFINING CONSTRAINT
Organisation	One independent customer business	Root	Every owned record traces to exactly one
User	An authenticated principal	Global	Identity alone grants no access to any organisation
Membership	Links a user to an organisation with a role	Org	Unique per user per organisation; revocation invalidates sessions
Property	A physical premises	Org	Carries timezone — billing dates are date-only in it
Building / Floor	Structure and level	Org + property	Floors ordered so above and below are meaningful
Room	An enclosed space with the property's own label	Org + property	Label unique within its scope
Bed	**The allocatable unit**	Org + property	Never deleted once assigned — deactivated
SpatialLayoutVersion	Effective-dated geometry for a floor	Org + property	Published, not edited live; history resolves against the version in force
Resident	A person	Org	Separate from any stay, so repeat admission loses nothing
Stay	One occupancy period for one resident	Org + property	Lifecycle transitions are events, not a mutated field
Assignment	Bed + stay over a date interval	Org + property	**No overlapping active intervals per bed** — DB-enforced
RentalTermsVersion	Approved commercial terms for a stay	Org	Effective-dated; an issued charge references the version used

ENTITY	PURPOSE	SCOPE	DEFINING CONSTRAINT
Charge	An amount owed for a period	Org	Immutable once posted; unique billing event key
Payment	Money that arrived	Org	Immutable; unique reference per property per method
Allocation	Links a payment to a charge for an amount	Org	Sum never exceeds either side's limit
Deposit / DepositEntry	A liability held, and its movements	Org	**Never allocatable to a charge**
Adjustment / Reversal	Signed corrections with reason and approver	Org	Reference the original, which stays visible
MaintenanceRequest	Something needing attention	Org + property	May reference a bed without exposing its occupant
Notification	An attempted or delivered message	Org	Recipient re-resolved at send; consent recorded
AuditEvent	Who did what, when, to which record	Org	Append-only to application roles
Subscription / Entitlement	Commercial plan and limits	Org	Future. Billable unit undefined.

Ownership and access

Every organisation-owned row carries `organization_id` physically, and `property_id` where applicable — denormalised rather than resolved through a join chain, so a scope predicate is always a direct column comparison. A missing scope column fails CI.

Cross-organisation references are prevented by composite foreign keys including the organisation, so an allocation cannot point at a charge belonging to another customer even if the application logic is wrong.

Concurrency implications

ENTITY	HAZARD	PROTECTION
Assignment	Two allocations of one bed over overlapping dates	Row lock plus an exclusion or partial-unique constraint on the interval
Charge	A retried billing run duplicating a period	Unique billing event key
Payment	Double submission creating two records	Unique idempotency key and unique real-world reference
Allocation	Two allocations consuming the same remaining amount	Serialised transaction plus a sum constraint
Deposit	Refund exceeding the balance	Balance derived inside the transaction, with a non-negative check

Lifecycle and retention

Financial records are permanent within the retention policy. Residents and stays are deactivated, never deleted, so historical charges and receipts continue to resolve. Audit events are append-only. Retention periods for personal data are OPEN and require professional confirmation (BR-OPS-003,

BR-OPS-004), and the conflict between a deletion request and an accounting retention obligation is explicitly not resolved here.

Decisions required before any schema is written

1. Billing period and due-date rule (BR-REN-001, BR-REN-002) — determines the charge's period columns and event key.
2. Date interval semantics — inclusive or exclusive ends, timezone handling (BR-REN-002, BR-LIF-004). The assignment overlap constraint cannot be written without this.
3. Payment status model (BR-PAY-003) — determines whether a status column exists at all.
4. Allocation rules (BR-ALLOC-001 … 005).
5. Deposit scope — per resident or per stay (BR-DEP-003).
6. Lifecycle states and permitted overlap (BR-LIF-001, BR-LIF-006).
7. Minimum resident fields and whether documents exist (BR-LIF-008, BR-SEC-004).
8. Opening-balance representation for migration (BR-OPEN-001 … 005).
9. Role-to-permission grants (BR-SEC-002, BR-SEC-003).
10. Whether row-level security is adopted (ADR-007).

Money representation is *not* on this list — it is decided as proposed in ADR-008: integer paise in `BIGINT`, `amountMinor` as a base-10 integer string across the API.

CHAPTER 17

API Design and Contracts

The proposed REST surface, the conventions that apply to every endpoint, and the error contract that makes an unresolved business rule visible rather than silently defaulted.

> **EVERY ENDPOINT IN THIS CHAPTER IS PROPOSED**
>
> Two endpoints exist in the repository: `GET /api/v1/health` and `GET /api/v1/health/database`. Neither has ever been successfully called, because the API process has not started in this environment. Everything else is specification.

Conventions that apply everywhere

- Versioned REST under `/api/v1`. Every protected endpoint authenticates and authorises server-side.
- **Organisation context comes from the session**, verified against membership. A client-supplied organisation identifier is never authority, in any position.
- **Money is `amountMinor`** — a base-10 integer string of paise — with an explicit `currency`. A JSON number is never accepted as an authoritative amount.
- ISO 8601 timestamps with explicit offset. **Business dates are date-only values**, separate from timestamps, in the property's timezone.
- Access is expressed as a **permission identifier**, never a role name.
- Financial writes require an `Idempotency-Key` bound to actor, organisation and request fingerprint.
- Lists are paginated with bounded page sizes, explicit sorting and validated filters.
- State changes are business actions — `move-in`, `room-change`, `checkout` — not arbitrary edits to a status field.

Endpoint groups

The full catalogue, with per-endpoint access, validation and behaviour, is **Appendix C — API Catalogue**. Summary:

GROUP	REPRESENTATIVE ROUTES	NOTABLE
Authentication & context	`auth/login`, `auth/me`, `organisations/current`, `organisations/{id}/switch`	Switch re-confirms membership
Inventory	`properties`, `rooms`, `beds`, `beds/availability`	Availability needs date semantics
Spatial layouts	`floors/{id}/layout`, `layout/{id}/publish`	Draft then publish
Residents & stays	`residents`, `stays/{id}/move-in`, `/room-change`, `/checkout`	Checkout performs no financial action
Billing	`charges`, `billing-runs`, `residents/{id}/statement`	`billing-runs` blocked
Payments	`payments`, `/confirm`, `/allocations`, `allocations/{id}/reverse`	Allocation requires explicit target

GROUP	REPRESENTATIVE ROUTES	NOTABLE
Deposits & settlement	`stays/{id}/deposit/*`, `/settlements/preview`, `/settlements`	Deductions, refunds, settlement all blocked
Reports & audit	`reports/outstanding`, `/collections`, `/occupancy`, `audit-events`	Definitions open
Migration	`import-batches`, `/validate`, `/commit`, `/report`	Never commits on upload
Public	`public/properties`, `public/properties/{id}/layout`	Separate projection; heavily rate-limited
Documents	—	**Absent from the P0 contract**

Representative request and response

Recording a payment. Note the money format, the explicit allocation target and the idempotency key.

```
POST /api/v1/payments
Idempotency-Key: 6f1c2a9e-4d3b-47aa-9f10-2c7e51b8d904

{
  "residentId": "res_8f21",
  "amountMinor": "850000",
  "currency": "INR",
  "method": "upi",
  "reference": "428817446201",
  "receivedOn": "2026-11-03",
  "allocations": [ { "chargeId": "chg_4411", "amountMinor": "850000" } ]
}

201 Created
{
  "paymentId": "pay_77c3",
  "amountMinor": "850000",
  "currency": "INR",
  "source": "staff_recorded",
  "receiptNumber": "NH/26-27/0412",
  "allocations": [ { "chargeId": "chg_4411", "amountMinor": "850000" } ],
  "unallocatedMinor": "0"
}
```

Error contract

STATUS AND CODE	MEANING
`400 BAD_REQUEST`	Malformed or unsupported shape
`401 UNAUTHENTICATED`	No valid session
`403 FORBIDDEN`	Authenticated but lacking the permission
`404 NOT_FOUND`	Absent **or deliberately concealed** — including any organisation the actor does not belong to
`409 CONFLICT`	Concurrency or state conflict; carries current state
`409 IDEMPOTENCY_KEY_REUSED`	Same key, materially different payload

STATUS AND CODE	MEANING
`422 UNPROCESSABLE_ENTITY`	Valid syntax, invalid business rule or state
`422 RULE_NOT_CONFIGURED`	**The operation depends on a business rule that has not been approved.** Names the unresolved rule, writes nothing, applies no default.
`429 TOO_MANY_REQUESTS`	Rate limited
`500 INTERNAL_ERROR`	Generic message with a logged request id; no internals

RULE_NOT_CONFIGURED IS THE CONTRACT'S MOST UNUSUAL FEATURE, AND ITS MOST IMPORTANT

It is how the API refuses to invent a business rule. A billing run with no approved due-date policy does not guess, does not return an empty result and does not fail obscurely — it states which rules are missing. The owner sees exactly what their decision is blocking, and the test suite asserts the refusal as an architecture invariant.

Cross-cutting validation

- Unknown fields rejected on privileged writes; no mass assignment from client objects to entities.
- Entity ownership verified before any join or linked update — the whole chain, not just the immediate parent.
- Financial totals validated **inside** the transaction, not before it opens.
- Frontend validation is usability; backend validation is authoritative.
- Document identifiers, bank references and resident details never appear in logs or error payloads.

Open API decisions

Session expiry, MFA and recovery channel; exact role-to-permission grants; payment status transitions; allocation ordering; billing event identity; checkout transition and bed-release date; settlement approval flow; import format and rollback granularity; rate limits and pagination defaults; and whether documents enter scope at all.

Resolved since the requirements package: money format (ADR-008), organisation context selection, and the session mechanism (ADR-004).

CHAPTER 18

Security, Privacy and Trust

What is being protected, who might attack it, the controls proposed, and an honest account of the one unresolved vulnerability currently blocking Milestone 0 acceptance.

Assets

Ranked by what their loss would cost. The ranking matters because it determines where effort goes.

ASSET	CONSEQUENCE OF COMPROMISE
Resident identity documents	Identity documents of ~95 women in the open. Unrecoverable. *This is why the pilot does not collect them.*
Cross-organisation data	One customer sees another's residents and finances. Company over.
Financial records	Wrong balances acted on as truth; disputes with no evidence.
Resident contact details	Personal data of a vulnerable population exposed.
Staff credentials and sessions	Full access under a legitimate identity.
Backups and exports	Everything above, outside the application's controls.

Threat model

ACTOR	CAPABILITY	PRIMARY CONTROL
Curious staff member	Legitimate login, looks beyond their role	Permission separation; audit of document and export access
Compromised staff account	Everything that account can do	Least privilege; revocable sessions; MFA for high privilege
Identifier manipulation	Substitutes identifiers in URLs and payloads	Server-derived scope; not-found for foreign organisations
Automated abuse	Scripted enumeration of the public surface or verification	Rate limits; separate public projection; no enumeration responses
Replay and duplication	Retried or replayed financial requests	Idempotency enforced at the persistence boundary
Dependency compromise	Malicious or vulnerable package	Lockfile, review, CI audit — **currently failing, see below**
Operator error	Accidental deletion or bad migration	Immutable financial records; tested restore; controlled migrations
Backup exposure	Obtains a backup	Separate credentials; encryption; restricted access

Controls

AREA	CONTROL	STATUS
Authentication	Server-side revocable sessions; opaque `HttpOnly` cookies; CSRF on state-changing requests; adaptive password hashing; rotation on privilege change	PROPOSED
Account provisioning	Owner-provisioned or invitation only; no public sign-up; single-use short-lived recovery tokens, hashed at rest; never security questions	PROPOSED
Authorisation	Permission identifiers checked server-side; an actor cannot grant what they do not hold	PROPOSED
Tenant isolation	Seven layers (Chapter 13); cross-tenant suite as release gate RG-2	PROPOSED
Financial integrity	Immutable posted records; transactional writes; DB-enforced invariants; idempotency	PROPOSED
Input validation	Unknown fields rejected; no mass assignment; parameterised queries	PROPOSED
Rate limiting	Login, recovery, verification, search, export, public endpoints	THRESHOLDS OPEN
File upload	Not applicable — upload excluded from the pilot	EXCLUDED
Secrets	Secret manager; `.env.example` placeholders only; no secret in a client-exposed variable; CI secret scanning	SCANNER PASSES
Audit	Append-only; actor, action, target, time, request id; access to the trail itself controlled	PROPOSED
Encryption	TLS in transit; managed encryption at rest; backup credentials separate	OPEN — HOSTING UNDECIDED
Backups	Automated, with a **tested** restore as the only acceptable evidence	NEVER TESTED
Dependencies	Lockfile; reviewed changes; CI vulnerability check	BLOCKED, SEE BELOW

The unresolved dependency vulnerability

MILESTONE 0 CANNOT BE ACCEPTED WHILE THIS STANDS

A file-upload dependency reached transitively through the NestJS platform adapter remains at a version covered by a maintainer advisory. The patched version is identified and is a compatible candidate, but **it is not installed**.

The diagnosis is specific rather than speculative: the root override reaches the workspace link nodes, but the target workspace nodes carry no override rules, so the adapter's dependency edge keeps its original specifier and is treated as valid. Switching between scoped and global override syntax does not repair this, and a lockfile-only update never touches the installed tree.

Resolution requires an environment with package-registry access: install the patched version with supported commands, verify both the locked *and* installed versions, confirm the override applies across the workspace graph, then re-run the checks and obtain a **successful live audit**.

The audit currently exits non-zero because its advisory endpoint is unreachable. **This is not a clean audit.** An offline install reporting zero findings is not accepted as security evidence, and no suppression, forced fix, prerelease install or manual lockfile edit was used to make it look otherwise. BLOCKED

Privacy and compliance

Nook processes personal data on behalf of the hostel. The division of responsibility between operator and platform must be established per processing activity rather than assumed.

AREA REQUIRING PROFESSIONAL CONFIRMATION	AFFECTS
Data protection law — obligations and their commencement	Notice, consent, rights, breach reporting, retention
Cyber-incident reporting directions	Logging scope, retention, reporting procedure
Identity-verification regulation	Whether a specific ID may be collected at all, and lawful alternatives
Accommodation and agreement requirements	Whether written or registered documentation is required
Local resident-verification procedure	What the property must report, and to whom
Tax treatment	Whether invoices need tax lines; the subscription's own treatment

NO LEGAL POSITION IS STATED AS SETTLED

This handbook names no commencement date, threshold, rate or procedure as established fact. Published commercial summaries of each disagree with one another on detail, and a product decision taken from a summary is a decision taken from a guess. Each requires confirmation by a qualified adviser against the current official text before the corresponding feature is enabled.

Incident response

Required before go-live: incident classification, named escalation contact, evidence preservation, containment and recovery steps, and a documented decision process for customer notification. Confirm applicable reporting obligations with qualified advice — but do not wait for a final legal interpretation to build basic incident handling.

Security testing

Cross-tenant access across every route including lists, reports, exports, imports, jobs and files; privilege escalation and delegation attempts; session revocation on role change and password reset; CSRF and cookie attributes; rate limits; redaction in logs and errors; secret scanning across repository, build artefacts and configuration; and dependency vulnerability checks. Detail in Chapter 20.

What is not claimed

- **No security audit has been performed.**
- **No penetration test has been performed.**
- **No compliance position is claimed** for any regulation.
- **No backup has been restored**, so no recovery capability is demonstrated.
- **The dependency audit does not pass.**

CHAPTER 19

Non-Functional Requirements

Measurable targets proposed for approval, separated carefully from commitments — of which there are currently none.

NO SLA EXISTS

Every figure below is a **proposed target** for a decision, not a guarantee to anyone. Availability, recovery and performance commitments cannot be made before a hosting provider is selected and the owner's tolerance for loss and downtime is established (BR-OPS-005). Publishing a number before both is how a product acquires an obligation nobody agreed to.

Performance

OPERATION	PROPOSED TARGET	RATIONALE
Dues list for ~100 residents	< 1.5 s	The owner's most frequent action; slower than the register defeats the product
Resident statement	< 1.5 s	Opened during a conversation with a resident
Record a payment	< 1 s	Done repeatedly; latency compounds into a workflow problem
Floor plan initial render	< 2 s on a mid-range phone	A prospective resident on mobile data abandons slowly-loading pages
Bed allocation commit	< 1 s	Includes the re-check and the lock
Nightly billing run	< 5 min for 100 beds	Must complete well before staff arrive

Measured at the 95th percentile on a representative dataset, not a developer laptop with ten records. Scale assumption: ~100 beds per property, tens of properties. **This is a small-data product** — correctness, not throughput, is the engineering challenge.

Reliability and availability

PROPERTY	POSITION
Availability target	OPEN — depends on hosting. A hostel office works business hours; continuous availability may not be the right expenditure.
Planned maintenance	Outside operating hours, with notice. PROPOSED
Degradation	An external provider outage degrades its feature only. Verification unavailable means staff verify manually; it never stops admissions.
Data durability	No acknowledged financial write may be lost. Non-negotiable.

Recovery

RPO and RTO are OPEN. They follow from two inputs that do not yet exist: how much data loss the owner can tolerate, and what the chosen host can actually deliver.

What *is* fixed is the standard of evidence. **Recovery readiness is claimed only from a completed isolated restore** whose record contains the backup identifier and timestamp, the restore outcome, the schema and application version, reconciliation totals, the named verifier, elapsed time and any limitations. A dashboard reporting a successful backup is not evidence.

Scalability

One deployable process, vertically scaled, with the database as the constraint. Adequate well beyond the pilot. The architecture permits extraction of a module later; nothing requires it. Premature horizontal scaling would add distributed-consistency problems to a product whose hard requirement is transactional correctness.

Accessibility

WCAG 2.1 AA PROPOSED, with specific obligations: keyboard operation of every interactive element including bed selection; a non-visual equivalent for the spatial map shipped in the same release; status never carried by colour alone; 44 px minimum touch targets; and `prefers-reduced-motion` respected.

Privacy and security

Data minimisation as a standing constraint; permission separation between profile, financial and document access; organisation isolation verified by an automated suite on every commit; and no personal data in third-party monitoring without a reviewed configuration. Retention periods are OPEN.

Maintainability

REQUIREMENT	TARGET	STATUS
Clean-clone setup by a new developer	Under 30 minutes	UNVERIFIED — second-developer reproduction not performed
Formatting, lint, typecheck, build	All pass in CI	PASS LOCALLY
Pinned toolchain versions	Node and npm pinned	VERIFIED
No secret in repository	Scanner passes	VERIFIED
Business logic outside controllers and components	Enforced by review	PROPOSED

Observability

Structured logs with correlation identifiers matching audit events; error reporting; health endpoints exposing status without connection details; alerts on repeated authentication failures, abnormal permission denials, failed backups, failed restores, import anomalies and unexpected financial operation rates. PROVIDER UNDECIDED.

Summary of what is actually committed

Nothing. Every figure in this chapter is a proposal. The only non-negotiable properties — which are correctness requirements rather than service levels — are that no acknowledged financial write is lost, that cross-organisation access is impossible, and that recovery is claimed only from a tested restore.

CHAPTER 20

Testing and Quality Assurance

What must be tested, in what order of priority, and the classification rule that stops the test suite from quietly inventing the business rules nobody has decided.

The classification rule

Every test is one of two kinds, and the distinction decides whether it may exist before the owner has decided anything.

CLASS	DEFINITION	MAY BE WRITTEN NOW?
(a) Architecture invariant	Holds regardless of which commercial policy the property chooses. No duplicate on retry; no over-allocation; traceable corrections; no cross-tenant access; atomic commit; exact money arithmetic.	**Yes.** Required, and a release gate.
(b) Owner-policy example	Expected output depends on a rule the owner chooses — a prorated first charge, an allocation order, a late fee, a deposit net-off, a refund deadline.	**No.** The scenario may be listed; no expected value may be asserted.

WHY A CLASS (B) TEST WITH A GUESSED EXPECTATION IS WORSE THAN NO TEST

A test suite is a specification. Writing one for an undecided rule *decides the rule by default* — and makes the guess look reviewed, because it now appears in a passing suite with a green tick beside it. The next developer treats it as settled, and the owner is never asked.

Consequence: an unsigned class (b) test is reported as **pending**, never as passing or failing. A suite summary that counts pending policy tests as green misrepresents readiness.

Priority order

1. Financial correctness and transaction integrity
2. Cross-organisation isolation and permissions
3. Bed and resident lifecycle consistency
4. Migration and reconciliation
5. Backup and recovery
6. Workflow usability and responsive behaviour
7. Ordinary component behaviour and edge-case validation

Test layers

LAYER	COVERS	NON-NEGOTIABLE
Unit	Money arithmetic, date logic, allocation algorithm, permission mapping	Property-based tests on the money type; an injectable clock so billing can be driven across awkward dates

LAYER	COVERS	NON-NEGOTIABLE
Integration	Transactions, constraints, concurrency, idempotency	**Real PostgreSQL** at the deployed major version. A mock cannot demonstrate that a constraint prevents a double allocation.
API contract	Every route against its schema and the OpenAPI document	Contract drift fails the build
Cross-tenant	Every registered route as Org B against Org A's identifiers	**Enumerated from the router automatically**, so a new route cannot be forgotten. One leak fails the build.
Permission matrix	Every role against every sensitive action	Table-driven. Adding a role or action without a test row fails the build.
Browser	The workflows staff perform daily	Record a payment, allocate a bed, check out, read the dues list, print a receipt
Accessibility	Keyboard, screen reader, contrast, targets	Bed selection without a pointer is a pass condition, not a nice-to-have
Recovery	Backup → isolated restore → reconcile	Dated, recorded, performed by a human

Financial scenario suite

Twenty scenarios, classified. Full expected-result detail belongs in **Appendix D — Acceptance Criteria** once the owner supplies the policy answers.

ID	SCENARIO	CLASS	BLOCKED BY
FIN-001	Approved charge is created	(b)	BR-REN-001/002
FIN-002	Billing request retried → no duplicate	(a)	—
FIN-003	Partial payment settles only what it covers	(a)	—
FIN-004	One payment across multiple charges	(a)	—
FIN-005	Several payments settle one charge	(a)	—
FIN-006	Unallocated receipt recorded	(b)	BR-ALLOC-004
FIN-007	Payment pending verification	(b)	BR-PAY-003
FIN-008	Duplicate submit after timeout → one transaction	(a)	—
FIN-009	Concurrent allocations of the same amount	(a)	—
FIN-010	Reversal keeps the original traceable	(a)	BR-PAY-009 for authority
FIN-011	Adjustment records reason and actor	(a)/(b)	BR-REN-011 for approval
FIN-012	Deposit receipt moves no rent figure	(a)	—
FIN-013	Refund above available balance refused	(a)	—
FIN-014	**Checkout changes no balance; no net-off**	(a)	BR-DEP-006 for any approved net-off
FIN-015	Statement and aggregate reconcile	(a)/(b)	BR-REP-001

ID	SCENARIO	CLASS	BLOCKED BY
FIN-016	Terms change does not rewrite an issued charge	(a)	—
FIN-017	Payment with unknown reference	(b)	BR-PAY-006
FIN-018	Duplicate real-world reference	(b)	BR-PAY-005
FIN-019	Allocation with no target and no ordering policy → refused	(a)	—
FIN-020	Any blocked operation returns RULE_NOT_CONFIGURED and writes nothing	(a)	—

Eleven are class (a) and enforceable now. **Seven are class (b)** and pending. Two are mixed. A suite summary must report those three counts separately rather than as one number.

Concurrency tests

Independent connections and coordinated parallel requests — not sequential unit tests pretending to be concurrent. Two staff allocating the last amount of one payment; two staff assigning one bed; two billing jobs on one event; two import commits on one batch; a terms change racing a charge generation; a correction racing an allocation.

Spatial map tests

2D: concurrent allocation; allocation against stale availability; selection creating no server state; occupied-bed deletion refused; overlapping geometry refused on publish; historical layout resolution; keyboard-only selection; screen-reader announcement; no resident data in any public response.

3D, when built: identical availability semantics to 2D verified against the same fixtures — the 3D view must not be able to develop its own notion of what is free; plus a performance budget on a mid-range device.

Migration and recovery

Migration proceeds **synthetic → owner-approved redacted → real**, and a stage is never skipped. Import mechanics and idempotence are proven on synthetic fixtures; counts and totals reconcile on redacted data; real cutover happens only after the reconciliation report is signed.

Recovery: backup, restore into an isolated environment that cannot write to production, verify migrations applied, compare counts and financial totals to pre-restore values, confirm audit events survived, and record the full evidence set.

Defect severity

SEVERITY	DEFINITION	RELEASE EFFECT
Critical	Cross-organisation exposure; corrupt or unrecoverable financial data; authentication bypass	Blocks go-live
High	Wrong charge, payment or deposit outcome; duplicate financial record; bed conflict; failed recovery; **a blocked operation executing on an unapproved default**	Blocks go-live
Medium	Significant workflow issue with a safe workaround and no integrity impact	Owner accepts with documented workaround

SEVERITY	DEFINITION	RELEASE EFFECT
Low	Cosmetic or minor usability	Backlog

No critical or high defect involving security or financial integrity may be waived to meet a schedule.

Current test reality

Five tests exist and pass. They cover environment configuration parsing and brand-token invariants. They are genuine class (a) tests of what exists; they test no domain behaviour, because none exists. Integration tests have never run against a database, and no concurrency, isolation, financial or recovery test exists.

CHAPTER 21

Deployment and Operations

How a developer runs Nook today, what CI does, and the gap between the current engineering environment and anything that could carry a real hostel's records.

Developer setup — as documented in the repository

1. Install Node 24.22.1 and npm 11.6.2.
2. Install locked dependencies with `npm ci`; review lockfile changes before committing.
3. Start PostgreSQL with `docker compose up -d postgres` and wait for health.
4. Run the API and the web shell in separate terminals.
5. Run tests, typecheck, lint, format check, build, the scope verifier, the secret scanner and the audit.
6. Validate the schema with a placeholder `DATABASE_URL` supplied inline — no `.env` file is created.

Web on `:5173`, API on `:3000`. Health at `/api/v1/health` and `/api/v1/health/database`; database health reports a generic status without connection details.

Environment and secrets

RULE	DETAIL	STATUS
No real credential committed	`.env.example` holds placeholders only	SCANNER PASSES
Client variables are public	A build-time variable exposed to the bundle may never hold a secret	ENFORCED BY CONVENTION
Config validated at startup	Missing or invalid required values fail loudly rather than starting broken	TESTED
Per-environment credentials	Local, CI, staging and production each separate and least-privileged	NO DEPLOYED ENVIRONMENT EXISTS
Committed secret procedure	Revoke, rotate, assess exposure. Deleting it in a later commit is insufficient.	DOCUMENTED

Continuous integration

A workflow exists and expects a committed lockfile and `npm ci`. It is configured to run formatting, lint, typecheck, unit tests, build, migration-against-empty-database, dependency review and secret scanning, using synthetic values only.

> **CI HAS NEVER EXECUTED**
>
> There is **no Git metadata** in the working folder — `git status` exits 128. The workflow file is present but has never run, no required-check configuration has been exercised, and no change has been through review. Before anything can be reviewed or merged, this folder needs to be placed in its intended repository. NOT IMPLEMENTED

Database migration policy

- Versioned migrations reviewed in pull requests.
- Applied to a clean database in CI to prove the history is reproducible.
- Executed once in production by a controlled release job with least-privilege credentials.
- **Production `db push` and manual schema editing are prohibited.**
- Pre-change backup where risk warrants, with a forward-fix or restore plan.
- No domain schema until the conceptual model and the relevant business rules are approved.

Deployment candidates

OPEN. No provider, region or deployment target has been selected, and nothing has been deployed. Requirements when it is chosen: managed PostgreSQL with automated backups and point-in-time recovery; application and database environments separated; a secret manager; and a region decision taken with the data-residency question in Chapter 18.

Hosting choice determines backup retention, RPO and RTO, which is why those remain open in Chapter 19 rather than being stated optimistically.

Backup, restore and rollback

Automated backups; a restore into an isolated environment that cannot write to production; verification of counts, financial totals and audit events against pre-restore values; and a full evidence record. Rollback requires a documented trigger, a named decision-maker, and — critically — a way to return to the paper process and reconcile anything entered after cutover.

No backup has been configured and no restore has been performed. NOT IMPLEMENTED

Current engineering limitations

Stated separately from the target architecture, because they are facts about the environment rather than design positions.

LIMITATION	EVIDENCE	EFFECT
No running database	Docker daemon socket absent; `pg_isready` exits 2	No SQL, no database health, no integration test
API cannot start	Binding the listener is prohibited in this environment	Health endpoints never exercised; compilation is not runtime verification
No registry access	Package registry DNS resolution fails	Security advisory unresolved; audit cannot complete
No Git metadata	`git status` exits 128	No history, no diff, no review, no CI run
Browser runtime unverified	Not performed	The web shell builds but has not been exercised in a browser

> **THE DISTINCTION THAT MATTERS**
>
> These are limitations of the *environment the work was done in*, not defects in the design. Every one is resolved by provisioning a normal development machine with Docker, registry access and a Git remote. They are listed because Milestone 0 cannot be accepted until they are, and because "it builds" has repeatedly been mistaken for "it works".

Production release procedure

Proposed, and not yet exercisable: merge through review with required checks green; tag; run migrations as a controlled job; deploy; verify health and a representative read; confirm monitoring and backups; and keep the rollback path documented and tested. No production deployment occurs during Milestone 0.

PART VI

Delivery

The sequence of work, an honest account of what exists today, and every decision still waiting on someone.

22 Delivery Roadmap
23 Current Implementation Status
24 Risks, Assumptions and Open Decisions
25 Glossary, References and Appendices



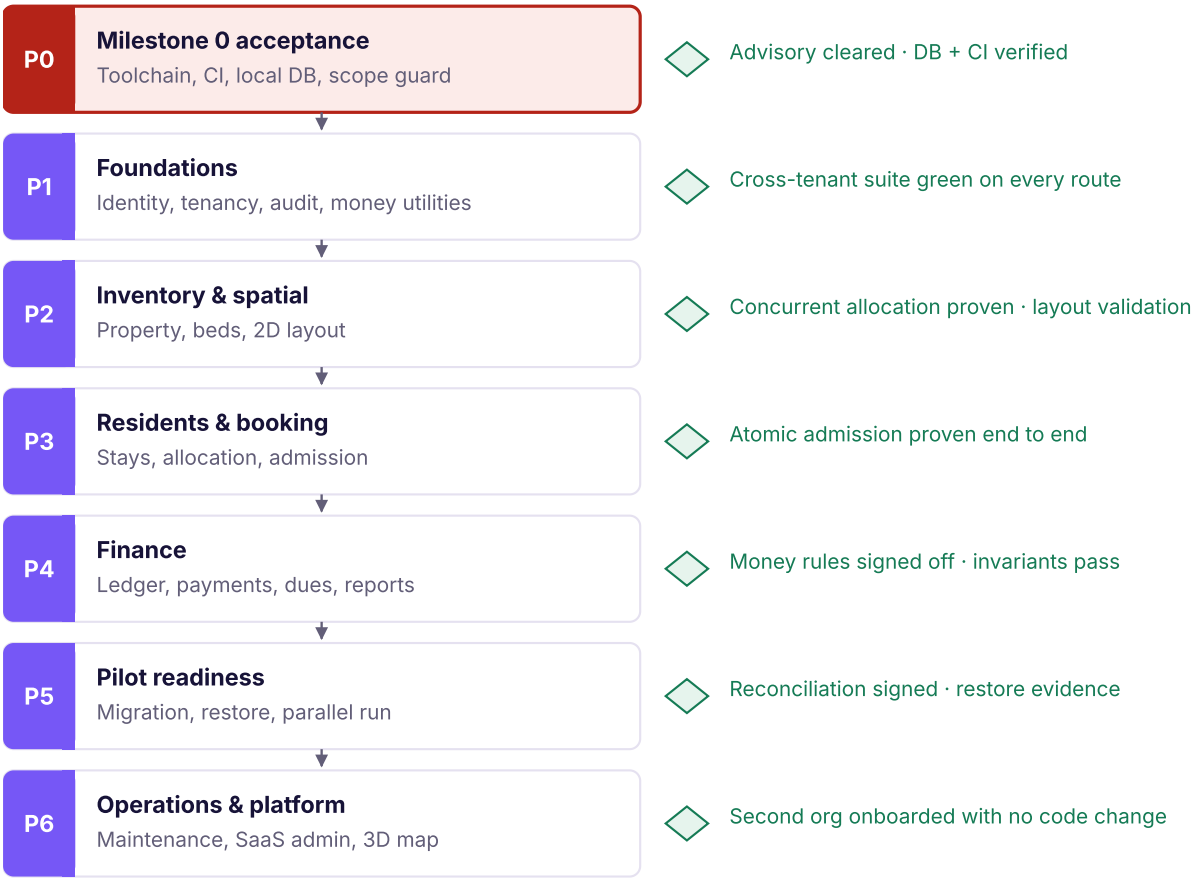
CHAPTER 22

Delivery Roadmap

Seven phases, what each delivers, what it depends on, and the evidence that opens the gate to the next. No dates — a gate opens when its evidence exists.

Delivery phases and their gates

Sequence and dependency only. A gate opens when its evidence exists, not on a date.



P0 is blocked, and everything downstream waits on it
Unresolved advisory, no database connection, an API that has never started, CI that has never run.

Figure 22.1 — Delivery phases and gates. Strictly sequential, because each phase depends on the correctness of the one before. The first phase is currently blocked.

NO DELIVERY DATES ARE GIVEN, DELIBERATELY

Three of the inputs a credible estimate needs do not exist: the business rules are unanswered, the architecture is unapproved, and the team's real availability has not been established. A date derived from effort hours and presented as a commitment is the most common way these arrangements end badly.

What is given instead is the **order**, the **dependencies** and the **exit evidence**. Those are the parts that are actually knowable now.

Phase detail

PHASE	DELIVERABLES	PREREQUISITES	EXIT GATE — EVIDENCE REQUIRED
P0 BLOCKED	Milestone 0 acceptance: toolchain, CI, local PostgreSQL, scope guard, secret scanning	An environment with Docker, registry access and a Git remote	Patched dependency installed and verified; live audit passes; database health endpoint returns; API starts and serves; CI runs green on a clean clone
P1	Identity, sessions, memberships, permissions, tenant context, audit contract, money utilities, idempotency infrastructure	P0 accepted; ADR-001 to ADR-011 approved	Cross-tenant suite enumerated from the router and green on every route; permission matrix table-driven and complete; money round-trips across the supported range
P2	Property, buildings, floors, rooms, beds; layout versions; 2D floor plan; accessible list; availability service	P1; conceptual data model approved; BR-LIF-002, 004, 006 answered	Concurrent allocation yields exactly one success under real parallel load; overlapping geometry refused; occupied-bed deletion refused; keyboard-only selection works
P3	Residents, stays, lifecycle, admission, atomic allocation, room change, operational checkout	P2; BR-LIF-001, 005, 007, 010, 011 answered	Admission atomic end to end; repeat admission retains history; checkout changes no balance; date semantics agreed and tested at month boundaries
P4	Rent schedules, charges, payments, allocations, deposits, statements, dues and collection reports	P3; **money rules signed off**; report definitions agreed	Eleven class (a) invariants pass; signed-off class (b) scenarios pass; every unsigned rule's operation demonstrably refuses; a statement reconciles to the paisa
P5	Migration tooling, reconciliation, backup and restore, monitoring, staff guide, parallel run	P4; opening-balance rules agreed; hosting selected	Restore evidence record complete; reconciliation signed by the owner; parallel run with no unexplained discrepancy; owner accepts
P6	Maintenance and complaints; enquiry and verification; notifications; SaaS admin; subscriptions; 3D map	Pilot live and stable	A second organisation onboarded end to end **with no code change**

Why the order is not negotiable

Each phase depends on the correctness of the one before, not merely its existence. A rent charge against a wrong stay is a wrong number. A stay against wrong inventory is a resident in a bed that does not exist. Isolation retrofitted after P4 means migrating live financial data.

The one genuine sequencing choice is **where the 2D spatial map sits**. It is placed in P2 because it is the product's differentiator and because the layout model is cheaper to build alongside inventory than

bolted on afterwards. It could be deferred to P6 to reach a working ledger sooner. That is a real trade-off and an open decision (Chapter 24) — the only phase boundary genuinely available to move.

What can be built before the business rules are answered

A useful amount, which is why P1 is not blocked by the owner workshop:

- Identity, sessions, memberships, permissions, tenant context
- Audit event contract and correlation identifiers
- Exact-money parsing, serialisation and range validation
- Idempotency storage and request fingerprinting
- Organisation-scoped repository patterns and the cross-tenant suite
- Inventory structure and the spatial layout model
- Charge and payment *record structures* — without any calculation
- Manual charge entry with a user-supplied amount
- Import preview mechanics against synthetic fixtures

What cannot: rent calculation, billing runs, allocation ordering, late fees, deposit deductions, refunds, settlement, and any authoritative report total.

The critical path

1. **Provision a working environment.** Docker, registry access, Git remote. Unblocks P0 and costs nothing but access.
2. **Hold the owner workshop.** Seventy-five rules; the money ones gate P4. This can run in parallel with P1 and should start immediately.
3. **Record the architecture decisions.** Seventeen rows, currently blank. Gates P1.
4. **Decide the spatial scope question.** Whether the 2D map is pilot scope changes P2 materially.

The first three are independent and can proceed simultaneously. None requires writing code, and all three are currently stalled on decisions rather than on effort.

CHAPTER 23

Current Implementation Status

A factual inventory of the repository as inspected on 28 October 2026. Nothing in this chapter is a plan; everything is either present or absent, and every claim traces to a file or a recorded command result.

THE HEADLINE

The repository contains a **Milestone 0 engineering scaffold**. It is not a working hostel management product and contains no resident, booking, rent, payment, deposit or settlement logic. Milestone 0 itself is **not yet accepted**.

What exists — file inventory

PATH	CONTENTS	SIZE
`apps/api/src/`	5 files: `main.ts`, `app.module.ts`, `config.ts`, `health.controller.ts`, `database/database.service.ts`	~4 KB
`apps/api/test/`	1 file: configuration parsing tests	1 KB
`apps/web/src/`	`App.tsx`, `main.tsx`, `propertyDemo.ts`, `designTokens.ts`, `tokens.css`, `styles.css`, 2 test files	~49 KB
`prisma/schema.prisma`	Generator and datasource only. **No models.**	262 B
`scripts/`	`secret_scan.py`, `verify_m0_scope.py`	~4 KB
`docs/`	`MILESTONE_0_SCOPE.md`, `MILESTONE_0_VALIDATION.md`	~9 KB
`.github/workflows/ci.yml`	CI workflow definition	1 KB
`docker-compose.yml`	PostgreSQL 17 service	452 B

Excluding dependencies, the entire application source is **under 60 KB across roughly fifteen files**, of which two — `App.tsx` and `styles.css` — account for most of the volume and are the demonstration UI.

Implemented functionality

COMPONENT	WHAT IT ACTUALLY DOES	STATUS
API shell	NestJS application with two health endpoints and a database service wrapper	IMPLEMENTED
Configuration parsing	Reads and validates environment values; fails clearly on missing or invalid required values	VERIFIED
Web shell	React 19 + Vite application that builds and renders	IMPLEMENTED
Design tokens	38 tokens, light and dark, matching the approved Option A palette; asserted by test	VERIFIED

COMPONENT	WHAT IT ACTUALLY DOES	STATUS
Demonstration property UI	Renders a fictional four-floor property with rooms and beds, status filters, search, summary counts and empty states — from **static fixtures**. Read-only. No persistence, no API call, no positions, no layout.	IMPLEMENTED
Scope verifier	Static guardrail asserting Milestone 0 boundaries; 22 of 22 checks pass	VERIFIED
Secret scanner	Baseline pattern scan across the repository	VERIFIED

> **THE DEMONSTRATION UI IS NOT MODULE 4, 5 OR 7**
>
> It renders fictional fixtures with hard-coded bed states. There is no spatial layout, no bed position, no availability service, no persistence and no backend call. It demonstrates that the token system works and that a few UI states have been thought about. It is a prototype, and treating it as partial inventory management would overstate progress considerably.

Executed checks and their results

Run independently from the repository root with Node 24.22.1 and npm 11.6.2.

COMMAND	EXIT	RESULT
`npm test`	**0**	All 5 tests pass across API and web
`npm run typecheck`	**0**	Both workspaces pass
`npm run build`	**0**	API and web production builds succeed
`npm run lint`	**0**	Pass
`npm run format:check`	**0**	Pass
`npm run prisma:validate`	**0**	Empty schema validates; no database connection required
`npm run verify:secrets`	**0**	Baseline scanner passes — not a comprehensive security review
`npm run verify:m0-scope`	**0**	22 of 22 checks pass; no `.env` present
`npm audit`	**1**	**Fails.** Advisory endpoint unreachable; security blocker remains

What the five passing tests cover

Worth stating precisely, because "all tests pass" invites a wrong inference. They cover environment configuration parsing and brand-token invariants. **They test no domain behaviour**, because none exists. They are genuine tests of what is there, and what is there is a scaffold.

Untested runtime and infrastructure

AREA	EVIDENCE	CONSEQUENCE
PostgreSQL	Docker Compose installed, but `docker info` exits 1 — the daemon socket is absent. `pg_isready` exits 2.	No SQL connection has ever been made
API runtime	Startup attempted; exited 1 because binding the listener is prohibited in this environment	Health endpoints never exercised
Database health endpoint	Depends on both of the above	Never returned a response
Browser runtime	Not performed	The web shell builds; it has not been verified in a browser
CI	No `.git` directory; `git status` exits 128	Workflow has never run
Clean `npm ci`	Not verified	Reproducibility from a clean clone unproven

Compilation is not runtime verification. The API builds and the web app builds; neither has been observed to work.

Security findings

One unresolved advisory, diagnosed precisely rather than left vague. A file-upload dependency reached transitively through the NestJS platform adapter remains at an affected version. The patched release is identified, is compatible, and is **not installed**.

The cause: the root override reaches the workspace link nodes, but the target workspace nodes carry no override rules, so the adapter's dependency edge retains its original specifier and is considered valid. Changing between scoped and global override syntax does not repair this, and a lockfile-only update never touches the installed tree. Resolution needs registry access.

The audit exits 1 because its live advisory endpoint is unreachable. **This is not a clean audit**, and an offline install reporting zero findings is not accepted as evidence. No suppression, forced fix, prerelease install or manual lockfile edit was used.

Database state

`prisma/schema.prisma` declares a generator and a PostgreSQL datasource and **no models**. There are no migrations. This is deliberate and correct: domain entities may not be added until the conceptual model and the relevant business rules are approved. No schema was changed in producing this handbook.

Git status

There is no `.git` directory in the working folder. No history, no diff, no branch and no remote. Nothing can be reviewed or committed through Git until the folder is placed in its intended repository — which

also blocks CI, since the workflow has nothing to run against.

Known blockers to Milestone 0 acceptance

1. **Dependency security.** Install and verify the patched version in an environment with registry access; confirm both locked and installed versions; confirm the override applies across the workspace graph; obtain a successful live audit.
2. **PostgreSQL.** Start the service and demonstrate a successful connection and database health response.
3. **API runtime.** Start the process and exercise both health endpoints.
4. **CI.** Restore Git metadata, place the folder in its repository, and run the workflow to completion on a clean clone.
5. **Reproducibility.** A second developer sets up from a clean clone with no undocumented local state.

What is not implemented

Everything else. Specifically: authentication, sessions, users, memberships, roles, permissions, organisations, properties, buildings, floors, rooms, beds, spatial layouts, availability, enquiries, phone verification, booking, allocation, residents, stays, lifecycle, documents, rent schedules, charges, payments, allocations, deposits, refunds, checkout, settlement, reports, notifications, maintenance, complaints, audit events, migration tooling, subscriptions and platform administration.

> **AN ASSESSMENT, OFFERED PLAINLY**
>
> The scaffold is small, and that is the right outcome. It is correctly scoped, honestly documented, and has not quietly grown into feature work — the empty schema and the scope verifier are evidence of deliberate restraint rather than of slow progress.
>
> The blockers are environmental, not architectural. Every one is resolved by a development machine with Docker, registry access and a Git remote. The project is not behind because of engineering; it is waiting on decisions and on access.

CHAPTER 24

Risks, Assumptions and Open Decisions

Every unresolved item, who owns it, what it blocks and what the next action is.
This is the chapter to act on.

Decision register

Each row names the owner, because an open decision without one stays open. Impact is the cost of leaving it unresolved, not the difficulty of resolving it.

DECISION	STATUS	IMPACT	BLOCKS	NEXT ACTION
Booking and occupancy policy				
Bed holds — do they exist, how long, is money taken? BR-LIF-003	OPEN	High	Enquiry expiry, selection semantics, occupancy economics	Owner workshop
Instant allotment vs manual approval	MANUAL	High	Nothing — confirmed. Must not be changed without owner approval.	None
Date interval semantics — inclusive ends, timezone BR-REN-002, BR-LIF-004	OPEN	High	The assignment overlap constraint cannot be written	Owner workshop + engineering
Cancellation, expiry, no-show BR-LIF-012	OPEN	Medium	Those transitions; any associated charge	Owner workshop
Bed release date on checkout BR-LIF-004, 011	OPEN	High	Automated bed release; re-letting a vacated bed	Owner workshop
Finance and deposits — the largest cluster				
Due date, billing period, proration, first charge, rent stop, calendar edges BR-REN-001…006	OPEN	**Critical**	All automated rent generation	**Owner workshop — highest priority**
Grace period and late fees BR-REN-008	OPEN	High	Overdue status with penalties; reminder timing	Owner workshop
Payment status and confirmation authority BR-PAY-002, 003	OPEN	High	Confirmed-collection figures; the status model itself	Owner workshop
Allocation ordering BR-ALLOC-002	OPEN	High	Automatic allocation. **No oldest-first default.**	Owner workshop
Advance credit and unmatched receipts BR-ALLOC-003, 004	OPEN	Medium	Overpayment handling	Owner workshop
Deposit amount, deductions, offset, refund deadline, approvals BR-DEP-001…010	OPEN	**Critical**	All deductions, refunds and checkout settlement	Owner workshop
Report definitions BR-REP-001…004	OPEN	High	Authoritative totals on every report	Owner workshop

DECISION	STATUS	IMPACT	BLOCKS	NEXT ACTION
Tax treatment BR-REN-012	EXTERNAL	Medium	Tax lines; invoice structure. Retrofitting is painful.	Hostel's accountant
Data, access and retention				
Mandatory resident fields BR-LIF-008	OPEN	Medium	Forms, migration mapping	Owner workshop
Whether documents are collected at all BR-SEC-004	DEFAULT NO	High	Upload, storage, retention, maintenance photos	Owner decision — "no" costs nothing reversible
Identity verification lawfulness BR-SEC-005	EXTERNAL	High	Any verification feature	Qualified counsel
Retention and deletion BR-OPS-003, 004	EXTERNAL	High	Retention jobs; deletion-request handling	Qualified counsel
Role-to-permission grants BR-SEC-001…003	OPEN	High	Staff provisioning; go-live	Owner workshop
Product scope				
Is the 2D spatial map pilot scope?	OPEN	**Critical**	Phase P2 content; the pilot's size and timeline	**Project owner — see below**
Public discovery, enquiry, phone verification in pilot?	OPEN	High	Phase scope; a public attack surface; messaging cost	Project owner
Resident portal	DEFERRED	Low	Nothing in the pilot	Revisit after pilot
Architecture				
Seventeen ADRs, approval record blank	PROPOSED	**Critical**	Phase P1 and all implementation	Project owner — record a decision per row
Row-level security as backstop ADR-007	OPEN	Medium	Nothing immediately; harder to add later	Engineering, with a tested decision
Session expiry, MFA, recovery channel	OPEN	High	Exposing any shared environment	Engineering + owner
3D rendering library	CANDIDATE	Low	Nothing now	Re-measure at the 3D phase
Operations, commercial and brand				
Dependency security advisory	BLOCKED	**Critical**	Milestone 0 acceptance, hence everything	Provision registry access, install, verify, re-audit
Database, API runtime, CI never exercised	BLOCKED	**Critical**	Milestone 0 acceptance	Provision Docker and a Git remote
Hosting, region, RPO and RTO	OPEN	High	Deployment; any availability statement	Owner tolerance, then provider selection
Pricing, billable-bed definition, suspension policy	OPEN	Medium	Subscription billing — not pilot scope	Build a cost model before quoting anything
Trademark clearance and naming conflict	BLOCKED	High	Public launch, domain commitment, Option A/B palette	Legal review

DECISION	STATUS	IMPACT	BLOCKS	NEXT ACTION
Option A vs Option B palette	A APPROVED	Low	Nothing — A is in use	Decide with the naming conflict, not separately

The scope decision that deserves a conversation

THE SPATIAL MAP IS THE BEST IDEA HERE AND THE BIGGEST UNAPPROVED ADDITION

The 2D floor plan, public discovery, enquiry and phone verification are not in the approved requirements package. They arrived with this handbook. The map is genuinely what makes Nook more interesting than its competitors — and it also expands the pilot by a module group, adds a public attack surface, adds a paid messaging dependency, and introduces an accessibility obligation that must be met rather than deferred.

Two defensible positions. **Include it**: the differentiator ships with version one and the layout model is cheaper to build alongside inventory than retrofitted. **Defer it**: reach a trustworthy ledger sooner, since that is what the first customer is actually paying for, and add the map once the money is right.

What should not happen is the decision being taken by default, by it simply appearing in a handbook.

Assumptions worth stating

1. The pilot property's inventory maps cleanly onto property → building → floor → room → bed. **Not surveyed.** If entire rooms are let as units, the allocatable-unit model changes.
2. The owner will accept a staff-facing web application with no resident app in the pilot.
3. Historical data exists in a form that can be migrated. It may be registers that cannot be reliably digitised, which would change migration from "import" to "opening balances only".
4. Both partners will agree one set of financial rules. If they do not, that is a business problem to surface at the workshop, not a configuration feature.
5. There is no existing software or spreadsheet the hostel already depends on which must be replaced or read from.
6. ~100 beds and tens of properties means a single modest database is ample for years.

Risks that are not decisions

RISK	WHY IT MATTERS	MITIGATION
The owner quietly keeps the register	The most likely failure and rarely on a risk list. Happens when the software is 90% right: one number looks wrong, one task takes six taps instead of one, and within three weeks there are two sources of truth — and paper wins.	Supervised pilot in the office. Every observed workaround treated as a defect, not user error. Go-live defined as the day Nook answers "who owes money?" faster than the register.

RISK	WHY IT MATTERS	MITIGATION
One developer, a student, holding a business's money records	If unavailable during exams, placements or illness, the hostel has no fallback. This is a commitment problem, not a code problem.	A written support window that is not 24/7; a runbook a competent stranger could follow; backups the owner can trigger; and an **owner-controlled data export** — the thing that makes the arrangement fair.
Scope growth through plausibility	Every module in Chapter 4 is reasonable on its own. Collectively they are years of work, and the ledger is what is being paid for.	Phase gates with evidence; the explicit exclusion list; and treating an unapproved rule as a blocker rather than a prompt to guess.
A guessed business rule becoming permanent	A default chosen for pilot convenience becomes the billing rule for every later customer, invisibly.	The policy guardrail: blocked operations refuse explicitly; class (b) tests cannot assert an unsigned expectation.
Public surface leaking resident data	A public map over a resident database is the highest-risk thing in the product.	A separate projection, never a filtered internal record; an automated test asserting the exposed field set.

CHAPTER 25

Glossary, References and Appendices

Shared vocabulary, the source material this handbook was built from, and what the supporting appendices contain.

Glossary

TERM	MEANING IN THIS HANDBOOK
Allocatable unit	The thing a resident occupies and availability is computed for. Modelled as a bed, including for single-occupancy rooms.
Allocation	Two distinct meanings, always qualified. *Bed allocation*: assigning a bed to a stay. *Payment allocation*: linking a payment to a charge for an amount.
Adjustment	A signed financial correction with a reason and an approver. Never an edit to a posted record.
amountMinor	A money value as a base-10 integer string of paise, carried with an explicit currency.
Assignment	An effective-dated record linking one bed to one stay over a date interval. Append-only.
Charge	A claim on a resident for a billing period. Immutable once posted.
Displayed availability	A derived, cacheable status computed when a view rendered. Advisory, never a reservation.
Financial settlement	Calculating a departing resident's final position. Separate from operational checkout, with its own permission and approval.
Idempotency key	A client-supplied operation identifier ensuring a retry returns the original result rather than creating a second record.
Milestone 0	The engineering foundation — toolchain, CI, local database, scope guard. Implements no product capability.
Operational checkout	Recording a confirmed departure. Changes no balance, nets no deposit, releases no bed.
Organisation	One independent customer business. The absolute tenancy boundary. **Never used for a person.**
P0 / Pilot	*P0* is a scope label on product capability. *Pilot* is the controlled first-hostel launch. Neither is Milestone 0.
Permission identifier	A named capability such as `payment.record`. The server checks these, never role names.
Projection	A separate, restricted representation served to a public surface. Never a filtered internal record.
Resident	A person living at a property. Distinct from a stay.
Reversal	A record negating a posted record. The original remains visible.
RULE_NOT_CONFIGURED	The API response when an operation depends on an unapproved business rule. Names the rule, writes nothing, applies no default.
Spatial layout version	Effective-dated geometry for a floor — room outlines and bed positions. Published, not edited live.
Stay	One period during which a resident occupied a bed at a property.
Tenant context	The server-constructed object carrying user, organisation, role, properties and permissions through every service call.

Identifier families

PREFIX	MEANING	CANONICAL HOME
`BR-*`	Business rules requiring owner decision	Business Rules Register
`ADR-001…017`	Architecture decisions	Architecture Approval Register
`RG-0…RG-5`	Product release gates	Requirements package, PRD §9
`M0-*`	Milestone 0 tasks, gates and evidence	Milestone 0 Preparation Plan
`FIN-001…020`	Financial test scenarios	Chapter 20 and Appendix D
`FR-*`, `NFR-*`	Functional and non-functional requirements	Requirements package, PRD

Source material

Every factual claim in this handbook traces to one of these. Nothing was taken from memory where a source could be inspected.

SOURCE	INSPECTED	USED FOR
`/Users/atharva/Downloads/nook`	28 Oct 2026	Chapter 23 in full; toolchain versions; design tokens; demo UI behaviour
`docs/MILESTONE_0_VALIDATION.md`	28 Oct 2026	Command exit codes, the dependency diagnosis, acceptance blockers
`docs/MILESTONE_0_SCOPE.md`	28 Oct 2026	Scope inclusions and exclusions
`nook-brand-assets.zip`	28 Oct 2026	All logo assets used here; palette; clear space; minimum sizes; incorrect use
`nook-brand-guidelines.md`	28 Oct 2026	Construction geometry, contrast measurements, open brand items
`nook-complete.zip`	28 Oct 2026	Token provenance, Option A/B decision, competitive colour research
Requirements package v0.2, 16 documents	27 Oct 2026	Business rules, ADRs, permission vocabulary, test classification, blocker register

External reference consulted

3D rendering comparison (Chapter 7): PkgPulse — three.js vs React Three Fiber vs Babylon.js, 2026. The source contradicts itself on bundle sizes; figures are treated as indicative and flagged as such in the text. No library has been selected or added as a dependency.

SOURCES DELIBERATELY NOT USED

`nook-rejected-explorations.png` contains design directions that were explicitly rejected. No image, mark or treatment from that file appears anywhere in this publication.

No legal, tax or regulatory position was taken from a commercial summary. Published summaries of the applicable data-protection, tax and verification regimes disagree with one another on detail, and a product decision taken from one would be a decision taken from a guess. Every such question is marked for professional confirmation instead.

Brand assets used in this publication

All assets are the genuine approved masters, verified against the package manifest by SHA-256 before use. No substitute, redrawn or typed reproduction of the wordmark appears anywhere.

ASSET	CHECKSUM (FIRST 12)	USED FOR
`svg/nook-logo-white.svg`	`f8dbc18b1d1e`	Cover wordmark
`svg/nook-logo-primary.svg`	`f8e698498ce6`	Light-ground reference
`svg/nook-symbol.svg`	`124ec158c80c`	Page footer mark
`svg/nook-symbol-white.svg`	`66d69c402d85`	Part divider pages
`png/nook-brand-board.png`	`026576b47b91`	Figure 6.1

A further ten approved assets are included in the package's `assets/` directory so the publication can be regenerated without the original archives. Logo variants were selected by background per the guidelines: white on indigo, indigo on light.

Supporting appendices

Large reference matrices are kept out of the main handbook so it stays readable, and are published alongside it in the same visual style.

APPENDIX	TITLE	CONTENTS
A	Business Rules Register	All open business rules by family, with what each blocks and who decides
B	Data Dictionary	Entity attributes, types, constraints, scope columns and lifecycle
C	API Catalogue	Every proposed endpoint with permission, validation, behaviour and blocked status
D	Acceptance Criteria & Traceability	Requirement → acceptance criterion → test, with class (a)/(b) marking
E	Owner Decision Worksheet	The decisions from Chapter 24 formatted for a workshop. Blank by design.

The owner decision worksheet **supports** this handbook rather than replacing it. It is a working instrument; the handbook is the explanation.

Document control

Version	v1.0 — first issue
Date	28 October 2026
Status	Draft for review. No content has been approved.
Prepared for	Project-owner review
Regeneration	Sources, diagrams, assets and the build script are in the package. The PDF regenerates from them without the original archives.
Next review	After the owner workshop and the architecture approval record are completed — both of which change substantial parts of Chapters 11, 16 and 24.

CLOSING NOTE ON HOW TO USE THIS DOCUMENT

The four things that would move this project furthest, in order: provision a development environment with Docker, registry access and a Git remote; hold the owner workshop on the money rules; record a decision against each of the seventeen architecture rows; and settle whether the spatial map is pilot scope.

None of them requires writing code. All four are currently the critical path.

EOF echo ok; cd /home/claude/nook/handbook && python3 source/build/build.py 2>&1 | grep -vE "FutureWarning|warnings.warn" | tail -2